Effect Reflection

LEO WHITE, Jane Street, UK

Programming languages increasingly support algebraic effects and handlers for writing effectful code in a principled way by temporarily extending the language with additional effectful operations. However, integrating effects into type systems typically requires specialized effect systems that add complexity to the language.

We introduce *effect reflection*, an alternative presentation of algebraic effects and handlers that provides, for any algebraic signature Σ and type τ, an isomorphism:

$$W_\Sigma(\tau) \quad \cong \quad \text{⅄}(\Sigma) \to \Box\,\tau$$

where:

- $W_\Sigma(\tau)$ is the free algebra of Σ at τ, sometimes called the *freer monad* of Σ.
- $\text{⅄}(\Sigma)$ is a type representing how to inject operations of Σ into the current *ambient monad*: the monad describing which effects ordinary functions can perform.
- $\Box$ is a comonadic *global modality* used to track which values do not depend on the ambient monad.

This isomorphism emerges naturally, as a corollary of the Yoneda lemma, from the *Ambient Monad model*: a novel semantic model of algebraic effects in terms of presheaves over Lawvere theories.

Effect reflection provides all the expressiveness of typed algebraic effects without requiring language-level support for an effect system. We demonstrate this by implementing effect reflection as a library in OxCaml.

ACM Reference Format:
Leo White. 2026. Effect Reflection. 1, 1 (July 2026), 56 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 EFFECT REFLECTION AND THE AMBIENT MONAD MODEL

Algebraic effects and effect handlers have emerged as a powerful approach to structured programming with effects. However, integrating effects into type systems typically requires adding specialized effect systems. Unfortunately, such effect systems require extensive effect annotations even for functions that do not use algebraic effects. This limits the wider adoption of algebraic effects into mainstream languages.

Recent work, in particular Brachthäuser et al. [5] and Tang et al. [25], improves the situation by lowering the annotation burden for functions not using effects. However, these systems still require specialized type system support. Can we get the benefits of algebraic effects and handlers without the need for specialized type system support?

One of the attractive properties of algebraic effects is their elegant mathematical model. However, these more recent approaches do not come with such a model. Can we give a simple semantic model of algebraic effects that justifies our design?

1.1 Algebraic effects and effect handlers

Algebraic effects and effect handlers allow the programmer to temporarily extend the set of available effectful operations.

For example, in OCaml, one can write:

```
type 'a Effect.t +=
  | Get : int Effect.t
  | Set : int -> unit Effect.t
```

Author's address: Leo White, Jane Street, UK.

2026. XXXX-XXXX/2026/7-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

```
let handle_state (f : unit -> 'a) : int -> 'a =
  match f () with
  | x -> fun _ -> x
  | effect Get, k -> fun s -> continue k s s
  | effect Set s', k -> fun _ -> continue k () s'
```

and for the duration of the `f ()` function call two additional effectful operations are available:

```
let get () : int = perform Get
let set (i : int) : unit = perform (Set i)
```

Effect handlers have a couple of key benefits compared to using an explicit monad:

- They allow writing code in "direct style" – i.e. using ordinary functions and native arrow types. In particular, higher-order functions can automatically support effectful functions.
- Multiple effect handlers compose with each other automatically in an easy-to-understand manner.

However, they also have an obvious issue: what happens if you try to call `get` or `set` outside of the `handle_state` handler?

In OCaml, the effects are untracked: nothing prevents attempting to perform an effectful operation that is not currently available, resulting in an `Unhandled` exception being raised. This is essentially the same approach as unchecked exceptions, which are a common source of bugs in many languages.

1.2 Effect systems

Most algebraic effect systems [4, 6, 8, 13, 17, 26] solve the problem of unhandled effects by extending their type system to explicitly track the effects that a function may perform.

In such a system, the operations of our integer state example would have types like:

```
val handle_state : (unit -[istate | 'e]-> 'a) -> int -['e]-> 'a
val get : unit -[istate]-> int
val set : int -[istate]-> unit
```

with effect annotations on function arrows describing the monad implicitly used by those functions, and effect variables used to relate the monads of different functions.

The details vary between systems, but to avoid excessive user annotation, such effect systems tend to have subtle inference built on row polymorphism [13, 17] or implicit subtyping.

A particular source of complexity in such systems is that they require *effect polymorphism* to type higher-order functions. For example, the `List.map` function, which maps a function over a list, must be given a type like:

```
val map : ('a -['e]-> 'b) -> 'a list -['e]-> 'b list
```

which includes effect annotations even though `map` does not use algebraic effects.

1.3 Extending the Ambient Monad

Some more recent systems [5, 6, 25] provide an elegant alternative to effect polymorphism: they treat effect handlers as extending the effects available in the *ambient monad* – the monad associated with ordinary unannotated arrows. This means that, within a handler for integer state, all functions are permitted to use integer state without needing to track this fact in their type. Any effect annotations on a function's type are requirements for additional effects over and above what is currently available in the ambient monad.

In such systems the type of `List.map` can be:

```
val map : ('a -> 'b) -> 'a list -> 'b list
```

whilst still allowing it to interoperate with functions using algebraic effects. The types of our integer state operations might be something like:

```
val get : unit -[istate]-> int
val set : int -[istate]-> unit
```

specifying that an integer state effect must be added to the ambient monad before they can be used. The type of their handler would be:

```
val handle_state : (unit -[istate]-> 'a) -> int -> 'a
```

indicating that it extends the ambient monad with an additional effect.

Brachthäuser et al. [6] describe this approach as "lightweight effect polymorphism" because it brings many of the benefits of effect polymorphism without requiring effect annotations on all higher-order functions.

However, allowing functions to use any effects available at their definition site requires restricting them in some way: they must be prevented from leaving the scope of an effect handler in case they are implicitly relying on that effect handler.

Existing systems accomplish this by making functions second-class, or via sophisticated modal effect or capability types that can be used to track which effect handlers a value may depend on. In this paper we show all that is required is some type-level way to track locality such as Rust's lifetimes or OxCaml's `local` mode.

1.4 The Ambient Monad model

If we are going to make changing the ambient monad the basis of our algebraic effects then we should build a changeable ambient monad right into our semantic model. *We should index the meaning of types and terms by the ambient monad.* Rather than have the semantics of our simple types be sets, $[\![\tau]\!] : \mathrm{Set}$, we should have them be functors from Law_κ to Set:

$$[\![\tau]\!] : \mathrm{Law}_\kappa \to \mathrm{Set}$$

where Law_κ is a suitable category of algebraic theories. For each object $L : \mathrm{Law}_\kappa$ we write T_L for its corresponding monad. This restriction to monads corresponding to algebraic theories is the essence of algebraic effects: it ensures that every pair of monads has a well-defined sum, allowing them to be handled by effect handlers.

These functors from Law_κ to Set are presheaves and so they support all the standard constructions (products, coproducts, exponentials, etc.) needed to support all the usual type formers (products, sums, arrows, etc.) in the standard way.

The monad we use for computations, $\mathcal{T}$, is just the current ambient monad:

$$\mathcal{T}([\![\tau]\!])(L) = T_L([\![\tau]\!](L))$$

which we use to give the semantics of functions:

$$[\![\tau_1 \to \tau_2]\!] = [\![\tau_1]\!] \to \mathcal{T}([\![\tau_2]\!])$$

This is what allows ordinary functions to perform effects from the ambient monad.

A few things fall out of this model for free, whose combination suggests an alternative presentation of algebraic effects.

1.4.1 Reflecting operations. The Yoneda embedding of $L : \mathrm{Law}_\kappa$ immediately gives us a suitable semantics for a family of *effect capability* types $\lambda\!\!\!/(L)$:

$$[\![\lambda\!\!\!/(L)]\!] = \mathrm{Hom}_{\mathrm{Law}_\kappa}(L, \_)$$

Elements of $[\![\text{ɬ}(L)]\!](L')$ represent the ability to reflect effectful operations from L into the ambient monad $T_{L'}$.

We can expose this ability as a reflection construct: for any parameterized operation $op : \tau_p \rightarrowtail \tau_a$ in an algebraic signature Σ, we can provide a computation $\Uparrow(v_h(op))(v_p)$ that uses the effect capability $v_h : \text{ɬ}(\Sigma)$ to perform op parameterized by the value $v_p : \tau_p$, returning a value of type τ_a.

These effect capabilities are closely related to *named handlers* [26] and *lexical effect handlers* [4]. They give you values that can represent the handlers that are currently in scope.

The Yoneda Lemma applied to the semantics of computations tells us that functions from $\text{ɬ}(L)$ correspond to the sum of the ambient monad $T_{L'}$ with the monad T_L:

$$[\![\text{ɬ}(L) \to \tau]\!](L') \cong T_{L'+L}([\![\tau]\!](L' + L)) \tag{1}$$

1.4.2 The global modality. Law_κ has an initial element $\bot$ corresponding to the identity monad. We can use this to define a global comonad, closely related to the purity comonad of Choudhury and Krishnaswami [7]:

$$\Box([\![\tau]\!])(L) = [\![\tau]\!](\bot)$$

This amounts to restricting τ to only the elements which do not depend on the ambient monad. In particular,

$$\Box([\![1 \to \tau]\!]) \cong \Box([\![\tau]\!])$$

showing that thunks within the global comonad are pure.

We will use this as the semantics for a global modality $\Box$: $[\![\Box\, \tau]\!] = \Box([\![\tau]\!])$.

1.4.3 Reifying terms. If we restrict our attention to monads that correspond to some algebraic signature Σ with no equations – i.e. the objects in Law_κ that correspond to free monads – then we can represent the sum of T_Σ with the ambient monad directly as an inductive type W_Σ. W_Σ is sometimes called the freer monad [16].

$W_\Sigma(\tau)$ has a unit constructor $\texttt{ret}(v_r)$ where $v_r : \tau$, and for each parameterized operation $op : \tau_p \rightarrowtail \tau_a$ in Σ it has a constructor $op(v_p, v_k)$ where $v_p : \tau_p$ and $v_k : \Box\, \tau_a \to W_\Sigma(\tau)$.

Example 1.1. W_Σ for the signature of the integer state effect in OxCaml is equivalent to:

```
type 'a t =
  | Return of 'a @@ global
  | Get of (int -> 'a t)
  | Set of int * (unit -> 'a t)
```

where `@@ global` is the OxCaml syntax corresponding to the global modality.

By a theorem of Hyland et al. [15], we have that:

$$T_L([\![W_\Sigma(\tau)]\!](L)) \cong T_{L+\Sigma}([\![\tau]\!](\bot))$$

which, combined with (1), gives us an isomorphism:

$$\mathcal{T}([\![W_\Sigma(\tau)]\!]) \cong [\![\text{ɬ}(\Sigma) \to \Box\, \tau]\!]$$

This isomorphism allows us to reify effectful computations as inductive types, giving the same expressive power as effect handlers. Note that it can be expressed with only effect capabilities, inductive types and a global modality.

The forwards direction of the isomorphism is already provided by reflection of operations. We can provide the backwards direction as a reification construct: for any signature Σ, we provide a computation $\Downarrow(\ell.\, c)$ that binds ℓ to an effect capability of type $\text{ɬ}(\Sigma)$ in the computation $c : \Box\, \tau$ and reifies the uses of ℓ in c as a $W_\Sigma(\tau)$.

1.5 Locality in OxCaml

The Ambient Monad model suggests expressing effect reflection and reification only really requires a global modality, or more broadly some type system mechanism for tracking locality. OxCaml has support for tracking locality in types, which we describe here. For a full treatment see Lorenzen et al. [19].

Values in OxCaml are associated with both a type – which describes how values can be introduced and eliminated – and a *mode* – which tracks properties related to operations other than introduction and elimination. Modes are a product of a number of different axes, tracking a variety of different properties, but here we are only interested in one of them: *locality*. Ignoring the other axes, a mode is either `local` or `global`.

Locality tracks whether a value can escape the current *region*. By default, the current region ends when a function returns, so local values cannot be returned from functions.

```
let leak_local x =
  let (o @ local) = Some x in
  o
Error: o
       ^
This value escapes its region.
```

This tight alignment between regions and function bodies can be inconvenient, so there is an `exclave` construct that allows functions to return local values:

```
let return_local x =
  exclave
    (let (o @ local) = Some x in
     o)
```

`exclave e` can only appear in tail position within a function. It ends the function's region early and then executes the expression `e`. Since `e` is executed within the region of the caller of the function, any local values created within it can be safely returned to that caller.

Arrow types are labelled with two modes: one for the argument and one for the return value.

```
val return_local : 'a @ local -> 'a option @ local
```

For convenience, and backwards compatibility with OCaml, either mode can be omitted, in which case it defaults to `global`.

By default, modes are deep: a local `string list` is a local list of local strings. However, the depth to which a mode applies can be controlled via *modalities*. Record fields can be annotated with the `global` modality to indicate that the contents of the field are global even when the surrounding record is local. For example:

```
type 'a gbl = { g : 'a @@ global }
```

gives a record whose only field has the `global` modality. The contents of the `g` field are always global, for example:

```
let project_global (r @ local) =
  r.g
```

gives a function of type:

```
val project_global : 'a gbl @ local -> 'a @ global
```

Variant constructor arguments can also be annotated with modalities.

Note that, whilst there is a `global` modality, there is no `local` modality. It does not make sense for a value which has the ability to leave a region to contain a value which doesn't have that ability:

when the first value leaves a region it will implicitly cause the second one to leave that region as well. This means the `global` mode is always deep.

1.6 Effect reflection in OxCaml

Using OxCaml's support for locality and the untracked algebraic effects it inherits from OCaml, we can implement effect reflection as a simple library with no specific compiler support.

Given an algebraic signature represented as a module with a single type constructor:

```
module E : sig type 'a t end
```

the library provides $\lambda(\Sigma)$ as an abstract type `Handler(E).t`, and W_Σ as `'a Term(E).t`:

```
type 'a t = 'a Term(E).t =
  | Return : 'a @@ global -> 'a t
  | Perform : 'r E.t @@ global * ('r, 'a) Continuation(E).t -> 'a t
```

For technical reasons (see Section 4.1) we use the type `('r, 'a) Continuation(E).t` in place of `'r -> 'a Term(E).t`. Elements of this type can be applied using a `continue` function.

The reflection and reification operations are provided by the `Reflection` functor as functions `perform` and `reify`:

```
val perform : 'a E.t -> Handler(E).t @ local -> 'a

val reify : (Handler(E).t @ local -> 'a) @ local -> 'a Term(E).t @ local
```

We can use this library to write an effect handler equivalent to the OCaml one in the opening section:

```
module State = struct
  type 'a t =
    | Get : int t
    | Set : int -> unit t
end

module State_eff = Reflection(State)

let handle_state (i : int) (f : Handler(State).t @ local -> 'a) : 'a =
  let rec loop s : 'a Term(State).t -> 'a  = function
    | Return x -> x
    | Perform(Get, k) -> loop s (State_eff.continue k s)
    | Perform(Set s', k) -> loop s' (State_eff.continue k ())
  in
  loop i (State_eff.reify f)
```

The handler calls `f` and passes it a local value of type `Handler(State).t`. For the duration of the call to `f` the ambient monad is extended with two additional effectful operations, which can be used via reflection:

```
let get (h : Handler(State).t @ local) : int =
  State_eff.perform Get h

let set (h : Handler(State).t @ local) (i : int) : unit =
  State_eff.perform (Set i) h
```

The locality of the `Handler(State).t` value passed to `f` ensures that these operations can only be performed when available in the ambient monad. The type system prevents this handler value from escaping the scope of `f`, which in turn prevents `get` or `set` from being called outside that scope.

1.6.1 Lightweight effect polymorphism. The following function uses the state handler to calculate the total of a list of integers:

```
let total l =
  handle_state 0 (fun h ->
    List.iter (fun x -> set h (get h + x)) l;
    get h)
```

This relies on the locality of the function parameter of `List.iter`:

```
val iter : ('a -> unit) @ local -> 'a list -> unit
```

The `local` annotation on the parameter ensures that any function passed will not escape the current region. This allows us to pass in a closure that closes over the `local` handler `h` and use it to perform some effects.

1.7 Contributions

We make the following contributions:

- We propose *effect reflection*, a new presentation of effect handlers that ensures all effectful operations are properly handled without requiring an effect system.
- We present a simply-typed lambda calculus with effect reflection. We prove that the extended system maintains subject reduction, confluence and strong normalization. We prove *effect safety*: a closed well-typed program never attempts to perform an unhandled operation. We prove that the ordinary simply-typed lambda calculus embeds into it. (Section 2)
- We propose the Ambient Monad model of effectful computations. We use it to give a denotational semantics of our lambda calculus with effect reflection. We prove that this semantics is sound, compositional and adequate. (Section 3)
- We give a simple implementation of effect reflection in OxCaml using its support for tracking locality in the type system and the untracked algebraic effects it inherits from OCaml. We demonstrate its usability with some classic examples of algebraic effects. (Section 4).

2 SIMPLY-TYPED LAMBDA CALCULUS WITH EFFECT REFLECTION

We present a simply-typed lambda calculus extended with effect reflection. This extension requires adding a separate syntax and typing judgement for computations, in the style of the Fine-Grained Call-By-Value lambda calculus [18], with function application treated as a computation.

2.1 The base language

Our language is simply typed, featuring all the usual type-formers. The syntax and typing rules for the base language are shown in Fig.1.

For convenience we allow some simple pattern forms in variable binding positions. We allow $()$ to be used in place of a variable when binding a unit value, and we allow (x_1, x_2) to be used in place of a single variable when binding a product value. Unit patterns are encoded as just an unused variable, whilst product bindings are encoded using a variable x_0 and applying the substitutions $[x_1 \setminus \pi_1\, x_0][x_2 \setminus \pi_2\, x_0]$ to the body of the binding.

$$
\begin{aligned}
\text{variables}, x &\in \mathcal{X} \\
\text{types}, \tau &::= \tau \times \tau \mid 1 \mid \tau + \tau \mid \tau \to \tau \mid \dots \\
\text{contexts}, \Gamma &::= \varepsilon \mid \Gamma;\, x : \tau \mid \dots \\
\text{values}, v &::= x \mid \lambda x.\, c \mid (v, v) \mid \pi_1\, v \mid \pi_2\, v \mid () \\
&\quad \mid \mathtt{in}_L\, v \mid \mathtt{in}_R\, v \mid \mathtt{case}\, v\, \{\, \mathtt{in}_L\, x \Rightarrow v\, ;\, \mathtt{in}_R\, x \Rightarrow v\, \} \mid \dots \\
\text{computations}, c &::= \mathtt{return}\, v \mid \mathtt{let}\, x = c\, \mathtt{in}\, c \mid v\, v \mid \dots
\end{aligned}
$$

$$
\frac{}{\Gamma_1;\, x : \tau;\, \Gamma_2 \vdash_V x : \tau}
\qquad
\frac{\Gamma;\, x : \tau_1 \vdash_C c : \tau_2}{\Gamma \vdash_V \lambda x.\, c : \tau_1 \to \tau_2}
\qquad
\frac{\Gamma \vdash_V v_1 : \tau_1 \qquad \Gamma \vdash_V v_2 : \tau_2}{\Gamma \vdash_V (v_1, v_2) : \tau_1 \times \tau_2}
$$

$$
\frac{\Gamma \vdash_V v : \tau_1 \times \tau_2}{\Gamma \vdash_V \pi_1\, v : \tau_1}
\qquad
\frac{\Gamma \vdash_V v : \tau_1 \times \tau_2}{\Gamma \vdash_V \pi_2\, v : \tau_2}
\qquad
\frac{}{\Gamma \vdash_V () : 1}
\qquad
\frac{\Gamma \vdash_V v : \tau_1}{\Gamma \vdash_V \mathtt{in}_L\, v : \tau_1 + \tau_2}
$$

$$
\frac{\Gamma \vdash_V v : \tau_2}{\Gamma \vdash_V \mathtt{in}_R\, v : \tau_1 + \tau_2}
\qquad
\frac{\Gamma \vdash_V v_1 : \tau_1 + \tau_2 \qquad \Gamma;\, x_1 : \tau_1 \vdash_V v_2 : \tau_3 \qquad \Gamma;\, x_2 : \tau_2 \vdash_V v_3 : \tau_3}{\Gamma \vdash_V \mathtt{case}\, v_1\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_2\, ;\, \mathtt{in}_R\, x_2 \Rightarrow v_3\, \} : \tau_3}
$$

$$
\frac{\Gamma \vdash_V v : \tau}{\Gamma \vdash_C \mathtt{return}\, v : \tau}
\qquad
\frac{\Gamma \vdash_C c_1 : \tau_1 \qquad \Gamma;\, x : \tau_1 \vdash_C c_2 : \tau_2}{\Gamma \vdash_C \mathtt{let}\, x = c_1\, \mathtt{in}\, c_2 : \tau_2}
\qquad
\frac{\Gamma \vdash_V v_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash_V v_2 : \tau_1}{\Gamma \vdash_C v_1\, v_2 : \tau_2}
$$

Fig. 1. Basic syntax and typing rules

Note that case – the eliminator for sum types – is a value rather than a computation. We can define a computation version as:

$$
\begin{aligned}
&\mathtt{case}\, v\, \{\, \mathtt{in}_L\, x_1 \Rightarrow c_1\, ;\, \mathtt{in}_R\, x_2 \Rightarrow c_2\, \} := \\
&\qquad (\mathtt{case}\, v\, \{\, \mathtt{in}_L\, x_1 \Rightarrow \lambda().\, c_1\, ;\, \mathtt{in}_R\, x_2 \Rightarrow \lambda().\, c_2\, \})\ ()
\end{aligned}
$$

2.2 Locality

We track locality via a global comonadic modality $\Box$. In our setting, being global means not depending on the ambient monad. For example, $1 \to 1$ represents computations that may use operations from the ambient monad, while $\Box(1 \to 1)$ contains only pure computations that ignore the ambient monad.

When using $\mathtt{box}\, v$ to build a value with a global type, we can only use existing values that also have a global type. We achieve this using a standard approach for modal types: we extend contexts with a binding form that requires a global type and then we restrict the context in which we type v to only include such bindings. The new global binding form is written $\Gamma;\, x :_\Box \tau$ and represents a value in the context of type $\Box\, \tau$. We create global bindings in the context using the elimination form: $\mathtt{let}(\mathtt{box}\, x) = v_1\, \mathtt{in}\, v_2$. As with case, this elimination form is a value, but we can define a computation version directly in terms of it.

Restricting the context is implemented via an operation on contexts $\Gamma/\Box$ that removes all local bindings, leaving only the global ones:

$$
\begin{aligned}
\varepsilon/\Box &= \varepsilon \\
(\Gamma;\, x : \tau)/\Box &= \Gamma/\Box \\
(\Gamma;\, x :_\Box \tau)/\Box &= \Gamma/\Box;\, x :_\Box \tau
\end{aligned}
$$

$$\begin{aligned}
\text{types, } \tau &::= \ldots \mid \Box\,\tau \mid \ldots \\
\text{modalities, } m &::= _ \mid \Box \\
\text{contexts, } \Gamma &::= \ldots \mid \Gamma;\, x :_{\Box} \tau \\
\text{values, } v &::= \ldots \mid \mathtt{box}\, v \mid \mathtt{let}(\mathtt{box}\, x) = v\ \mathtt{in}\, v \mid \\
&\qquad \mathtt{case}^{\Box}\, v\, \{\, \mathtt{in}_L\, x \Rightarrow v\,;\, \mathtt{in}_R\, x \Rightarrow v\,\} \mid \mathtt{run}(c) \mid \ldots
\end{aligned}$$

$$\frac{}{\Gamma_1;\, x :_m \tau;\, \Gamma_2 \vdash_V x : \tau} \qquad \frac{\Gamma/\Box \vdash_V v : \tau}{\Gamma \vdash_V \mathtt{box}\, v : \Box\,\tau} \qquad \frac{\Gamma \vdash_V v_1 : \Box\,\tau_1 \qquad \Gamma;\, x :_{\Box} \tau_1 \vdash_V v_2 : \tau_2}{\Gamma \vdash_V \mathtt{let}(\mathtt{box}\, x) = v_1\ \mathtt{in}\, v_2 : \tau_2}$$

$$\frac{\Gamma/m \vdash_V v_1 : \tau_1 + \tau_2 \qquad \Gamma;\, x_1 :_m \tau_1 \vdash_V v_2 : \tau_3 \qquad \Gamma;\, x_2 :_m \tau_2 \vdash_V v_3 : \tau_3}{\Gamma \vdash_V \mathtt{case}^m\, v_1\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_2\,;\, \mathtt{in}_R\, x_2 \Rightarrow v_3\,\} : \tau_3} \qquad \frac{\Gamma/\Box \vdash_C c : \tau}{\Gamma \vdash_V \mathtt{run}(c) : \tau}$$

Fig. 2. Locality syntax and typing rules

Global computations do not depend on the ambient monad, so we also include a value form $\mathtt{run}(c)$, which allows treating a global computation c as a value.

The global comonad distributes over sum types:

$$\Box(\tau_1 + \tau_2) \qquad \cong \qquad (\Box\,\tau_1) + (\Box\,\tau_2)$$

which we realize via $\mathtt{case}^{\Box}$, a global form of the `case` construct, which allows you to match on a global sum and get out global components.

For convenience we allow $(\mathtt{box}\, x)$ to be used as a pattern in variable binding positions. These are treated as binding a variable x_0 and prefixing the body of the binding with $\mathtt{let}(\mathtt{box}\, x) = x_0\ \mathtt{in}$.

We add a modalities category, m, to the formal syntax, which is either blank space or $\Box$, to make it easier to describe rules that are parametric in the presence or absence of the global modality. As part of this, the $\Gamma/\Box$ operation is extended to work on modalities Γ/m with $\Gamma/_{\_} = \Gamma$.

The syntax and typing rules related to locality are in Fig.2.

2.3 Signatures and inductive types

Effect reflection requires, for any signature Σ that is being reflected, an inductive type W_Σ to represent the free algebras of Σ. W_Σ is sometimes called the *freer monad* of Σ [16]. It is a form of W-type, and in a dependently-typed setting we would probably use W-types directly.

As is standard for algebraic effect systems, we only support handlers for algebraic theories without any equations, even though our semantics is defined in terms of arbitrary theories. We define algebraic signatures Σ as finite sets of operations:

$$\{op_1 : \tau_{p_1} \rightarrowtail \tau_{a_1}; \ldots; op_n : \tau_{p_n} \rightarrowtail \tau_{a_n}\}$$

An operation type $\tau_p \rightarrowtail \tau_a$ indicates an operation parameterized by τ_p and with arity τ_a.

Example 2.1. The signature for boolean state, which we'll call Σ_{bs}, is:

$$\{\mathrm{get} : 1 \rightarrowtail (1+1); \mathrm{set} : (1+1) \rightarrowtail 1\}$$

We also allow the sum of two signatures $\Sigma_1 + \Sigma_2$ to be used as a signature. It has operations:

$$\begin{aligned}
\{\ &\mathrm{left}(op_{1,1}) : \tau_{p_{1,1}} \rightarrowtail \tau_{a_{1,1}}\ ;\ \ldots\ ;\ \mathrm{left}(op_{1,n}) : \tau_{p_{1,n}} \rightarrowtail \tau_{a_{1,n}}\ ; \\
&\mathrm{right}(op_{2,1}) : \tau_{p_{2,1}} \rightarrowtail \tau_{a_{2,1}}\ ;\ \ldots\ ;\ \mathrm{right}(op_{2,m}) : \tau_{p_{2,m}} \rightarrowtail \tau_{a_{2,m}}\ \}
\end{aligned}$$

where $op_{1,1}\ ;\ \ldots\ ;\ op_{1,n}$ are the operations of Σ_1 and $op_{2,1}\ ;\ \ldots\ ;\ op_{2,m}$ are the operations of Σ_2.

$$\begin{aligned}
\text{types, } \tau &::= \ldots \mid W_\Sigma(\tau) \mid \ldots \\
\text{signatures, } \Sigma &::= \{op : \tau \rightarrowtail \tau\,;\ \ldots\ ;\ op : \tau \rightarrowtail \tau\} \mid \Sigma + \Sigma \\
\text{values, } v &::= \ldots \mid op(v,\, v) \mid \mathtt{ret}(v) \mid \\
&\qquad \mathtt{rec}\, v\, \{\mathtt{ret}(x) \Rightarrow v\,;\ op(x, x) \Rightarrow v\,;\ \ldots\ ;\ op(x, x) \Rightarrow v\} \ldots
\end{aligned}$$

$$\frac{op : \tau_1 \rightarrowtail \tau_2 \in \Sigma \qquad \Gamma/\Box \vdash_V v_1 : \tau_1 \qquad \Gamma \vdash_V v_2 : \Box\, \tau_2 \to W_\Sigma(\tau)}{\Gamma \vdash_V op(v_1,\, v_2) : W_\Sigma(\tau)} \qquad \frac{\Gamma/\Box \vdash_V v : \tau}{\Gamma \vdash_V \mathtt{ret}(v) : W_\Sigma(\tau)}$$

$$\frac{\begin{array}{c}\Gamma \vdash_V v_s : W_\Sigma(\tau_1) \qquad \Sigma = \{op_1 : \tau_{p_1} \rightarrowtail \tau_{a_1}; \ldots; op_n : \tau_{p_n} \rightarrowtail \tau_{a_n}\} \qquad \Gamma;\, x_r :_\Box \tau_1 \vdash_V v_e : \tau_2 \\ \Gamma;\, x_{p_1} :_\Box \tau_{p_1};\, x_{k_1} : \Box\, \tau_{a_1} \to \tau_2 \vdash_V v_1 : \tau_2 \qquad \ldots \qquad \Gamma;\, x_{p_n} :_\Box \tau_{p_n};\, x_{k_n} : \Box\, \tau_{a_n} \to \tau_2 \vdash_V v_n : \tau_2\end{array}}{\Gamma \vdash_V \mathtt{rec}\, v_s\, \{\mathtt{ret}(x_r) \Rightarrow v_e\,;\ op_1(x_{p_1}, x_{k_1}) \Rightarrow v_1\,;\ \ldots\ ;\ op_n(x_{p_n}, x_{k_n}) \Rightarrow v_n\} : \tau_2}$$

Fig. 3. Inductive type syntax and typing rules

For each operation $op : \tau_p \rightarrowtail \tau_a$, $W_\Sigma(\tau)$ has a constructor $op(v_p, v_k)$ where $v_p :_\Box \tau_p$ and $v_k : \Box\, \tau_a \to W_\Sigma(\tau)$. $W_\Sigma(\tau)$ also has a constructor $\mathtt{ret}(v)$ where $v :_\Box \tau$.

The eliminator for W_Σ is written

$$\begin{aligned}
&\mathtt{rec}\, v_s\, \{\mathtt{ret}(x) \Rightarrow v_e\,; \\
&\qquad op_1(x_{p_1}, x_{k_1}) \Rightarrow v_1\,; \\
&\qquad \ldots\,; \\
&\qquad op_n(x_{p_n}, x_{k_n}) \Rightarrow v_n\}
\end{aligned}$$

The syntax and typing rules for inductive types are shown in Fig.3.

As with `case`, `rec` is defined as a value and we can define a computation version in terms of it.

Example 2.2. The following value of type $W_{\Sigma_{bs}}(1)$ represents a reified computation that negates the value of the state:

$$\begin{aligned}
&get((), \\
&\quad \lambda(\mathsf{box}\, x_a). \\
&\qquad \mathtt{return}\ set(\mathsf{run}(negate\, x_a),\ \lambda(\mathsf{box}\, x_r).\ \mathtt{return}\ \mathtt{ret}(x_r)))
\end{aligned}$$

where *negate* is the function that negates a boolean.

2.4 Effect reflection

We provide a type $\curlywedge(\Sigma)$ for each algebraic signature Σ to represent mappings from the operations of Σ into the current ambient monad. We call such mappings *effect capabilities*. Effect reflection appears as two constructs: reflection and reification. Reflection $\Uparrow(v(op))$ uses a value $v : \curlywedge(\Sigma)$ to map the operation op from Σ into the ambient monad. Reification $\Downarrow$ extends the ambient monad with the operations from some Σ, and provides a value of type $\curlywedge(\Sigma)$ for constructing those operations via reflection. These constructs implement the core isomorphism of effect reflection: reification can convert functions $\curlywedge(\Sigma) \to \Box\, \tau$ into terms $W_\Sigma(\tau)$, while reflection provides the inverse direction. Their syntax and typing rules are shown in Fig.4.

We introduce a separate set of variables ℓ for the bindings introduced by reification. This makes it easier to define the desired reduction rules and normal forms. We give them their own binding form $\ell : \Sigma$ in the context, which is also removed by the $/\Box$ operation.

$$\begin{array}{rl} \text{locations}, \ell \in & \mathcal{L} \\ \text{types}, \tau ::= & \ldots \mid \curlywedge(\Sigma) \mid \ldots \\ \text{contexts}, \Gamma ::= & \ldots \mid \Gamma;\, \ell : \Sigma \\ \text{values}, v ::= & \ldots \mid \ell \mid \mathtt{out}_L\, v \mid \mathtt{out}_R\, v \\ \text{computations}, c ::= & \ldots \mid \Uparrow(v(op))(v) \mid \Downarrow(\ell.\, c) \end{array}$$

$$\frac{}{\Gamma_1;\, \ell : \Sigma;\, \Gamma_2 \vdash_V \ell : \curlywedge(\Sigma)} \qquad \frac{\Gamma \vdash_V v : \curlywedge(\Sigma_1 + \Sigma_2)}{\Gamma \vdash_V \mathtt{out}_L\, v : \curlywedge(\Sigma_1)} \qquad \frac{\Gamma \vdash_V v : \curlywedge(\Sigma_1 + \Sigma_2)}{\Gamma \vdash_V \mathtt{out}_R\, v : \curlywedge(\Sigma_2)}$$

$$\frac{\Gamma \vdash_V v_1 : \curlywedge(\Sigma) \qquad op : \tau_1 \rightarrowtail \tau_2 \in \Sigma \qquad \Gamma/\Box \vdash_V v_2 : \tau_1}{\Gamma \vdash_C \Uparrow(v_1(op))(v_2) : \Box\, \tau_2} \qquad \frac{\Gamma;\, \ell : \Sigma \vdash_C c : \Box\, \tau}{\Gamma \vdash_C \Downarrow(\ell.\, c) : W_\Sigma(\tau)}$$

Fig. 4. Effect reflection syntax and typing rules

Note that we treat reflection of individual operations $\Uparrow(v(op))$ as primitive. We can define reflection of terms $\Uparrow(v)$ in terms of $\Uparrow(v(op))$:

$$\begin{array}{rl} \Uparrow(v)(c) := & \mathtt{let}\, x_w = c\, \mathtt{in} \\ & \mathtt{rec}\, x_w \{ \\ & \quad \mathtt{ret}(x_r) \Rightarrow \mathtt{return}\, (\mathtt{box}\, x_r)\, ; \\ & \quad op_1(x_{p_1}, x_{k_1}) \Rightarrow \mathtt{let}\, x_{a_1} = \Uparrow(v(op_1))(x_{p_1})\, \mathtt{in}\, x_{k_1}\, x_{a_1}\, ; \\ & \quad \ldots\, ; \\ & \quad op_n(x_{p_n}, x_{k_n}) \Rightarrow \mathtt{let}\, x_{a_n} = \Uparrow(v(op_n))(x_{p_n})\, \mathtt{in}\, x_{k_n}\, x_{a_n} \} \end{array}$$

which has this admissible typing rule:

$$\frac{\Gamma \vdash_V v : \curlywedge(\Sigma) \qquad \Gamma \vdash_C c : W_\Sigma(\tau)}{\Gamma \vdash_C \Uparrow(v)(c) : \Box\, \tau}$$

Example 2.3. The value from Example 2.2 can also be constructed via reflection:

$$\begin{array}{l} \Downarrow(\ell. \\ \quad \mathtt{let}(\mathtt{box}\, x) = \Uparrow(\ell(get))()\, \mathtt{in} \\ \quad \Uparrow(\ell(set))(\mathtt{run}(negate\, x))) \end{array}$$

where *negate* is the function that negates a boolean.

We also provide projections $\mathtt{out}_L$ and $\mathtt{out}_R$ that decompose an effect capability for $\Sigma_1 + \Sigma_2$ into effect capabilities for Σ_1 and Σ_2 respectively. This is actually crucial to obtain the full expressivity of effect handlers (see Section 4.6).

2.5 Operational semantics

We define the operational semantics as a pair of mutually defined reduction relations $\rightsquigarrow_V$ and $\rightsquigarrow_C$ for values and computations respectively. These are shown in Fig.5, omitting the congruence rules. We write $\rightsquigarrow_V^*$ and $\rightsquigarrow_C^*$ for the reflexive transitive closures of these relations. The definitions of normal forms for values and computations are given in Fig.6.

$$\pi_1(v_1, v_2) \rightsquigarrow_V v_1$$
$$\pi_2(v_1, v_2) \rightsquigarrow_V v_2$$
$$\mathsf{case}^m(\mathsf{in}_L\, v_1)\,\{\,\mathsf{in}_L\, x_1 \Rightarrow v_2\,;\,\mathsf{in}_R\, x_2 \Rightarrow v_3\,\} \rightsquigarrow_V v_2[x_1\backslash v_1]$$
$$\mathsf{case}^m(\mathsf{in}_R\, v_1)\,\{\,\mathsf{in}_L\, x_1 \Rightarrow v_2\,;\,\mathsf{in}_R\, x_2 \Rightarrow v_3\,\} \rightsquigarrow_V v_3[x_2\backslash v_1]$$
$$\mathsf{let}(\mathsf{box}\, x) = \mathsf{box}\, v_1\, \mathsf{in}\, v_2 \rightsquigarrow_V v_2[x\backslash v_1]$$
$$\mathsf{run}(\mathsf{return}\, v) \rightsquigarrow_V v$$
$$\mathsf{rec}(\mathsf{ret}(v_r))\,\{\mathsf{ret}(x) \Rightarrow v_e\,;\,\ldots\} \rightsquigarrow_V v_e[x\backslash v_r]$$
$$\mathsf{rec}(op(v_1,\, v_2))\,\{H_1\,;\,op(x_1, x_2) \Rightarrow v_3\,;\,H_2\} \rightsquigarrow_V$$
$$v_3[x_1\backslash v_1][x_2\backslash \lambda x_a.\ \mathsf{let}\, x_w = v_2\, x_a\, \mathsf{in}\, \mathsf{return}\,(\mathsf{rec}\, x_w\,\{H_1\,;\,op(x_1, x_2) \Rightarrow v_3\,;\,H_2\})]$$
$$(\lambda x.\, c)\, v \rightsquigarrow_C c[x\backslash v]$$
$$\mathsf{let}\, x = \mathsf{return}\, v\, \mathsf{in}\, c \rightsquigarrow_C c[x\backslash v]$$
$$\mathsf{let}\, x_2 = \mathsf{let}\, x_1 = \Uparrow(\ell(op))(v)\, \mathsf{in}\, c_1\, \mathsf{in}\, c_2 \rightsquigarrow_C \mathsf{let}\, x_1 = \Uparrow(\ell(op))(v)\, \mathsf{in}\, \mathsf{let}\, x_2 = c_1\, \mathsf{in}\, c_2$$
$$\Uparrow((\mathsf{out}_L\, v_1)(op))(v_2) \rightsquigarrow_C \Uparrow(v_1(\mathrm{left}(op)))(v_2)$$
$$\Uparrow((\mathsf{out}_R\, v_1)(op))(v_2) \rightsquigarrow_C \Uparrow(v_1(\mathrm{right}(op)))(v_2)$$
$$\Downarrow(\ell.\ \mathsf{return}\,(\mathsf{box}\, v)) \rightsquigarrow_C \mathsf{return}\ \mathsf{ret}(v)$$
$$\Downarrow(\ell_1.\ \Uparrow(\ell_1(op))(v)) \rightsquigarrow_C \mathsf{return}\ op(v,\, \lambda(\mathsf{box}\, x).\ \mathsf{return}\ \mathsf{ret}(x))$$
$$\Downarrow(\ell_1.\ \mathsf{let}\, x = \Uparrow(\ell_1(op))(v)\, \mathsf{in}\, c) \rightsquigarrow_C \mathsf{return}\ op(v,\, \lambda x.\ \Downarrow(\ell_1.\, c))$$
$$\Downarrow(\ell_1.\ \Uparrow(\ell_2(op))(v)) \rightsquigarrow_C \mathsf{let}(\mathsf{box}\, x) = \Uparrow(\ell_2(op))(v)\, \mathsf{in}\, \mathsf{return}\ \mathsf{ret}(x) \qquad (\ell_1 \neq \ell_2)$$
$$\Downarrow(\ell_1.\ \mathsf{let}\, x = \Uparrow(\ell_2(op))(v)\, \mathsf{in}\, c) \rightsquigarrow_C \mathsf{let}\, x = \Uparrow(\ell_2(op))(v)\, \mathsf{in}\, \Downarrow(\ell_1.\, c) \qquad (\ell_1 \neq \ell_2)$$

Fig. 5. Reduction relations

$$\begin{aligned}
\text{neutral normal values, } \eta ::= {} & x \mid \pi_1\, \eta \mid \pi_2\, \eta \mid \\
& \mathsf{case}^m\, \eta\, \{\,\mathsf{in}_L\, x \Rightarrow \omega\,;\,\mathsf{in}_R\, x \Rightarrow \omega\,\} \mid \\
& \mathsf{let}(\mathsf{box}\, x) = \eta\, \mathsf{in}\, \omega \mid \mathsf{run}(v) \mid \\
& \mathsf{rec}\, \eta\, \{\mathsf{ret}(x) \Rightarrow \omega\,;\, op(x, x) \Rightarrow \omega\,;\,\ldots\,;\, op(x, x) \Rightarrow \omega\} \\
\text{normal values, } \omega ::= {} & \eta \mid (\omega, \omega) \mid () \mid \mathsf{in}_L\, \omega \mid \mathsf{in}_R\, \omega \mid \lambda x.\, \mu \mid \\
& op(\omega,\, \omega) \mid \mathsf{ret}(\omega) \mid \\
& \mathsf{box}\, \omega \mid \ell \mid \mathsf{out}_L\, \omega \mid \mathsf{out}_R\, \omega \\
\text{neutral normal computations, } \nu ::= {} & \eta\, \omega \mid \Downarrow(\ell.\, \nu) \mid \Downarrow(\ell.\ \mathsf{return}\ \eta) \mid \Uparrow(\eta(op))(\omega) \mid \mathsf{let}\, x = \nu\, \mathsf{in}\, \mu \\
\text{normal computations, } \mu ::= {} & \nu \mid \mathsf{return}\ \omega \mid \Uparrow(\ell(op))(\omega) \mid \mathsf{let}\, x = \Uparrow(\ell(op))(\omega)\, \mathsf{in}\, \mu
\end{aligned}$$

Fig. 6. Normal forms

Note that the reduction rules are for strong reduction, i.e. there is a congruence rule for abstraction:

$$\frac{c_1 \rightsquigarrow_C c_2}{\lambda x.\, c_1 \rightsquigarrow_V \lambda x.\, c_2}$$

We prove subject reduction for this semantics in Appendix A by induction on the reduction relations.

Theorem 2.4.

$$\Gamma \vdash_V v_1 : \tau \quad \wedge \quad v_1 \rightsquigarrow_V v_2 \quad \Longrightarrow \quad \Gamma \vdash_V v_2 : \tau$$
$$\Gamma \vdash_C c_1 : \tau \quad \wedge \quad c_1 \rightsquigarrow_C c_2 \quad \Longrightarrow \quad \Gamma \vdash_C c_2 : \tau$$

These reduction relations are confluent:

Theorem 2.5.

$$v_1 \rightsquigarrow_V^* v_2 \quad \wedge \quad v_1 \rightsquigarrow_V^* v_3 \quad \Longrightarrow \quad \exists v_4.\ \ v_2 \rightsquigarrow_V^* v_4 \quad \wedge \quad v_3 \rightsquigarrow_V^* v_4$$
$$c_1 \rightsquigarrow_C^* c_2 \quad \wedge \quad c_1 \rightsquigarrow_C^* c_3 \quad \Longrightarrow \quad \exists c_4.\ \ c_2 \rightsquigarrow_C^* c_4 \quad \wedge \quad c_3 \rightsquigarrow_C^* c_4$$

which we prove using the standard approach via an equivalent notion of parallel reduction in Appendix B.

They are also strongly normalizing. We call the sets of values and computations for which there exist no infinite chains of reductions SN_V and SN_C respectively.

Theorem 2.6.

$$\Gamma \vdash_V v : \tau \quad \Longrightarrow \quad v \in SN_V$$
$$\Gamma \vdash_C c : \tau \quad \Longrightarrow \quad c \in SN_C$$

We prove this using logical relations in Appendix C.

Inspecting the normal forms in Fig.6, we can see that $\texttt{return}\ \omega$ is the only normal computation form that is well-typed in the empty context: the other forms all require a location or variable from the context. Combined with subject reduction, this makes effect safety a corollary of strong normalization: a well-typed program can never attempt to perform an effectful operation that is not handled.

Corollary 2.7 (Effect safety).

$$\vdash_C c : \tau \quad \Longrightarrow \quad \exists \omega.\ \ c \rightsquigarrow_C^* \texttt{return}\ \omega$$

2.6 Embedding the Simply-Typed Lambda Calculus

Using the global modality and $\texttt{run}(c)$ construct we can embed the Simply-Typed Lambda Calculus (STLC) into this language as values. This demonstrates that the language is a proper extension of STLC. We show this embedding and prove it correct in Appendix D.

3 THE AMBIENT MONAD MODEL

The design of effect reflection falls out naturally from a novel semantic model of algebraic effects. The key idea is to index the meaning of types and terms by an ambient monad of effects: rather than have the semantics of each type be a set of elements, the semantics of each type is a mapping from the ambient monad to a set of elements. We give a denotational semantics to the language from Section 2 using this model.

3.1 Algebraic Theories and Free Algebras

We need the monads we use for the ambient monad to have a well-defined notion of sum. This is a requirement for composable effect handlers: within the effect handler the ambient monad will be the sum of the handled effect and the ambient monad from outside of the handler. That rules out allowing all monads as the ambient monad, since the sum of two arbitrary monads may not exist.

Algebraic effects ensure that such a sum always exists by only supporting monads that are the free algebra of some algebraic theory. We use the same approach for the Ambient Monad model.

We recall the definitions of algebraic signature, terms of a signature, algebraic theory and models of a theory.

Definition 3.1 (Algebraic signature)**.** An *algebraic signature* is a set of operations with associated arities.

Example 3.1. The algebraic signature for boolean state Σ_{bs} is:

$$\{(\text{get}, 2); (\text{set\_true}, 1); (\text{set\_false}, 1)\}$$

Definition 3.2 (Term)**.** A *term* t of some algebraic signature Σ is either a free variable x or $op(t_1, \ldots, t_k)$ where op is an operation of Σ with arity k and each t_i is itself a term.

Definition 3.3 (Algebraic theory)**.** An *algebraic theory* consists of an algebraic signature Σ and a set of equations on terms of Σ.

Example 3.2. The algebraic theory for boolean state L_{bs} adds the following equations to the signature of boolean state Σ_{bs}:

$$\begin{aligned}
&\text{set_true}(\text{get}(x, y)) = \text{set_true}(x) && \text{set_false}(\text{get}(x, y)) = \text{set_false}(y)\\
&\text{set_true}(\text{set_true}(x)) = \text{set_true}(x) && \text{set_true}(\text{set_false}(x)) = \text{set_false}(x)\\
&\text{set_false}(\text{set_true}(x)) = \text{set_true}(x) && \text{set_false}(\text{set_false}(x)) = \text{set_false}(x)\\
&\text{get}(\text{set_true}(x), \text{set_false}(x)) = x
\end{aligned}$$

Definition 3.4 (Model)**.** A *model* M of some algebraic theory L consists of a carrier set S and for each operation op in the signature of L a function $op_M : (\Pi_k S) \to S$ where k is the arity of op, such that the equations of L are respected when interpreting the terms using these functions for the operations. Models of L are also called algebras of L.

Example 3.3. We can model the theory of boolean state L_{bs} using the carrier set $2 \to 2$ and interpreting the operations with:

$$\begin{aligned}
\text{get}(x, y) &= \lambda s.\ \text{if } s \text{ then } x(s) \text{ else } y(s)\\
\text{set_true}(x) &= \lambda s.\, x(\text{true})\\
\text{set_false}(x) &= \lambda s.\, x(\text{false})
\end{aligned}$$

Arities in algebraic signatures are often considered as natural numbers, but they can equivalently be sets. The arguments of an operator are then a family of terms indexed by that set. It is also convenient to allow families of operations parameterized by some set. This gives us signatures of the form:

$$\{op_1 : P_1 \rightarrowtail A_1; \ldots; op_n : P_n \rightarrowtail A_n\}$$

where $op : P \rightarrowtail A$ indicates an operation op parameterized by P and with arity A.

Example 3.4. The algebraic signature for boolean state Σ_{bs} can be written as:

$$\{\text{get} : 1 \rightarrowtail 2; \text{set} : 2 \rightarrowtail 1\}$$

Given an algebraic theory L and a set A we can construct the free model, or *free algebra*, $T_L(A)$. The values of $T_L(A)$ are terms built using the elements of A and the operations of L quotiented by the equations in L. T_L is a monad.

Example 3.5. The free algebras of the signature of boolean state $T_{\Sigma_{bs}}$ are equivalent to the type:

```
type 'a t =
  | Return of 'a
  | Get of unit * (bool -> 'a t)
  | Set of bool * (unit -> 'a t)
```

The free algebras of the theory of boolean state $T_{L_{bs}}$ are equivalent to the above type quotiented by the equations of L_{bs}.

For an algebraic signature Σ the free algebra T_Σ is the free monad of an endofunctor F_Σ. The values of $F_\Sigma(A)$ are of the form $op(a_1, \ldots, a_k)$ where op is an operation of Σ with arity k and each a_i is an element of A.

Example 3.6. $F_{\Sigma_{bs}}$ is equivalent to the type:

```
type 'a t =
  | Get of unit * (bool -> 'a)
  | Set of bool * (unit -> 'a)
```

3.2 Lawvere theories

Lawvere theories are a categorical representation of algebraic theories that doesn't depend on the particular choice of operations and equations. We will use them as our concrete representation of the ambient monad.

Definition 3.5. A *Lawvere theory* is a category L with finite products in which every object is isomorphic to some finite cartesian product $X^n = X \times \ldots \times X$ of a distinguished object X.

The objects of a Lawvere theory correspond to the different possible arities and the morphisms $X^n \to X$ correspond to terms of L with n free variables quotiented by the equations in L. A model of a Lawvere theory L is a product-preserving functor $L \to \mathrm{Set}$.

We will mostly describe Lawvere theories and constructions on them in terms of their presentations as algebraic theories, so an understanding of the details of Lawvere theories is not required for understanding the bulk of this paper.

Lawvere theories form a category Law whose morphisms are equivalent to maps from the operations of one theory to terms of another that preserve the equations of the first theory according to the equations of the second.

3.3 Bounded Lawvere theories

Traditional algebraic signatures are restricted to the case of a finite set of operations each with a finite arity, but programming languages require more generality. In terms of effect reflection, this would amount to demanding, for any operation $op : \tau_p \rightarrowtail \tau_a$, that τ_p and τ_a be finite types. This would allow for boolean state, but would not support $\mathbb{N}$ state.

In the algebraic effects literature the most common choice is to use countable algebraic signatures. However, this is still too restrictive, supporting $\mathbb{N}$ state but not supporting $\mathbb{N} \to \mathbb{N}$ state.

We want to allow arbitrary types for the parameters and arities of our operations. However, we also wish our category of Lawvere theories to be small. To achieve this we will parameterize our model and semantics by a regular cardinal κ, and use algebraic theories whose sets of operations and arities are bounded by κ. We will not worry about the choice of κ as for any given program there is always a sufficiently large κ available.

We can extend the definition of Lawvere theory to support algebraic theories bounded by κ:

Definition 3.6. A *κ-bounded Lawvere theory* is a small category whose set of objects has cardinality $\lambda < \kappa$ and whose set of morphisms also has cardinality strictly less than κ, which has all products smaller than λ, and in which every object is isomorphic to some product $\Pi_S X$ of a distinguished object X.

The models of a κ-bounded Lawvere theory L are the product-preserving functors from L to Set.

Morphisms between κ-bounded Lawvere theories are product-preserving functors that preserve the distinguished object X. Such functors correspond to homomorphisms of algebraic theories: maps from the operations of one theory to the terms of another that preserve the equations of the first theory according to the equations of the second.

κ-bounded Lawvere theories and their morphisms form a small category Law_κ. This category has coproducts $L + L'$ which correspond to the algebraic theory made by unioning the operations and equations of L and L'. It also has an initial object $\bot$, which is the algebraic theory with no operations and corresponds to the identity monad. Note that Law is $\mathrm{Law}_{\mathbb{N}}$.

3.4 Presheaves

Now that we have a concrete representation for potential ambient monads, we can define our semantics in terms of sets indexed by such monads. More specifically, we can use functors $\mathrm{Set}^{\mathrm{Law}_\kappa}$ as the semantics of our types. By using functors rather than ordinary functions we ensure terms and types at a particular ambient monad can always be mapped naturally to a larger ambient monad. These functors have many convenient properties because they are *presheaves*.

For a category $\mathbb{C}$, $\widehat{\mathbb{C}} = \mathrm{Set}^{\mathbb{C}^{\mathrm{op}}}$ is the category of presheaves over $\mathbb{C}$. Its objects are contravariant functors from $\mathbb{C}$ to Set and its morphisms are natural transformations between such functors.

In these terms, we use $\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}} = \mathrm{Set}^{\mathrm{Law}_\kappa}$ as the basis of the Ambient Monad model. The op in the definition of presheaves and the op in $\mathrm{Law}_\kappa^{\mathrm{op}}$ cancel out, but it is still profitable to think in terms of presheaves because presheaves come with a lot of useful technology.

The Ambient Monad model uses objects from $\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$ to represent types and contexts, and morphisms in $\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$ to represent values.

$$[\![\tau]\!] : \widehat{\mathrm{Law}_\kappa^{\mathrm{op}}} \qquad [\![\Gamma]\!] : \widehat{\mathrm{Law}_\kappa^{\mathrm{op}}} \qquad [\![\Gamma \vdash_V v : \tau]\!] : \mathrm{Hom}_{\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}}([\![\Gamma]\!], [\![\tau]\!])$$

A category of presheaves over a small category, such as Law_κ, is a topos, which gives it many useful properties. As a topos, $\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$ has products and coproducts, which are defined pointwise. We will use these as the semantics of product types and sum types.

$$[\![1]\!] = 1 \qquad [\![\tau_1 \times \tau_2]\!] = [\![\tau_1]\!] \times [\![\tau_2]\!] \qquad [\![\tau_1 + \tau_2]\!] = [\![\tau_1]\!] + [\![\tau_2]\!]$$

The semantics of the corresponding introduction and elimination forms follows the usual categorical semantics of such types.

We also use products for the semantics of contexts:

$$[\![\varepsilon]\!] = 1 \qquad [\![\Gamma; x : \tau]\!] = [\![\Gamma]\!] \times [\![\tau]\!]$$

3.5 Representing computations: The free algebras monad

Our aim is to have a semantics where the meaning of types is indexed by the ambient monad – or more precisely by a representation of the algebraic theory whose free algebras are the ambient monad. In particular, we would like the interpretation of computation types to be applications of the ambient monad.

Given a κ-bounded Lawvere theory L, the underlying sets of its free algebras form a monad T_L on Set. $T_\_$ is functorial and we can use it to construct a monad $\mathcal{T}$ on $\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$ such that:

$$\mathcal{T}(P)(L) = T_L(P(L))$$

We call $\mathcal{T}$ the *free algebras monad*, and it is the representation of computations in the Ambient Monad model.

$$[\![\Gamma \vdash_C c : \tau]\!] : \mathrm{Hom}_{\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}}([\![\Gamma]\!], \mathcal{T}([\![\tau]\!]))$$

$$[\![\Gamma \vdash_C \mathtt{return}\ v : \tau]\!] = \eta^{\mathcal{T}}_{[\![\tau]\!]} \circ [\![\Gamma \vdash_V v : \tau]\!]$$

$$[\![\Gamma \vdash_C \mathtt{let}\, x = c_1\, \mathtt{in}\, c_2 : \tau_2]\!] = \mu^{\mathcal{T}}_{[\![\tau_2]\!]} \circ \mathcal{T}([\![\Gamma; x : \tau_1 \vdash_C c_2 : \tau_2]\!]) \circ t^{\mathcal{T}}_{[\![\Gamma]\!],[\![\tau_1]\!]} \circ \langle id_{[\![\Gamma]\!]}, [\![\Gamma \vdash_C c_1 : \tau_1]\!]\rangle$$

where $\eta^{\mathcal{T}}$, $\mu^{\mathcal{T}}$ and $t^{\mathcal{T}}$ are the unit, multiplication and strength respectively of $\mathcal{T}$.

As a topos, $\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$ has exponentials, which combine with the free algebras monad to give us our semantics for function types.

$$[\![\tau_1 \to \tau_2]\!] = \mathcal{T}([\![\tau_2]\!])^{[\![\tau_1]\!]}$$

3.6 The global modality

For any category $\mathbb{C}$ with a terminal object $\top$, we can define the global modality $\Box : [\widehat{\mathbb{C}}, \widehat{\mathbb{C}}]$ such that: $\Box(P)(L) = P(\top)$. In the case of $\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$ this becomes:

$$\Box(P)(L) = P(\bot)$$

$\Box$ forms a comonad on $\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$. $\Box$ also distributes over both products: $\Box(P \times Q) = (\Box P) \times (\Box Q)$, and sums: $\Box(P + Q) = (\Box P) + (\Box Q)$.

Within the ambient monad model, this amounts to restricting a type to only the elements which do not depend on the ambient monad. This is very similar to the purity comonad of Choudhury and Krishnaswami [7]. In particular,

$$\Box\mathcal{T}(P) \quad = \quad T_\bot \circ \Box P \quad \cong \quad \Box P$$

as elements of the free algebras of the initial algebraic theory are just elements of their carrier sets.

We use the $\Box$ comonad as our semantics for $\Box$ types and $\Box$ bindings.

$$[\![\Box\tau]\!] = \Box[\![\tau]\!]$$

$$[\![\Gamma; x :_\Box \tau]\!] = [\![\Gamma]\!] \times \Box[\![\tau]\!]$$

$$[\![\Gamma; x :_\Box \tau \vdash_V x : \tau]\!] = \epsilon^{\Box}_{[\![\tau]\!]} \circ \pi_2$$

$$[\![\Gamma \vdash_V \mathtt{box}\, v : \Box\tau]\!] = \Box[\![\Gamma/\Box \vdash_V v : \tau]\!] \circ \delta^{\Box}_{[\![\Gamma/\Box]\!]} \circ !_{\Gamma/\Box}$$

$$[\![\Gamma \vdash_V \mathtt{let(box}\, x) = v_1\, \mathtt{in}\, v_2 : \tau_2]\!] = [\![\Gamma; x :_\Box \tau_1 \vdash_V v_2 : \tau_2]\!] \circ \langle id_{[\![\Gamma]\!]}, [\![\Gamma \vdash_V v_1 : \Box\tau_1]\!]\rangle$$

$$[\![\Gamma \vdash_V \mathtt{run}(c) : \tau]\!] = \epsilon^{\Box}_{[\![\tau]\!]} \circ \rho_{[\![\tau]\!]} \circ \Box[\![\Gamma/\Box \vdash_C c : \tau]\!] \circ \delta^{\Box}_{[\![\Gamma/\Box]\!]} \circ !_{\Gamma/\Box}$$

where $\epsilon^{\Box}$ and $\delta^{\Box}$ are the counit and comultiplication of $\Box$, $!_{\Gamma/\Box} : \Gamma \to \Gamma/\Box$ is the morphism that projects the global bindings from Γ, and ρ_P is the isomorphism from $\Box\mathcal{T}(P)$ to $\Box P$.

3.7 Reflecting operations: The Yoneda embedding

We want to be able to reflect operations into the ambient monad. This requires some way to represent operations of the ambient monad. We can do that by using the *Yoneda embedding*. For some category $\mathbb{C}$, the Yoneda embedding $\yen(\_)$ is a functor that maps objects $C : \mathbb{C}$ to the presheaf of the morphisms into it:

$$\yen(C) = \mathrm{Hom}_{\mathbb{C}}(\_, C)$$

It is a full and faithful embedding of $\mathbb{C}$ into $\widehat{\mathbb{C}}$.

In our setting, this means that for each κ-bounded Lawvere theory L there is a presheaf $\text{ɬ}(L)$ in $\widehat{\text{Law}_\kappa^{\text{op}}}$ such that:

$$\begin{aligned}\text{ɬ}(L)(L') &= \text{Hom}_{\text{Law}_\kappa}(L, L') \\ \text{ɬ}(L)(l') &= \lambda f.\ l' \circ f\end{aligned}$$

In other words, $\text{ɬ}(L)$ gives you the set of mappings from L into the current ambient monad.

For an operation $op : P \rightarrowtail A$ of a Lawvere theory L, we can build a morphism:

$$\begin{aligned}op_{\mathcal{T}} &: \text{Hom}_{\widehat{\text{Law}_\kappa^{\text{op}}}}(\text{ɬ}(L) \times \Box P, \mathcal{T}(\Box A)) \\ (op_{\mathcal{T}})_{L'} &= \lambda(y, p).(y(op))_{T_{L'}}(p, \eta_A^{T_{L'}})\end{aligned}$$

which uses an element of $\text{ɬ}(L)$ to map op to a term of the ambient monad and then uses that to construct a term in the free algebras monad.

We use $\text{ɬ}(L)$ as the semantics for our effect capabilities, and $op_{\mathcal{T}}$ as the semantics for reflection.

$$\begin{aligned}&[\![\text{ɬ}(\Sigma)]\!] = \text{ɬ}([\![\Sigma]\!]) \\ &[\![\Gamma \vdash_C \Uparrow(v_1(op))(v_2) : \Box\, \tau_a]\!] = \\ &\quad op_{\mathcal{T}} \circ \langle [\![\Gamma \vdash_V v_1 : \text{ɬ}(\Sigma)]\!], \Box[\![\Gamma/\Box \vdash_V v_2 : \tau_p]\!] \circ \delta^{\Box}_{[\![\Gamma/\Box]\!]} \circ !_{\Gamma/\Box} \rangle\end{aligned}$$

where $[\![\Sigma]\!]$ is the κ-bounded Lawvere theory corresponding to the signature Σ. If Σ is:

$$\{\, op_1 : \tau_{p_1} \rightarrowtail \tau_{a_1}\, ;\, \ldots\, ;\, op_n : \tau_{p_n} \rightarrowtail \tau_{a_n}\, \}$$

then $[\![\Sigma]\!]$ is the κ-bounded Lawvere theory for:

$$\{\, op_1 : [\![\tau_{p_1}]\!](\bot) \rightarrowtail [\![\tau_{a_1}]\!](\bot)\, ;\, \ldots\, ;\, op_n : [\![\tau_{p_n}]\!](\bot) \rightarrowtail [\![\tau_{a_n}]\!](\bot)\, \}$$

Note that by applying the types to the initial theory $\bot$ we restrict them to their global elements: those that do not depend on the current ambient monad.

The Yoneda embedding preserves finite products, which for our case means:

$$\text{ɬ}(L + L') \cong \text{ɬ}(L) \times \text{ɬ}(L')$$

which we use for the semantics of $\texttt{out}_L$ and $\texttt{out}_R$.

3.8 Representing effectful computations: The Yoneda lemma

We can represent computations in a specific monad using the Yoneda lemma. The Yoneda lemma says that, for any $P : \widehat{\mathbb{C}}$ and $C : \mathbb{C}$, there is an isomorphism:

$$\text{Hom}_{\widehat{\mathbb{C}}}(\text{ɬ}(C), P) \cong P(C)$$

Applying that to the composition of the free algebras monad and the global modality we get:

$$\text{Hom}_{\widehat{\text{Law}_\kappa^{\text{op}}}}(\text{ɬ}(L), \mathcal{T}(\Box\, \tau)) \cong \mathcal{T}(\Box\, \tau)(L) = T_L(\tau(\bot))$$

In terms of our semantics, this means that computations of type $\Box\, \tau$ with a single free variable of type $\text{ɬ}(\Sigma)$ are isomorphic to values of the free algebra of $[\![\Sigma]\!]$ at $[\![\tau]\!](\bot)$.

3.9 Effect reification and inductive types

The essence of algebraic effect handlers is taking a computation in $T_{L+\Sigma}(S)$, the free algebras of the sum of an algebraic theory L and an algebraic signature Σ, splitting out and handling Σ, leaving a computation in only the algebraic theory $T_L(S')$.

The key to this process is the following lemma [15].

Lemma 3.7. *For any κ-bounded Lawvere theory L and algebraic signature Σ*

$$T_{L+\Sigma}(S) \cong \mu X.T_L(S + F_\Sigma(X))$$

where $\mu X.G(X)$ is the initial algebra of some endofunctor G. This lemma is crucial because it shows that computations using both ambient effects L and the signature Σ can be reorganized as a recursive structure that separates the two effect sources – exactly what effect handlers need.

Effect reification works by exposing $\mu X.T_L(S+F_\Sigma(X))$ to the user via an inductive type. Presheaves over small categories, such as $\mathrm{Law}_\kappa^{\mathrm{op}}$, have W-types, so we can construct the required initial algebra as a presheaf. Given an algebraic signature Σ and a presheaf P, we define the following family of initial algebras:

$$W_\Sigma : [\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}, \widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}]$$
$$W_\Sigma(P) = \mu X.((\Box P) + F_\Sigma \circ \mathcal{T} X)$$

and observe the following variation on Lemma 3.7.

Lemma 3.8. *For any κ-bounded Lawvere theory L and algebraic signature Σ*

$$T_{L+\Sigma}(P(\bot)) \cong \mathcal{T}(W_\Sigma(P))(L)$$

We use these initial algebras as the semantics of W_Σ:

$$[\![W_\Sigma(\tau)]\!] = W_{[\![\Sigma]\!]}([\![\tau]\!])$$

Exponentials in categories of presheaves are defined such that:

$$Q^P(C) \cong \mathrm{Hom}_{\widehat{\mathbb{C}}}(\text{ɬ}(C) \times P, Q)$$

Applying this to $\mathcal{T}(\Box P)^{\text{ɬ}(L)}$ and then applying the Yoneda Lemma we get:

$$\begin{aligned}
\mathcal{T}(\Box P)^{\text{ɬ}(L)}(L') &\cong \mathrm{Hom}_{\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}}(\text{ɬ}(L') \times \text{ɬ}(L), \mathcal{T}(\Box P)) \\
&\cong \mathrm{Hom}_{\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}}(\text{ɬ}(L' + L), \mathcal{T}(\Box P)) \\
&\cong \mathcal{T}(\Box P)(L' + L) \\
&\cong T_{(L'+L)}(P(\bot))
\end{aligned}$$

In the case where L is a signature Σ, we can further apply Lemma 3.8 and, observing that all the steps in this proof were natural in L', produce this isomorphism:

$$\mathcal{T}(W_\Sigma(P)) \cong \mathcal{T}(\Box P)^{\text{ɬ}(\Sigma)}$$

which we call $\Downarrow_\Sigma$.

$\Downarrow_\Sigma$ gives us our semantics for reification.

$$[\![\Gamma \vdash_C \Downarrow(\ell.\, c) : W_\Sigma(\tau)]\!] = \Downarrow_{\Sigma[\![\tau]\!]} \circ \lambda([\![\Gamma;\, \ell : \Sigma \vdash_C c : \Box\tau]\!])$$

```
module type Sig = sig type 'a t end

module Continuation(E : Sig) : sig
  type ('a, 'b) t
end

module Term (E : Sig) : sig
  type 'a t =
    | Return : 'a @@ global -> 'a t
    | Perform : 'r E.t @@ global * ('r, 'a) Continuation(E).t -> 'a t
end

module Handler(E : Sig) : sig
  type t
end
```

Fig. 7. Effect reflection types

3.10 Properties of the denotational semantics

The denotational semantics is sound:

Theorem 3.9.

$$\Gamma \vdash_V v_1 : \tau \wedge v_1 \rightsquigarrow_V v_2 \implies [\![\Gamma \vdash_V v_1 : \tau]\!] = [\![\Gamma \vdash_V v_2 : \tau]\!]$$

and adequate for contextual equality $\cong_{\text{ctx}}$:

Theorem 3.10.

$$[\![\vdash_V v_1 : \tau]\!] = [\![\vdash_V v_2 : \tau]\!] \implies \vdash_V v_1 \cong_{\text{ctx}} v_2 : \tau$$

We present the full semantics, precisely define contextual equality, and prove these properties in Appendix E.

4 EFFECT REFLECTION IN PRACTICE

We provide effect reflection as a simple library API in OxCaml based on OxCaml's support for locality (see Section 1.5). In this section we describe this API and its implementation.

4.1 Effect reflection interface

The types of the effect reflection interface are shown in Fig. 7. These types are higher-kinded: they are parameterized by type constructors representing the operations of an algebraic effect. In OxCaml, such types must be built using functors. We use a simple module type `Sig` to act as the parameter type for these functors.

The `'a Term(E).t` type corresponds to $W_\Sigma(\tau)$ and represents terms in the free algebra of `E`.

The `Handler(E).t` type corresponds to $\lambda(\Sigma)$ and represents the capability to perform operations from `E`.

`('a, 'b) Continuation(E).t` is equivalent to `'a -> 'b Term(E).t` when global and equivalent to `'a -> 'b Term(E).t @ local` when local. This ensures that the global modality distributes over `Term(E).t`, which is an important property of W_Σ. The need for this type is an artifact of OxCaml's approach to locality: the global modality does not distribute over the function arrow in OxCaml.

```
module Reflection (E : Sig) : sig
  val reify :
    (Handler(E).t @ local -> 'a) @ local -> 'a Term(E).t @ local

  val reify_global : (Handler(E).t @ local -> 'a) -> 'a Term(E).t

  val perform : 'a E.t -> (Handler(E).t @ local -> 'a)

  val continue :
    ('a, 'b) Continuation(E).t @ local -> 'a -> 'b Term(E).t @ local

  val continue_global : ('a, 'b) Continuation(E).t -> 'a -> 'b Term(E).t
end
```

Fig. 8. Effect reflection interface

```
module Sum (L : Sig) (R : Sig) : sig
  type 'a t =
    | Left : 'a L.t -> 'a t
    | Right : 'a R.t -> 'a t
end

module Select(L : Sig) (R : Sig) : sig
  val outl : Handler(Sum(L)(R)).t @ local -> Handler(L).t @ local

  val outr : Handler(Sum(L)(R)).t @ local -> Handler(R).t @ local
end
```

Fig. 9. Effect selection

The main piece of the API is a family of isomorphisms:

```
'a Term(E).t  ≅  Handler(E).t @ local -> 'a
```

which we expose as three functions: `reify`, `reify_global` and `perform`. These are provided by the `Reflection` functor (Fig. 8), along with the functions for continuing a continuation.

`reify` and `reify_global` implement the right-to-left direction of the isomorphism. We need two separate functions because OxCaml currently lacks any form of locality polymorphism. `perform` is equivalent to the left-to-right direction of the isomorphism. It reflects individual operations as opposed to full terms, which is a more convenient primitive in practice.

The final piece of the interface is the support for selection from effect capabilities for sums of signatures (i.e. $\texttt{out}_L$ and $\texttt{out}_R$). These are provided via `Sum` and `Select` functions shown in Fig. 9.

4.2 Effect reflection implementation

We can implement the effect reflection interface directly in terms of OCaml's untracked effect handlers [24]. The entire implementation is given in Appendix F. The trick is to generate a fresh effect constructor for each reification, pass that constructor around as part of the `handler`, and then use it to perform operations during reflection. Performing effects involves a single indirect

```
type ints = Finished | More of int * (unit -> ints)

let handle_gen f =
  let rec loop : unit Term(Gen).t -> ints = function
    | Return () -> Finished
    | Perform(Gen i, k) ->
        More(i, fun () -> loop (Gen_eff.continue_global k ()))
  in
  loop (Gen_eff.reify_global f)
```

Fig. 10. Handler for integer generators

```
let handle_await f =
  let rec loop : 'a Term(Await).t -> 'a Deferred.t = function
    | Return x -> Deferred.return x
    | Perform(Await d, k) ->
        Deferred.bind d (fun x -> loop (Await_eff.continue_global k x))
  in
  loop (Await_eff.reify_global f)
```

Fig. 11. Handler for Asynchronous I/O

call and then an underlying perform operation, so its performance is essentially the same as the performance of OCaml's algebraic effects.

For various practical reasons, OCaml's algebraic effects are restricted to only allow continuations to be resumed a single time. This rules out interesting effects like backtracking search, but improves the ability to reason about the interaction of effects and resources.

There is no fundamental reason that effect reflection should be restricted to only one-shot continuations, so for the purposes of this paper we ignore the restriction. However, for the same reasons as OCaml, our actual implementation is restricted to one-shot continuations. Note that OxCaml's `once` mode [19] allows us to enforce this restriction statically.

4.3 Example: Generators

A more realistic use of effect reflection than local state is a *generator* effect. This effect has a single operation `Gen` from `int` to `unit`, along with a handler that turns computations using `Gen` into streams of integers, shown in Fig. 10. Unlike the state handler, here we must use `reify_global` rather than `reify` because our intention is that the resulting stream of integers be a global value that can escape the current region. This in turn forces us to require that the handled function be global.

4.4 Example: Asynchronous I/O

Perhaps the most common use case for algebraic effects is for concurrency and asynchronous I/O. Implementing asynchronous I/O from scratch is beyond the scope of this paper, but we can build a simple version on top of an existing monadic concurrency library (Fig. 11). Assuming we have a monadic future type `'a Deferred.t` we can write concurrent code in direct style by using an effect with a single `Await` operation that waits for a future to be completed.

```
module Eff = Reflection(Sum(Await)(Gen))
module Sel = Select(Await)(Gen)
type aints = Finished | More of int * (Handler(Await).t @ local -> aints)

let handle_gen_in_await f ah =
  let rec loop ah : unit Term(Sum(Await)(Gen))).t -> aints = function
    | Return () -> Finished
    | Perform(Left op, k) -> loop ah (Eff.continue_global k (Await_eff.perform o
    | Perform(Right(Gen i), k) ->
        More(i, fun ah -> loop ah (Eff.continue_global k ()))
  in
  loop ah (Eff.reify_global (fun h -> f (Sel.outl h) (Sel.outr h)))
```

Fig. 12. Handler for generators that can use asynchronous I/O

4.5 Nested effect handlers

A key feature of algebraic effects is how easily they compose. We can use state whilst using asynchronous I/O by simply nesting their handlers:

```
handle_await (fun ah ->
  handle_state i (fun sh ->
    List.iter (fun fl ->
      let s = await ah (File.read fl) in
      set sh (int_of_string s + get sh)) files;
    get sh))
```

This relies on the fact that `handle_state` uses `reify` rather than `reify_global` and so is able to operate on a local computation that closes over the local await handler value `ah`.

4.6 Effect sums and effect forwarding

If we tried to do something similar using our `handle_gen` handler instead of `handle_state`, then we would get an error complaining that the await handler `ah` would escape its scope. We could change the definition of `handle_gen` to use `reify` instead of `reify_global`, but then we would lose the property that the generator we produce can be passed around freely.

Instead we need to build a handler that can *forward* the `Await` operation to different effect handlers at different points in the program's execution. This more general form of composition requires the selection operations `outl` and `outr`. Using these we can write our desired handler for the `Gen` effect by instead writing a handler for `Sum(Await)(Gen)` that forwards all the `Await` operations on to another handler (Fig. 12).

Explicit effect forwarding is required by all systems based on effect capabilities. Examples like this one fundamentally require that performing an `Await` operation does work proportional to the number of handlers it is forwarded through. Effect capabilities only ever do a constant amount of work to perform operations and so cannot handle such examples directly. Traditional effect handlers, on the other hand, always do work proportional to the number of nested handlers, allowing them to support such examples directly but paying a higher cost for the cases that do not need forwarding.

5 RELATED WORK

5.1 Lightweight effect polymorphism

There have been a number of efforts to avoid the complexity of effect polymorphism by treating effect annotations as relevant to some ambient effect. Frank [8] proposed that higher-order functions should state the effects available to their arguments relative to the effects that they perform. This allows functions like `List.map` to have types that do not mention effects. However, in Frank, this was merely syntactic sugar: under the hood the type for `List.map` was still using effect polymorphism.

Effekt [6] takes the idea further than just syntax, with all functions stating their effects relative to the current set of available effects. However, it requires that functions be second-class values to ensure that they don't escape the scope of a handler that they might be relying on.

More recent work on Effekt [5] adds support for turning second-class functions into first-class values by wrapping them in a modality that tracks which effect capabilities the function uses. There is no lightweight effect polymorphism for these wrapped values though, so any code which places its functions inside of data structures must fall back to ordinary effect polymorphism. This approach also requires native support for these effect capability modalities.

Modal effect types [25] also make effect annotations relative to the current set of available effects. This system uses modal effect types to track the difference between the current available effects and the effects required by a value. This avoids needing to treat functions as second class, but does still require specific effect support in the type system.

5.2 Lexical and named effect handlers

Effect capabilities are closely related to *lexical effect handlers* in systems such as Biernacki et al. [4]. Lexical effect handlers have a value that represents the effect handler, similar to $\lightning(\Sigma)$ values, but these values are second class and require their own special typing rules. These systems still use absolute effect annotations that indicate all the effects used by a computation.

Named effect handlers [26] also have values that represent effect handlers. Rather than make these values second class, they use higher-rank polymorphism to emulate locality tracking in much the same way as the `ST` monad in Haskell. This still requires using an effect system to track which named effects are used by a computation. Using higher-rank polymorphism prevents global type inference, requiring complex inference that extends beyond classic Hindley-Milner, and demanding a higher annotation burden on the programmer.

5.3 Monadic reflection

Effect reflection's presentation as reification and reflection operations is very similar to monadic reflection [9]. Monadic reflection allows arbitrary monads to be reified and reflected. As arbitrary monads do not always have well-defined sums, the user must also define *glue* functions that describe the interaction between the monad being reified and the current computation monad.

Monadic reflection also relies on an effect system to track the monads being used. By using locality and $\lightning(\Sigma)$ values we avoid the need to have an effect system.

5.4 Semantics of algebraic effect handlers

Hyland et al. [15] investigate combining algebraic effects, including the key property of sums of algebraic theories (our Lemma 3.7) on which handlers are based.

The original work on algebraic effect handlers [21, 22] gives a semantics in terms of free algebras but the semantics work with a fixed algebraic theory: handlers do not actually change the ambient monad. Our Ambient Monad model naturally accommodates changing the ambient monad.

Bauer and Pretnar [3] give a denotational semantics for Eff but it is untyped. The same authors [2] also give a second denotational semantics for Eff that is typed, but where effects are treated as extrinsic to the semantics.

Forster et al. [11] give a set-theoretic denotational semantics of a calculus with algebraic effect handlers. It is typed and represents effects using their free algebras. However, their calculus supports neither nesting effect handlers nor effect polymorphism, which means the handlers described are not modular or composable.

5.5 Comonadic purity

Our use of a comonadic global modality to track values which do not depend on the ambient monad is essentially the same as the purity comonad described by Choudhury and Krishnaswami [7]. Their semantic description of their comonad is very different – based on a semantics of capabilities – but the typing rules, and the practical result, are the same.

5.6 Locality and presheaves

The Ambient Monad model is closely related to presheaf semantics for higher-order abstract syntax [14]. In both cases presheaves are used to index the semantics of types and terms by some implicit notion of context, with the Yoneda embedding used to access this context and the global modality used to control dependence on it.

6 FUTURE WORK

There are a number of interesting avenues for future work.

Reification with equations. In this paper we only support reifying effect signatures with no equations as W-types. It should be possible to extend this to reifying arbitrary effect theories as QW-types [10]. This would require working in a base language that can support such types, e.g. Observational Type Theory [1, 23], and generally extending the scheme to dependent types. It would also require using something more general than Lemma 3.7 for representing the sum of two algebraic theories.

Reference types as effect capabilities. In this paper we require that the arities of operations be global. That restriction could be lifted to allow local values to be returned by operations. That would be sufficient to represent references as an effect: the allocation operation would insert a new state effect handler and return its effect capability as the reference. Such references would be restricted to global values, and in particular could not hold other references.

Separation and commuting effects. In addition to having a sum, κ-bounded Lawvere theories L and L' also have a tensor product $L \otimes L'$ [15]. Like the sum, this has the operations and equations of both L and L', but unlike the sum it also includes additional equations that ensure that every operation of L commutes with every operation of L'. This tensor product gives us a symmetric monoidal structure on Law_κ. Presheaves over the opposite of a symmetric monoidal category are the standard categorical semantics of the logic of Bunched Implications [20], which is the basis of the assertion language in Separation Logic. This means we can use the Ambient Monad model as a model of languages that have a notion of separation. Operations on two separated effect capabilities would be known to commute. For the effect capabilities corresponding to reference types, described above, this notion of separation coincides with the usual notion of separation for references.

REFERENCES

[1] Thorsten Altenkirch, Conor McBride, and Wouter Swierstra. 2007. Observational equality, now!. In *Proceedings of the 2007 workshop on Programming languages meets program verification*. 57–68.

[2] Andrej Bauer and Matija Pretnar. 2014. An effect system for algebraic effects and handlers. *Logical methods in computer science* 10 (2014).

[3] Andrej Bauer and Matija Pretnar. 2015. Programming with algebraic effects and handlers. *Journal of logical and algebraic methods in programming* 84, 1 (2015), 108–123.

[4] Dariusz Biernacki, Maciej Piróg, Piotr Polesiuk, and Filip Sieczkowski. 2019. Binders by day, labels by night: effect instances via lexically scoped handlers. *Proceedings of the ACM on Programming Languages* 4, POPL (2019), 1–29.

[5] Jonathan Immanuel Brachthäuser, Philipp Schuster, Edward Lee, and Aleksander Boruch-Gruszecki. 2022. Effects, capabilities, and boxes: from scope-based reasoning to type-based reasoning and back. *Proceedings of the ACM on Programming Languages* 6, OOPSLA1 (2022), 1–30.

[6] Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. 2020. Effects as capabilities: effect handlers and lightweight effect polymorphism. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–30.

[7] Vikraman Choudhury and Neel Krishnaswami. 2020. Recovering purity with comonads and capabilities. *Proceedings of the ACM on Programming Languages* 4, ICFP (2020), 1–28.

[8] Lukas Convent, Sam Lindley, Conor McBride, and Craig McLaughlin. 2020. Doo bee doo bee doo. *Journal of Functional Programming* 30 (2020), e9.

[9] Andrzej Filinski. 1999. Representing layered monads. In *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 175–188.

[10] Marcelo P Fiore, Andrew M Pitts, and SC Steenkamp. 2020. Constructing infinitary quotient-inductive types. In *International Conference on Foundations of Software Science and Computation Structures*. Springer, 257–276.

[11] Yannick Forster, Ohad Kammar, Sam Lindley, and Matija Pretnar. 2017. On the expressive power of user-defined effects: Effect handlers, monadic reflection, delimited control. *Proceedings of the ACM on Programming Languages* 1, ICFP (2017), 1–29.

[12] Jean-Yves Girard, Paul Taylor, and Yves Lafont. 1989. *Proofs and types*. Vol. 7. Cambridge university press Cambridge.

[13] Daniel Hillerström and Sam Lindley. 2016. Liberating effects with rows and handlers. In *Proceedings of the 1st International Workshop on Type-Driven Development*. 15–27.

[14] Jason ZS Hu, Brigitte Pientka, and Ulrich Schöpp. 2022. A category theoretic view of contextual types: From simple types to dependent types. *ACM Transactions on Computational Logic* 23, 4 (2022), 1–36.

[15] Martin Hyland, Gordon Plotkin, and John Power. 2006. Combining effects: Sum and tensor. *Theoretical computer science* 357, 1-3 (2006), 70–99.

[16] Oleg Kiselyov and Hiromi Ishii. 2015. Freer monads, more extensible effects. *ACM SIGPLAN Notices* 50, 12 (2015), 94–105.

[17] Daan Leijen. 2014. Koka: Programming with Row Polymorphic Effect Types. *Electronic Proceedings in Theoretical Computer Science* 153 (June 2014), 100–126. https://doi.org/10.4204/eptcs.153.8

[18] PaulBlain Levy, John Power, and Hayo Thielecke. 2003. Modelling environments in call-by-value programming languages. *Information and computation* 185, 2 (2003), 182–210.

[19] Anton Lorenzen, Leo White, Stephen Dolan, Richard A Eisenberg, and Sam Lindley. 2024. Oxidizing OCaml with modal memory management. *Proceedings of the ACM on Programming Languages* 8, ICFP (2024), 485–514.

[20] Peter W O'Hearn and David J Pym. 1999. The logic of bunched implications. *Bulletin of Symbolic Logic* 5, 2 (1999), 215–244.

[21] Gordon Plotkin and Matija Pretnar. 2009. Handlers of algebraic effects. In *European Symposium on Programming*. Springer, 80–94.

[22] Gordon D Plotkin and Matija Pretnar. 2013. Handling algebraic effects. *Logical methods in computer science* 9 (2013).

[23] Loïc Pujet and Nicolas Tabareau. 2022. Observational equality: now for good. *Proceedings of the ACM on Programming Languages* 6, POPL (2022), 1–27.

[24] K. C. Sivaramakrishnan, Stephen Dolan, Leo White, Tom Kelly, Sadiq Jaffer, and Anil Madhavapeddy. 2021. Retrofitting effect handlers onto OCaml. In *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021*, Stephen N. Freund and Eran Yahav (Eds.). ACM, 206–221. https://doi.org/10.1145/3453483.3454039

[25] Wenhao Tang, Leo White, Stephen Dolan, Daniel Hillerström, Sam Lindley, and Anton Lorenzen. 2025. Modal effect types. *Proceedings of the ACM on Programming Languages* 9, OOPSLA1 (2025), 1130–1157.

[26] Ningning Xie, Youyou Cong, Kazuki Ikemori, and Daan Leijen. 2022. First-class names for effect handlers. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022), 30–59.

A SUBJECT REDUCTION

In this section we prove subject reduction for our operational semantics. We start by proving some general lemmas that we'll need in the proof. Note that we will refer to a property that applies to both value and computation terms, and which must be shown by mutual induction, as a single lemma.

A.1 Context division lemmas

We will need some simple lemmas describing the behaviour of the context division operator $\Gamma/\square$, all of which follow directly by induction on the context.

Lemma A.1.

$$(\Gamma_1; x : \tau; \Gamma_2)/\square = (\Gamma_1; \Gamma_2)/\square$$

Lemma A.2.

$$(\Gamma_1; \ell : \Sigma; \Gamma_2)/\square = (\Gamma_1; \Gamma_2)/\square$$

Lemma A.3.

$$(\Gamma_1; x :_\square \tau; \Gamma_2)/\square = \Gamma_1/\square; x :_\square \tau; \Gamma_2/\square$$

Lemma A.4.

$$\Gamma/\square/\square = \Gamma/\square$$

A.2 Weakening lemmas

We need lemmas stating that we can weaken typing judgements with new variable bindings of both x and ℓ variables.

Lemma A.5.

$$\begin{aligned} \Gamma_1; \Gamma_2 \vdash_V v : \tau_1 \quad &\Longrightarrow \quad \Gamma_1; x : \tau_2; \Gamma_2 \vdash_V v : \tau_1 \\ \Gamma_1; \Gamma_2 \vdash_C c : \tau_1 \quad &\Longrightarrow \quad \Gamma_1; x : \tau_2; \Gamma_2 \vdash_C c : \tau_1 \end{aligned}$$

Lemma A.6.

$$\begin{aligned} \Gamma_1; \Gamma_2 \vdash_V v : \tau \quad &\Longrightarrow \quad \Gamma_1; \ell : \Sigma; \Gamma_2 \vdash_V v : \tau \\ \Gamma_1; \Gamma_2 \vdash_C c : \tau \quad &\Longrightarrow \quad \Gamma_1; \ell : \Sigma; \Gamma_2 \vdash_C c : \tau \end{aligned}$$

The proofs of these follow by mutual induction on the value and computation typing rules. The only aspect that isn't straightforward is that rules with premises featuring $\Gamma/\square$ rely on A.1 and A.2 to reuse those premises with the weakened context.

We similarly need a lemma stating that we can weaken typing judgements with new global variable bindings.

Lemma A.7.

$$\begin{aligned} \Gamma_1; \Gamma_2 \vdash_V v : \tau_1 \quad &\Longrightarrow \quad \Gamma_1; x :_\square \tau_2; \Gamma_2 \vdash_V v : \tau_1 \\ \Gamma_1; \Gamma_2 \vdash_C c : \tau_1 \quad &\Longrightarrow \quad \Gamma_1; x :_\square \tau_2; \Gamma_2 \vdash_C c : \tau_1 \end{aligned}$$

This again follows by mutual induction on the value and computation typing rules. Rules with premises featuring $\Gamma/\square$ rely on A.3 in order to make use of the inductive hypothesis on those premises.

We also need a lemma stating that we can weaken from $\Gamma/\square$ to Γ.

Lemma A.8.

$$\Gamma_1/\Box;\ \Gamma_2 \vdash_V v : \tau \quad \Longrightarrow \quad \Gamma_1;\ \Gamma_2 \vdash_V v : \tau$$
$$\Gamma_1/\Box;\ \Gamma_2 \vdash_C c : \tau \quad \Longrightarrow \quad \Gamma_1;\ \Gamma_2 \vdash_C c : \tau$$

This follows by induction on Γ_1 via the local weakening lemmas (A.5 and A.6).

A.3 Substitution lemmas

We need a simple lemma stating that substituting unbound variables does nothing, which follows directly from mutual induction on typing rules:

Lemma A.9.

$$\Gamma \vdash_V v_1 : \tau \quad \wedge \quad x \notin \Gamma \quad \Longrightarrow \quad v_1[x\backslash v_2] = v_1$$
$$\Gamma \vdash_C c : \tau \quad \wedge \quad x \notin \Gamma \quad \Longrightarrow \quad c[x\backslash v] = c$$

We need a lemma demonstrating that substitution for local bindings preserves typing.

Lemma A.10.

$$\Gamma_1;\ x : \tau_1;\ \Gamma_2 \vdash_V v_1 : \tau_2 \quad \wedge \quad \Gamma_1;\ \Gamma_2 \vdash_V v_2 : \tau_1 \quad \Longrightarrow \quad \Gamma_1;\ \Gamma_2 \vdash_V v_1[x\backslash v_2] : \tau_2$$
$$\Gamma_1;\ x : \tau_1;\ \Gamma_2 \vdash_C c : \tau_2 \quad \wedge \quad \Gamma_1;\ \Gamma_2 \vdash_V v : \tau_1 \quad \Longrightarrow \quad \Gamma_1;\ \Gamma_2 \vdash_C c[x\backslash v] : \tau_2$$

This follows by mutual induction on the typing rules for v_1 and c. Cases with premises that extend the context rely on weakening (A.5 and A.7) to apply the inductive hypothesis on those premises. Cases with premises that use $/\Box$ rely on A.9 to reuse the original premise.

Similarly, we need a lemma demonstrating that substitution for global bindings preserves typing.

Lemma A.11.

$$\Gamma_1;\ x :_\Box \tau_1;\ \Gamma_2 \vdash_V v_1 : \tau_2 \quad \wedge \quad \Gamma_1;\ \Gamma_2/\Box \vdash_V v_2 : \tau_1 \quad \Longrightarrow \quad \Gamma_1;\ \Gamma_2 \vdash_V v_1[x\backslash v_2] : \tau_2$$
$$\Gamma_1;\ x :_\Box \tau_1;\ \Gamma_2 \vdash_C c : \tau_2 \quad \wedge \quad \Gamma_1;\ \Gamma_2/\Box \vdash_V v : \tau_1 \quad \Longrightarrow \quad \Gamma_1;\ \Gamma_2 \vdash_C c[x\backslash v] : \tau_2$$

This follows by mutual induction on the typing rules for v_1 and c. Cases with premises that extend the context with global bindings rely on weakening (A.7) to apply the inductive hypothesis on those premises. Cases with premises that extend the context with local bindings can already apply the inductive hypothesis without weakening. Cases with premises that use $/\Box$ rely on the idempotence of $/\Box$ (A.4) to apply the inductive hypothesis. The variable case that actually substitutes v_2 uses Lemma A.8 to weaken v_2 to the right context.

A.4 Subject reduction

We prove subject reduction (Theorem 2.4):

$$\Gamma \vdash_V v_1 : \tau \quad \wedge \quad v_1 \rightsquigarrow_V v_2 \quad \Longrightarrow \quad \Gamma \vdash_V v_2 : \tau$$
$$\Gamma \vdash_C c_1 : \tau \quad \wedge \quad c_1 \rightsquigarrow_C c_2 \quad \Longrightarrow \quad \Gamma \vdash_C c_2 : \tau$$

by mutual induction over the two reduction relations. The reduction relations are the least relations closed under the rules in Fig. 5 combined with the obvious congruence rules.

Proof. As all the typing rules are entirely compositional, the cases for the congruence rules all follow directly by induction. This leaves only the rules from Fig. 5.

Note that the typing rules are syntax directed so there is always only a single typing rule that we need to consider.

Products For the product beta rules:

$$\pi_1(v_1, v_2) \rightsquigarrow_V v_1$$
$$\pi_2(v_1, v_2) \rightsquigarrow_V v_2$$

the cases follow directly by inversion on the typing rule.

Sums For the sum rules:

$$\mathsf{case}^m(\mathsf{in}_L\, v_1)\,\{\,\mathsf{in}_L\, x_1 \Rightarrow v_2\,;\,\mathsf{in}_R\, x_2 \Rightarrow v_3\,\} \rightsquigarrow_V v_2[x_1\backslash v_1]$$
$$\mathsf{case}^m(\mathsf{in}_R\, v_1)\,\{\,\mathsf{in}_L\, x_1 \Rightarrow v_2\,;\,\mathsf{in}_R\, x_2 \Rightarrow v_3\,\} \rightsquigarrow_V v_3[x_2\backslash v_1]$$

the cases follow by inversion on the typing rules followed by case splitting on the modality m and applying either the global (A.11) or local (A.10) substitution lemma.

Global modality For the global modality rule:

$$\mathsf{let}(\mathsf{box}\, x) = \mathsf{box}\, v_1\, \mathsf{in}\, v_2 \rightsquigarrow_V v_2[x\backslash v_1]$$

the case follows by inversion on the typing rules followed by the global substitution lemma (A.11).

For the run rule:

$$\mathsf{run}(\mathsf{return}\ v) \rightsquigarrow_V v$$

the case follows by inversion on the typing rules along with weakening via Lemma A.8.

Inductive types For the `rec`/`ret` rule:

$$\mathsf{rec}(\mathsf{ret}(v_s))\,\{\mathsf{ret}(x) \Rightarrow v_e\,;\,\ldots\} \rightsquigarrow_V v_e[x\backslash v_s]$$

the case follows by inversion on the typing rules followed by the global substitution lemma (A.11).

For the `rec`/op rule:

$$\mathsf{rec}(op(v_1,\, v_2))\,\{H_1\,;\,op(x_1, x_2) \Rightarrow v_3\,;\,H_2\} \rightsquigarrow_V$$
$$v_3[x_1\backslash v_1][x_2\backslash \lambda x_a.\ \mathsf{let}\, x_w = v_2\, x_a\, \mathsf{in}\, \mathsf{return}\, (\mathsf{rec}\, x_w\, \{H_1\,;\,op(x_1, x_2) \Rightarrow v_3\,;\,H_2\})]$$

the case follows by inversion on the typing rules followed by the global substitution lemma (A.11) for x_1 and the local substitution lemma (A.10) for x_2.

Functions The case for the function beta rule:

$$(\lambda x.\, c)\, v \rightsquigarrow_C c[x\backslash v]$$

follows by inversion on the typing rules followed by the local substitution lemma (A.10).

Bind The case for the bind/return rule:

$$\mathsf{let}\, x = \mathsf{return}\ v\, \mathsf{in}\, c \rightsquigarrow_C c[x\backslash v]$$

follows by inversion on the typing rules followed by the local substitution lemma (A.10).

The case for the bind commuting rule:

$$\mathsf{let}\, x_2 = \mathsf{let}\, x_1 = \Uparrow(\ell(op))(v)\, \mathsf{in}\, c_1\, \mathsf{in}\, c_2 \rightsquigarrow_C \mathsf{let}\, x_1 = \Uparrow(\ell(op))(v)\, \mathsf{in}\, \mathsf{let}\, x_2 = c_1\, \mathsf{in}\, c_2$$

follows by inversion on the typing rules followed by weakening c_2 by the local binding of x_1.

Reflection The cases for the out_L and out_R rules

$$\Uparrow((\mathsf{out}_L\, v_1)(op))(v_2) \rightsquigarrow_C \Uparrow(v_1(\mathrm{left}(op)))(v_2)$$
$$\Uparrow((\mathsf{out}_R\, v_1)(op))(v_2) \rightsquigarrow_C \Uparrow(v_1(\mathrm{right}(op)))(v_2)$$

follow directly by inversion on the typing rules.

Reification The cases for the $\Downarrow$ rules:

$$\Downarrow(\ell.\ \texttt{return}\,(\texttt{box}\,v)) \rightsquigarrow_C \texttt{return}\ \texttt{ret}(v)$$
$$\Downarrow(\ell_1.\ \Uparrow(\ell_1(op))(v)) \rightsquigarrow_C \texttt{return}\ op(v,\ \lambda(\texttt{box}\,x).\ \texttt{return}\ \texttt{ret}(x))$$
$$\Downarrow(\ell_1.\ \texttt{let}\,x = \Uparrow(\ell_1(op))(v)\,\texttt{in}\,c) \rightsquigarrow_C \texttt{return}\ op(v,\ \lambda x.\ \Downarrow(\ell_1.\,c))$$
$$\Downarrow(\ell_1.\ \Uparrow(\ell_2(op))(v)) \rightsquigarrow_C \texttt{let}(\texttt{box}\,x) = \Uparrow(\ell_2(op))(v)\,\texttt{in}\,\texttt{return}\ \texttt{ret}(x) \qquad (\ell_1 \neq \ell_2)$$
$$\Downarrow(\ell_1.\ \texttt{let}\,x = \Uparrow(\ell_2(op))(v)\,\texttt{in}\,c) \rightsquigarrow_C \texttt{let}\,x = \Uparrow(\ell_2(op))(v)\,\texttt{in}\Downarrow(\ell_1.\,c) \qquad (\ell_1 \neq \ell_2)$$

follow by inversion on the typing rules followed by A.2.

□

B CONFLUENCE

We prove that the reduction relations are confluent:

$$v_1 \rightsquigarrow_V^* v_2 \quad\wedge\quad v_1 \rightsquigarrow_V^* v_3 \quad\Longrightarrow\quad \exists\, v_4.\quad v_2 \rightsquigarrow_V^* v_4 \quad\wedge\quad v_3 \rightsquigarrow_V^* v_4$$
$$c_1 \rightsquigarrow_C^* c_2 \quad\wedge\quad c_1 \rightsquigarrow_C^* c_3 \quad\Longrightarrow\quad \exists\, c_4.\quad c_2 \rightsquigarrow_C^* c_4 \quad\wedge\quad c_3 \rightsquigarrow_C^* c_4$$

As is standard, we start by constructing notions of parallel reduction $\rightsquigarrow\!\!\!\!\rightarrow_V$ and $\rightsquigarrow\!\!\!\!\rightarrow_C$. The inference rules describing parallel reduction are given in Fig. 13 and Fig. 14. We omit the congruence rules. We write $\rightsquigarrow\!\!\!\!\rightarrow_V^*$ and $\rightsquigarrow\!\!\!\!\rightarrow_C^*$ for the transitive closures of $\rightsquigarrow\!\!\!\!\rightarrow_V$ and $\rightsquigarrow\!\!\!\!\rightarrow_C$ respectively.

Parallel reductions are reflexive:

Lemma B.1.

$$v \rightsquigarrow\!\!\!\!\rightarrow_V v \qquad c \rightsquigarrow\!\!\!\!\rightarrow_C c$$

which follows by induction on v and c via the congruence rules.

A single step of ordinary reduction is a valid parallel reduction:

Lemma B.2.

$$v_1 \rightsquigarrow_V v_2 \quad\Longrightarrow\quad v_1 \rightsquigarrow\!\!\!\!\rightarrow_V v_2$$
$$c_1 \rightsquigarrow_C c_2 \quad\Longrightarrow\quad c_1 \rightsquigarrow\!\!\!\!\rightarrow_C c_2$$

This follows by induction on the ordinary reductions. Each rule of the ordinary reduction has a corresponding rule in the parallel reduction that can be used, applying reflexivity of the parallel reduction B.1 on subterms as needed.

A single step of parallel reduction corresponds to a sequence of steps in the ordinary reduction.

Lemma B.3.

$$v_1 \rightsquigarrow\!\!\!\!\rightarrow_V v_2 \quad\Longrightarrow\quad v_1 \rightsquigarrow_V^* v_2$$
$$c_1 \rightsquigarrow\!\!\!\!\rightarrow_C c_2 \quad\Longrightarrow\quad c_1 \rightsquigarrow_C^* c_2$$

This follows by induction on the parallel reductions. Each rule of the parallel reduction relation has a corresponding rule of the ordinary reduction relation. We use transitivity of $\rightsquigarrow^*$ to first apply congruence rules to reduce subterms before applying the actual corresponding rule.

$$\frac{v_1 \rightsquigarrow_V v_1'}{\pi_1(v_1, v_2) \rightsquigarrow_V v_1'} \qquad \frac{v_2 \rightsquigarrow_V v_2'}{\pi_2(v_1, v_2) \rightsquigarrow_V v_2'}$$

$$\frac{v_1 \rightsquigarrow_V v_1' \qquad v_2 \rightsquigarrow_V v_2'}{\mathsf{case}^m(\mathsf{in}_L\, v_1)\,\{\,\mathsf{in}_L\, x_1 \Rightarrow v_2\,;\,\mathsf{in}_R\, x_2 \Rightarrow v_3\,\} \rightsquigarrow_V v_2'[x_1 \backslash v_1']}$$

$$\frac{v_1 \rightsquigarrow_V v_1' \qquad v_3 \rightsquigarrow_V v_3'}{\mathsf{case}^m(\mathsf{in}_R\, v_1)\,\{\,\mathsf{in}_L\, x_1 \Rightarrow v_2\,;\,\mathsf{in}_R\, x_2 \Rightarrow v_3\,\} \rightsquigarrow_V v_3'[x_2 \backslash v_1']}$$

$$\frac{v_1 \rightsquigarrow_V v_1' \qquad v_2 \rightsquigarrow_V v_2'}{\mathtt{let}(\mathtt{box}\, x) = \mathtt{box}\, v_1\, \mathtt{in}\, v_2 \rightsquigarrow_V v_2'[x \backslash v_1']} \qquad \frac{v \rightsquigarrow_V v'}{\mathtt{run}(\mathtt{return}\ v) \rightsquigarrow_V v'}$$

$$\frac{v_1 \rightsquigarrow_V v_1' \qquad v_2 \rightsquigarrow_V v_2'}{\mathtt{rec}(\mathtt{ret}(v_1))\,\{\mathtt{ret}(x) \Rightarrow v_2\,;\,\ldots\} \rightsquigarrow_V v_2'[x \backslash v_1']}$$

$$\frac{v_p \rightsquigarrow_V v_p' \qquad v_k \rightsquigarrow_V v_k' \qquad v_e \rightsquigarrow_V v_e' \qquad \overline{v_j \rightsquigarrow_V v_j'}}{\begin{array}{c}\mathtt{rec}\, op_i(v_p,\, v_k)\,\{\mathtt{ret}(x_r) \Rightarrow v_e\,;\,\overline{op_j(x_{p_j}, x_{k_j}) \Rightarrow v_j}\} \\ \rightsquigarrow_V \\ v_i'[x_{p_i} \backslash v_p'][x_{k_i} \backslash \lambda x_a.\ \mathtt{let}\, x_w = v_k'\, x_a\, \mathtt{in}\, \mathtt{return}\, (\mathtt{rec}\, x_w\, \{\mathtt{ret}(x_r) \Rightarrow v_e'\,;\,\overline{op_j(x_{p_j}, x_{k_j}) \Rightarrow v_j'}\})]\end{array}}$$

Fig. 13. Parallel reduction relations (values)

B.1 Substitution

The above lemmas mean that showing confluence of $\rightsquigarrow_V^*$ is sufficient to show confluence of $\rightsquigarrow^*$. To do that we first need a lemma showing that substitution commutes with parallel reduction:

Lemma B.4.

$$v_1 \rightsquigarrow_V v_3 \quad \wedge \quad v_2 \rightsquigarrow_V v_4 \quad \Longrightarrow \quad v_1[x \backslash v_2] \rightsquigarrow_V v_3[x \backslash v_4]$$
$$c_1 \rightsquigarrow_C c_2 \quad \wedge \quad v_1 \rightsquigarrow_V v_2 \quad \Longrightarrow \quad c_1[x \backslash v_1] \rightsquigarrow_C c_2[x \backslash v_2]$$

Proof. We prove this by mutual induction on the parallel reduction relations on v_1 and c_1.

Variables If the variable is x then this case follows directly from $v_2 \rightsquigarrow_V v_4$, otherwise it follows directly from $v_1 \rightsquigarrow_V v_3$.

Other congruence rules For congruence rules other than the variable rule we apply the same congruence rule and use the inductive hypothesis for any subcomponents.

Reduction rules For reduction rules we apply the same reduction rule, commute any substitutions produced by the reduction with the substitution of x, and then use the inductive hypothesis for any subcomponents.

$$\frac{c \rightsquigarrow_C c' \qquad v \rightsquigarrow_V v'}{(\lambda x.\, c)\, v \rightsquigarrow_C c'[x\backslash v']} \qquad \frac{v \rightsquigarrow_V v' \qquad c \rightsquigarrow_C c'}{\mathtt{let}\, x = \mathtt{return}\, v\, \mathtt{in}\, c \rightsquigarrow_C c'[x\backslash v']}$$

$$\frac{v \rightsquigarrow_V v' \qquad c_1 \rightsquigarrow_C c_1' \qquad c_2 \rightsquigarrow_C c_2'}{\mathtt{let}\, x_2 = \mathtt{let}\, x_1 = \Uparrow(\ell(op))(v)\, \mathtt{in}\, c_1\, \mathtt{in}\, c_2 \rightsquigarrow_C \mathtt{let}\, x_1 = \Uparrow(\ell(op))(v')\, \mathtt{in}\, \mathtt{let}\, x_2 = c_1'\, \mathtt{in}\, c_2'}$$

$$\frac{v_1 \rightsquigarrow_V v_1' \qquad v_2 \rightsquigarrow_V v_2'}{\Uparrow((\mathtt{out}_L\, v_1)(op))(v_2) \rightsquigarrow_C \Uparrow(v_1'(\mathrm{left}(op)))(v_2')}$$

$$\frac{v_1 \rightsquigarrow_V v_1' \qquad v_2 \rightsquigarrow_V v_2'}{\Uparrow((\mathtt{out}_R\, v_1)(op))(v_2) \rightsquigarrow_C \Uparrow(v_1'(\mathrm{right}(op)))(v_2')} \qquad \frac{v \rightsquigarrow_V v'}{\Downarrow(\ell.\, \mathtt{return}\, (\mathtt{box}\, v)) \rightsquigarrow_C \mathtt{return}\, \mathtt{ret}(v')}$$

$$\frac{v \rightsquigarrow_V v'}{\Downarrow(\ell_1.\, \Uparrow(\ell_1(op))(v)) \rightsquigarrow_C \mathtt{return}\, op(v', \lambda(\mathtt{box}\, x).\, \mathtt{return}\, \mathtt{ret}(x))}$$

$$\frac{v \rightsquigarrow_V v' \qquad c \rightsquigarrow_C c'}{\Downarrow(\ell_1.\, \mathtt{let}\, x = \Uparrow(\ell_1(op))(v)\, \mathtt{in}\, c) \rightsquigarrow_C \mathtt{return}\, op(v', \lambda x.\, \Downarrow(\ell_1.\, c'))}$$

$$\frac{v \rightsquigarrow_V v'}{\Downarrow(\ell_1.\, \Uparrow(\ell_2(op))(v)) \rightsquigarrow_C \mathtt{let}(\mathtt{box}\, x) = \Uparrow(\ell_2(op))(v')\, \mathtt{in}\, \mathtt{return}\, \mathtt{ret}(x) \qquad (\ell_1 \neq \ell_2)}$$

$$\frac{v \rightsquigarrow_V v' \qquad c \rightsquigarrow_C c'}{\Downarrow(\ell_1.\, \mathtt{let}\, x = \Uparrow(\ell_2(op))(v)\, \mathtt{in}\, c) \rightsquigarrow_C \mathtt{let}\, x = \Uparrow(\ell_2(op))(v')\, \mathtt{in} \Downarrow(\ell_1.\, c') \qquad (\ell_1 \neq \ell_2)}$$

Fig. 14. Parallel reduction relations (computations)

□

B.2 The diamond property

We can now show confluence of $\rightsquigarrow^*$ by proving the stronger property that $\rightsquigarrow$ satisfies the diamond property:

Lemma B.5.

$$v_a \rightsquigarrow_V v_b \quad \wedge \quad v_a \rightsquigarrow_V v_c \quad \Longrightarrow \quad \exists\, v_d.\ \ v_b \rightsquigarrow_V v_d \quad \wedge \quad v_c \rightsquigarrow_V v_d$$

$$c_a \rightsquigarrow_C c_b \quad \wedge \quad c_a \rightsquigarrow_C c_c \quad \Longrightarrow \quad \exists\, c_d.\ \ c_b \rightsquigarrow_C c_d \quad \wedge \quad c_c \rightsquigarrow_C c_d$$

Proof. We prove this by mutual induction on the parallel reductions to v_b and c_b, followed by inversion of the parallel reductions to v_c and c_c. As all congruence rules operate on different

constructs, and no two reduction rules can apply to the same term, there are only two cases we must consider:

The same rule If both reductions are the same rule then the induction hypotheses give us a value and associated parallel reductions for each subcomponent. We use these values to build v_d/c_d and apply the rule to build the required parallel reductions.

A reduction rule and a congruence rule Cases where one reduction is a reduction rule and the other is a congruence rule all follow the same reasoning. We illustrate this reasoning by describing the case for the `let/return` reduction rule.
We have $c_a = \texttt{let}\, x_l = \texttt{return}\, v_{ar}\, \texttt{in}\, c_{as}$, $c_b = c_{bs}[x_l \backslash v_{br}]$ and $c_c = \texttt{let}\, x_l = v_{cl}\, \texttt{in}\, c_{cs}$, and the rules in question are:

$$\frac{v_{ar} \rightsquigarrow_V v_{br} \qquad c_{as} \rightsquigarrow_C c_{bs}}{\texttt{let}\, x_l = \texttt{return}\, v_{ar}\, \texttt{in}\, c_{as} \rightsquigarrow_C c_{bs}[x_l \backslash v_{br}]}$$

$$\frac{\texttt{return}\, v_{ar} \rightsquigarrow_V v_{cl} \qquad c_{as} \rightsquigarrow_C c_{cs}}{\texttt{let}\, x_l = \texttt{return}\, v_{ar}\, \texttt{in}\, c_{as} \rightsquigarrow_C \texttt{let}\, x_l = v_{cl}\, \texttt{in}\, c_{cs}}$$

All our reduction rules have a single subcomponent that is not a metavariable – in this case it is `return` v – and there are no reduction rules which could apply to that subcomponent. That means we know by inversion that $\texttt{return}\, v_{ar} \rightsquigarrow_V v_{cl}$ must be the congruence rule on `return`, which gives us $v_{cl} = \texttt{return}\, v_{cr}$, and the combined congruence rules amount to:

$$\frac{v_{ar} \rightsquigarrow_V v_{cr} \qquad c_{as} \rightsquigarrow_C c_{cs}}{\texttt{let}\, x_l = \texttt{return}\, v_{ar}\, \texttt{in}\, c_{as} \rightsquigarrow_C \texttt{let}\, x_l = \texttt{return}\, v_{cr}\, \texttt{in}\, c_{cs}}$$

For some of the cases, e.g. the $\Downarrow$/`return` rule, the subcomponent itself has a nested subcomponent that is not a metavariable and this inversion step must be repeated a second time.
By the inductive hypothesis, we know that:

$$\exists\, v_{dr}.\, v_{br} \rightsquigarrow_V v_{dr} \;\wedge\; v_{cr} \rightsquigarrow_V v_{dr}$$

$$\exists\, c_{ds}.\, c_{bs} \rightsquigarrow_C c_{ds} \;\wedge\; c_{cs} \rightsquigarrow_C c_{ds}$$

We take c_d to be $c_{ds}[x_l \backslash v_{dr}]$. We have $c_{bs}[x_l \backslash v_{br}] \rightsquigarrow_C c_{ds}[x_l \backslash v_{dr}]$ using B.4, and we have $\texttt{let}\, x_l = \texttt{return}\, v_{cr}\, \texttt{in}\, c_{cs} \rightsquigarrow_C c_{ds}[x_l \backslash v_{dr}]$ by applying the reduction rule. In some cases there is no substitution and so the lemma is not needed, and in some other cases the result of the reduction is not a metavariable and so we must first apply the relevant congruence rule.

□

C STRONG NORMALIZATION

$$\Gamma \vdash_V v : \tau \quad \Longrightarrow \quad v \in SN_V$$

$$\Gamma \vdash_C c : \tau \quad \Longrightarrow \quad c \in SN_C$$

We prove strong normalization in the standard way for a simply typed calculus via logical relations. In addition to being indexed by types these relations will be parameterized by an *effect context*: a context restricted to bindings of the form $\ell : \Sigma$.

$$\text{effect contexts, } \Delta ::= \ \varepsilon \mid \Delta; \ell : \Sigma$$

We write $\text{locs}(\Gamma)$ for the effect context obtained by removing all the ordinary variable bindings from Γ and keeping only the location bindings.

As in Girard [12], we will use the fact that strongly normalizable terms must have a bound on the length of their possible reduction sequences.

Lemma C.1. *For any strongly normalizable value v (or computation c) there is a number $\nu(v)$ (or $\nu(c)$) which bounds the length of every normalization sequence beginning with v (or c).*

Proof. We can represent the possible normalization sequences as a tree, where the nodes are reducts. Since there are a finite number of subterms of any value this tree must be finitely branching. As the term is strongly normalizing it cannot contain any infinite branches. Thus, by Kőnig's lemma, the whole tree is finite and there must be a $\nu(v)$ that bounds its height. □

We define a pair of mutually defined logical relations $RED_V(\Delta, \tau, \alpha)$ and $RED_C(\Delta, \tau, \alpha)$ indexed by an effect context, a type and an ordinal. The ordinal controls the rank of the well-founded trees corresponding to inductive types and computations. The definitions are given in Fig. 15.

We write $RED_V(\Delta, \tau)$ for $\lim_\alpha RED_V(\Delta, \tau, \alpha)$ and $RED_C(\Delta, \tau)$ for $\lim_\alpha RED_C(\Delta, \tau, \alpha)$. These limits are well-defined because the predicates are monotonic in α.

C.1 Properties of reducibility

We prove the three properties of reducibility referred to as CR1, CR2 and CR3 in Girard [12].

Reducible values and computations are strongly normalizing (CR1):

Lemma C.2.

$$RED_V(\Delta, \tau) \subseteq SN_V$$
$$RED_C(\Delta, \tau) \subseteq SN_C$$

Proof. We prove this by induction on τ. Most of the cases follow directly by the definitions of RED_V and RED_C. The exceptions are pair and function types.

- $\tau_1 \times \tau_2$**:** We have $\pi_1\, v \in SN_V$ and $\pi_2\, v \in SN_V$ and must show that $v \in SN_V$. This holds because if there was an infinite chain of reductions for v then there would be an infinite chain of reductions for $\pi_1\, v$ by congruence.
- $\tau_1 \to \tau_2$**:** We have $\forall v_2 \in RED_V(\Delta, \tau_1).\ v_1\, v_2 \in SN_C$ and must show that $v_1 \in SN_V$. This holds because if there was an infinite chain of reductions for v_1 then there would be an infinite chain of reductions for $v_1\, v_2$ by congruence.

□

Reducibility predicates are compatible with reductions (CR2):

Lemma C.3.

$$v_1 \in RED_V(\Delta, \tau, \alpha) \ \wedge \ v_1 \rightsquigarrow_V^* v_2 \implies v_2 \in RED_V(\Delta, \tau, \alpha)$$
$$c_1 \in RED_C(\Delta, \tau, \alpha) \ \wedge \ c_1 \rightsquigarrow_C^* c_2 \implies c_2 \in RED_C(\Delta, \tau, \alpha)$$

Proof. We prove this by induction on τ. The reasoning for computations is the same for all types so we treat it as a separate case.

1**:** If $v_1 \in SN_V$ then $v_2 \in SN_V$ because an infinite chain in v_2 would produce an infinite chain in v_1.

$$
\begin{array}{rcl}
RED_V(\Delta, 1, \alpha) & = & SN_V \\
RED_V(\Delta, \tau_1 \times \tau_2, \alpha) & = & \{\, v \mid \pi_1\, v \in RED_V(\Delta, \tau_1) \ \wedge\ \pi_2\, v \in RED_V(\Delta, \tau_2) \,\} \\
RED_V(\Delta, \tau_1 + \tau_2, \alpha) & = & \{\, v_1 \in SN_V \mid \forall v_2.\ v_1 \rightsquigarrow_V^* \mathtt{in}_L\, v_2 \implies v_2 \in RED_V(\Delta, \tau_1) \\
 & & \qquad \wedge\ \forall v_2.\ v_1 \rightsquigarrow_V^* \mathtt{in}_R\, v_2 \implies v_2 \in RED_V(\Delta, \tau_2) \,\} \\
RED_V(\Delta, \tau_1 \to \tau_2, \alpha) & = & \{\, v_1 \mid \forall v_2 \in RED_V(\Delta, \tau_1).\ v_1\, v_2 \in RED_C(\Delta, \tau_2) \,\} \\
RED_V(\Delta, \Box\, \tau, \alpha) & = & \{\, v_1 \in SN_V \mid \forall v_2.\ v_1 \rightsquigarrow_V^* \mathtt{box}\, v_2 \implies v_2 \in RED_V(\varepsilon, \tau) \,\} \\
RED_V(\Delta, \lightning(\Sigma_1), \alpha) & = & \{\, v_1 \in SN_V \mid \forall \ell.\ v_1 \rightsquigarrow_V^* \ell \implies \ell : \Sigma_1 \in \Delta \\
 & & \qquad \wedge\ \forall v_2.\ v_1 \rightsquigarrow_V^* \mathtt{out}_L\, v_2 \\
 & & \qquad\quad \implies \exists \Sigma_2.\ \exists \beta < \alpha.\quad v_2 \in RED_V(\Delta, \lightning(\Sigma_1 + \Sigma_2), \beta) \\
 & & \qquad \wedge\ \forall v_2.\ v_1 \rightsquigarrow_V^* \mathtt{out}_R\, v_2 \\
 & & \qquad\quad \implies \exists \Sigma_2.\ \exists \beta < \alpha.\quad v_2 \in RED_V(\Delta, \lightning(\Sigma_2 + \Sigma_1), \beta) \,\} \\
RED_V(\Delta, W_\Sigma(\tau_1), \alpha) & = & \\
 & & \{\, v_1 \in SN_V \mid (\forall v_2.\ v_1 \rightsquigarrow_V^* \mathtt{ret}(v_2) \implies v_2 \in RED_V(\varepsilon, \tau_1)) \\
 & & \qquad \wedge\ (\forall op : \tau_2 \rightarrowtail \tau_3 \in \Sigma, v_3, v_4.\quad v_1 \rightsquigarrow_V^* op(v_3,\, v_4) \\
 & & \qquad\quad \implies v_3 \in RED_V(\varepsilon, \tau_2) \\
 & & \qquad\quad \wedge\ (\forall v_5 \in SN_V.\ (\forall v_6.\ v_5 \rightsquigarrow_V^* \mathtt{box}\, v_6 \implies v_6 \in RED_V(\varepsilon, \tau_3)) \\
 & & \qquad\qquad \implies \exists \beta < \alpha.\quad v_4\, v_5 \in RED_C(\Delta, W_\Sigma(\tau_1), \beta))) \,\} \\
RED_C(\Delta, \tau, \alpha) & = & \\
 & & \{\, c_1 \in SN_C \mid (\forall v_1.\ c_1 \rightsquigarrow_C^* \mathtt{return}\ v_1 \implies v_1 \in RED_V(\Delta, \tau, \alpha)) \\
 & & \qquad \wedge\ (\forall \ell, op, v_2.\quad c_1 \rightsquigarrow_C^* \Uparrow(\ell(op))(v_2) \\
 & & \qquad\quad \implies \exists \Sigma, \tau_2, \tau_3.\ \ell : \Sigma \in \Delta\ \wedge\ op : \tau_2 \rightarrowtail \tau_3 \in \Sigma\ \wedge\ v_2 \in RED_V(\varepsilon, \tau_2)) \\
 & & \qquad \wedge\ (\forall \ell, op, v_3, c_2.\quad c_1 \rightsquigarrow_C^* \mathtt{let}\, x = \Uparrow(\ell(op))(v_3)\ \mathtt{in}\, c_2 \\
 & & \qquad\quad \implies \exists \Sigma, \tau_4, \tau_5.\ \ell : \Sigma \in \Delta\ \wedge\ op : \tau_4 \rightarrowtail \tau_5 \in \Sigma\ \wedge\ v_3 \in RED_V(\varepsilon, \tau_4) \\
 & & \qquad\qquad \wedge\ (\forall v_4 \in SN_V.\ (\forall v_5.\ v_4 \rightsquigarrow_V^* \mathtt{box}\, v_5 \implies v_5 \in RED_V(\varepsilon, \tau_5)) \\
 & & \qquad\qquad\quad \implies \exists \beta < \alpha.\quad c_2[x \backslash v_4] \in RED_C(\Delta, \tau, \beta))) \,\}
\end{array}
$$

Fig. 15. Reducibility predicates

$\tau_1 \times \tau_2$: By congruence we have $\pi_1\, v_1 \rightsquigarrow_V^* \pi_1\, v_2$ and $\pi_2\, v_1 \rightsquigarrow_V^* \pi_2\, v_2$, so $\pi_1\, v_2 \in RED_V(\Delta, \tau_1)$ and $\pi_2\, v_2 \in RED_V(\Delta, \tau_2)$ follow by the induction hypothesis.

$\tau_1 + \tau_2$: Given $v_2 \rightsquigarrow_V^* \mathtt{in}_L\, v_3$ we have $v_1 \rightsquigarrow_V^* \mathtt{in}_L\, v_3$ and thus $v_3 \in RED_V(\Delta, \tau_1)$. Similarly, given $v_2 \rightsquigarrow_V^* \mathtt{in}_R\, v_4$ we have $v_4 \in RED_V(\Delta, \tau_2)$.

$\tau_1 \to \tau_2$: Given $v_3 \in RED_V(\Delta, \tau_1)$, we have that $v_1\, v_3$ reduces to $v_2\, v_3$ by congruence, so $v_2\, v_3 \in RED_C(\Delta, \tau_2)$ by the induction hypothesis.

$\Box\, \tau$: Given $v_2 \rightsquigarrow_V^* \mathtt{box}\, v_3$ we have $v_1 \rightsquigarrow_V^* \mathtt{box}\, v_3$ and thus $v_3 \in RED_V(\varepsilon, \tau)$.

$\lightning(\Sigma)$: Given $v_2 \rightsquigarrow_V^* \ell$, $v_2 \rightsquigarrow_V^* \mathtt{out}_L\, v_3$ or $v_2 \rightsquigarrow_V^* \mathtt{out}_R\, v_3$ then v_1 also reduces to those values and thus they have the required properties.

$W_\Sigma(\tau)$: Given $v_2 \rightsquigarrow_V^* \mathtt{ret}(v_3)$ or $v_2 \rightsquigarrow_V^* op(v_4, v_5)$ then v_1 also reduces to those values and thus they have the required properties.

RED_C: Given $c_2 \rightsquigarrow_C^* \texttt{return}\ v_3$, $c_2 \rightsquigarrow_C^* \Uparrow(\ell(op))(v_4)$ or $c_2 \rightsquigarrow_C^* \texttt{let}\, x = \Uparrow(\ell(op))(v_5)\ \texttt{in}\, c_3$ then c_1 also reduces to those computations and thus they have the required properties.

□

A term is neutral if it is not an introduction form. We consider $\texttt{return}\ v$, $\Uparrow(\ell(op))(v)$ and $\texttt{let}\, x = \Uparrow(\ell(op))(v)\ \texttt{in}\, c$ to be the introduction forms for computations. If a term is neutral, and whenever we reduce it we obtain a term that is reducible, then the original term is reducible (CR3).

Lemma C.4. *For all neutral values v_n and neutral computations c_n:*

$$(\forall v_r.\ v_n \rightsquigarrow_V v_r \implies v_r \in RED_V(\Delta, \tau, \alpha)) \implies v_n \in RED_V(\Delta, \tau, \alpha)$$
$$(\forall c_r.\ c_n \rightsquigarrow_C c_r \implies c_r \in RED_C(\Delta, \tau, \alpha)) \implies c_n \in RED_C(\Delta, \tau, \alpha)$$

Proof. We prove this by induction on τ. The reasoning for computations is the same for all types so we treat it as a separate case.

1: Any chain from v_n must pass through one of the v_r and so must be finite.

$\tau_1 \times \tau_2$: As v_n is neutral, the only reduction rule that applies to $\pi_1\, v_n$ is congruence, so any reduction must be of the form $\pi_1\, v_n \rightsquigarrow_V \pi_1\, v_r$. We have $v_r \in RED_V(\Delta, \tau_1 \times \tau_2, \alpha)$ and thus $\pi_1\, v_r \in RED_V(\Delta, \tau_1)$, so by the induction hypothesis we have $\pi_1\, v_n \in RED_V(\Delta, \tau_1)$. Similarly, we have $\pi_2\, v_n \in RED_V(\Delta, \tau_2)$.

$\tau_1 + \tau_2$: Any reduction $v_n \rightsquigarrow_V^* \texttt{in}_L\, v_i$ must go through one of the v_r because v_n cannot equal $\texttt{in}_L\, v_i$ as it is neutral. From $v_r \rightsquigarrow_V^* \texttt{in}_L\, v_i$ we have $v_i \in RED_V(\Delta, \tau_1)$. Similarly we have that if $v_n \rightsquigarrow_V^* \texttt{in}_R\, v_i$ then $v_i \in RED_V(\Delta, \tau_2)$.

$\tau_1 \to \tau_2$: For any $v_a \in RED_V(\Delta, \tau_1)$, we have by CR1(C.2) that $v_a \in SN_V$. We prove that $v_n\, v_a \in RED_C(\Delta, \tau_2)$ by induction on $\nu(v_a)$. By the outer induction hypothesis it is sufficient to show that any reduct of $v_n\, v_a$ is itself in $RED_C(\Delta, \tau_2)$. As v_n is neutral the only reduction rules that apply to $v_n\, v_a$ are the congruences, so any reduction must be one of:

- $v_n\, v_a \rightsquigarrow_C v_r\, v_a$
- $v_n\, v_a \rightsquigarrow_C v_n\, v_b$ where $v_a \rightsquigarrow_V v_b$

In the first case, we have $v_r \in RED_V(\Delta, \tau_1 \to \tau_2, \alpha)$ and thus $v_r\, v_a \in RED_C(\Delta, \tau_2)$. In the second case, we have $v_b \in RED_V(\Delta, \tau_1)$ by CR2(C.3), and $\nu(v_b) < \nu(v_a)$, so we have $v_n\, v_b \in RED_C(\Delta, \tau_2)$ by the inner induction hypothesis.

$\Box\,\tau$: Any reduction $v_n \rightsquigarrow_V^* \texttt{box}\, v_b$ must go through one of the v_r because v_n cannot equal $\texttt{box}\, v_b$ as it is neutral. From $v_r \rightsquigarrow_V^* \texttt{box}\, v_b$ we have $v_b \in RED_V(\Delta, \tau)$.

$\bot(\Sigma)$: Any reduction $v_n \rightsquigarrow_V^* \ell$, $v_n \rightsquigarrow_V^* \texttt{out}_L\, v_i$ or $v_n \rightsquigarrow_V^* \texttt{out}_R\, v_i$ must go through one of the v_r because v_n cannot equal ℓ, $\texttt{out}_L\, v_i$ or $\texttt{out}_R\, v_i$ as it is neutral. Since they reduce from v_r and $v_r \in RED_V(\Delta, \bot(\Sigma), \alpha)$ they must have the required properties.

$W_\Sigma(\tau)$: Any reduction $v_n \rightsquigarrow_V^* \texttt{ret}(v_t)$ or $v_n \rightsquigarrow_V^* op(v_p, v_a)$ must go through one of the v_r because v_n cannot equal $\texttt{ret}(v_t)$ or $op(v_p, v_a)$ as it is neutral. Since they reduce from v_r and $v_r \in RED_V(\Delta, W_\Sigma(\tau), \alpha)$ they must have the required properties.

RED_C: Any reduction $c_n \rightsquigarrow_C^* \texttt{return}\ v_t$, $c_n \rightsquigarrow_C^* \Uparrow(\ell(op))(v_p)$ or $c_n \rightsquigarrow_C^* \texttt{let}\, x = \Uparrow(\ell(op))(v_p)\ \texttt{in}\, c_k$ must go through one of the c_r because c_n cannot equal $\texttt{return}\ v_t$, $\Uparrow(\ell(op))(v_p)$ or $\texttt{let}\, x = \Uparrow(\ell(op))(v_p)\ \texttt{in}\, c_k$ as it is neutral. Since they reduce from c_r and $c_r \in RED_C(\Delta, \tau, \alpha)$ they must have the required properties.

□

As a corollary, we get that any neutral normal form is reducible. In particular, the variable term x is reducible.

C.2 Effect weakening

Reducibility can have its effect context weakened:

Lemma C.5.

$$RED_V(\Delta, \tau, \alpha) \subseteq RED_V(\Delta;\ \ell : \Sigma, \tau, \alpha)$$

Proof. We prove this by induction on τ and α. Most of the value cases follow immediately by the induction hypotheses as they do not mention Δ otherwise. The effect capability and computation cases follow by the induction hypotheses and the fact that context membership is preserved by weakening. □

C.3 Products

Pair terms are reducible when built from reducible subterms.

Lemma C.6.

$$v_1 \in RED_V(\Delta, \tau_1) \ \wedge\ v_2 \in RED_V(\Delta, \tau_2) \implies (v_1, v_2) \in RED_V(\Delta, \tau_1 \times \tau_2)$$

Proof. By induction on $\nu(v_1) + \nu(v_2)$. $\pi_1(v_1, v_2)$ is neutral so we can use CR3(C.4). There are three possibilities for a reduct of $\pi_1(v_1, v_2)$.

- It can reduce by the π_1 reduction rule to v_1.
- It can reduce by congruence to $\pi_1(v_3, v_2)$ where $v_1 \rightsquigarrow_V v_3$.
- It can reduce by congruence to $\pi_1(v_1, v_4)$ where $v_2 \rightsquigarrow_V v_4$.

The first case is reducible by hypothesis. The second and third cases hold by the inductive hypothesis using CR2(C.3). The same reasoning applies to $\pi_2(v_1, v_2)$. □

Projection terms are reducible when built from reducible subterms.

Lemma C.7.

$$v \in RED_V(\Delta, \tau_1 \times \tau_2) \implies \pi_1\, v \in RED_V(\Delta, \tau_1)$$
$$v \in RED_V(\Delta, \tau_1 \times \tau_2) \implies \pi_2\, v \in RED_V(\Delta, \tau_2)$$

Proof. Follows directly from the definition of $RED_V(\Delta, \tau_1 \times \tau_2)$ □

C.4 Functions

Abstraction terms are reducible when built from a reducible subterm.

Lemma C.8.

$$(\forall v_a \in RED_V(\Delta, \tau_1).\ c_b[x \backslash v_a] \in RED_C(\Delta, \tau_2)) \implies \lambda x.\, c_b \in RED_V(\Delta, \tau_1 \to \tau_2)$$

Proof. We need to show that $\lambda x.\, c_b\, v_a \in RED_C(\Delta, \tau_2)$ for any $v_a \in RED_V(\Delta, \tau_1)$. We proceed by induction on $\nu(c_b) + \nu(v_a)$. As $\lambda x.\, c_b\, v_a$ is neutral, we can use CR3(C.4). There are three possibilities for a reduct of $\lambda x.\, c_b\, v_a$.

- It can reduce by beta reduction to $c_b[x \backslash v_a]$.
- It can reduce by congruence to $\lambda x.\, c_d\, v_a$ where $c_b \rightsquigarrow_C c_d$.
- It can reduce by congruence to $\lambda x.\, c_b\, v_e$ where $v_a \rightsquigarrow_V v_e$.

The first case is reducible by hypothesis. The second and third cases are reducible by the inductive hypothesis using CR2(C.3). □

Application terms are reducible when built from reducible subterms.

Lemma C.9.

$$v_1 \in RED_V(\Delta, \tau_1 \to \tau_2) \ \wedge \ v_2 \in RED_V(\Delta, \tau_1) \implies v_1\, v_2 \in RED_C(\Delta, \tau_2)$$

Proof. Follows directly from the definition of $RED_V(\Delta, \tau_1 \to \tau_2)$ □

C.5 Sums

Injection terms are reducible when built from reducible subterms.

Lemma C.10.

$$\begin{aligned} v_i \in RED_V(\Delta, \tau_1) &\implies \mathtt{in}_L\, v_i \in RED_V(\Delta, \tau_1 + \tau_2) \\ v_i \in RED_V(\Delta, \tau_2) &\implies \mathtt{in}_R\, v_i \in RED_V(\Delta, \tau_1 + \tau_2) \end{aligned}$$

Proof. By induction on $\nu(v_i)$. For left injection, we must show two things:

- We must show that $\mathtt{in}_L\, v_i \in SN_V$. The only reduction rule that can apply to $\mathtt{in}_L\, v_i$ is the congruence rule. Thus any reduct would be of the form $\mathtt{in}_L\, v_j$ with $v_i \rightsquigarrow_V v_j$ and so, by the inductive hypothesis via CR2(C.3), would be strongly normalizing. Since every reduct is strongly normalizing then so is $\mathtt{in}_L\, v_i$.
- We must show that $v_j \in RED_V(\Delta, \tau_1)$ for any v_j such that $\mathtt{in}_L\, v_i \rightsquigarrow_V^* \mathtt{in}_L\, v_j$. The reduction is either from reflexivity or by stepping to some intermediate term. If it is by reflexivity then v_j is reducible by hypothesis. Only the congruence reduction rule applies to $\mathtt{in}_L\, v_i$, so the intermediate term must be of the form $\mathtt{in}_L\, v_k$, which means we can use the inductive hypothesis on v_k CR2(C.3) to show that v_j is reducible.

The case for right injection follows by the same reasoning. □

Case terms are reducible when built from reducible subterms.

Lemma C.11.

$$\begin{aligned} & v_1 \in RED_V(\Delta_1, \tau_1 + \tau_2) \\ & \wedge \ (\forall v_4 \in RED_V(\Delta_1, \tau_1).\ v_2[x_1 \backslash v_4] \in RED_V(\Delta_2, \tau_3)) \\ & \wedge \ (\forall v_5 \in RED_V(\Delta_1, \tau_2).\ v_3[x_2 \backslash v_5] \in RED_V(\Delta_2, \tau_3)) \\ & \implies \mathtt{case}^m\, v_1\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_2\, ;\, \mathtt{in}_R\, x_2 \Rightarrow v_3\, \} \in RED_V(\Delta_2, \tau_3) \end{aligned}$$

Proof. By induction on $\nu(v_1) + \nu(v_2) + \nu(v_3).\ \mathtt{case}^m\, v_1\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_2\, ;\, \mathtt{in}_R\, x_2 \Rightarrow v_3\, \}$ is neutral so we can use CR3(C.4). There are five possibilities for a reduct of $\mathtt{case}^m\, v_1\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_2\, ;\, \mathtt{in}_R\, x_2 \Rightarrow v_3\, \}$.

- It can reduce to $v_2[x_1 \backslash v_4]$ if v_1 is of the form $\mathtt{in}_L\, v_4$.
- It can reduce to $v_3[x_2 \backslash v_5]$ if v_1 is of the form $\mathtt{in}_R\, v_5$.
- It can reduce by congruence to $\mathtt{case}^m\, v_6\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_2\, ;\, \mathtt{in}_R\, x_2 \Rightarrow v_3\, \}$ where $v_1 \rightsquigarrow_V v_6$.
- It can reduce by congruence to $\mathtt{case}^m\, v_1\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_7\, ;\, \mathtt{in}_R\, x_2 \Rightarrow v_3\, \}$ where $v_2 \rightsquigarrow_V v_7$.
- It can reduce by congruence to $\mathtt{case}^m\, v_1\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_2\, ;\, \mathtt{in}_R\, x_2 \Rightarrow v_8\, \}$ where $v_3 \rightsquigarrow_V v_8$.

In the first case we have $v_4 \in RED_V(\Delta_1, \tau_1)$ from $v_1 \in RED_V(\Delta_1, \tau_1 + \tau_2)$ and so the reduct is reducible by hypothesis. Similarly, in the second case we have $v_5 \in RED_V(\Delta_1, \tau_2)$ and so the reduct is reducible by hypothesis. The last three cases are reducible by the inductive hypothesis using CR2(C.3). □

C.6 Unit

Unit terms are reducible.

Lemma C.12.

$$() \in RED_V(\Delta, 1)$$

Proof. No reduction rules apply to () so it is strongly normalizable. □

C.7 Global modality

Boxing terms are reducible when built from reducible subterms.

Lemma C.13.

$$v_b \in RED_V(\varepsilon, \tau) \implies \mathsf{box}\, v_b \in RED_V(\Delta, \Box\, \tau)$$

Proof. By induction on $\nu(v_b)$. We must show two things:

- We must show that $\mathsf{box}\, v_b \in SN_V$. The only reduction rule that can apply to $\mathsf{box}\, v_b$ is the congruence rule. Thus any reduct would be of the form $\mathsf{box}\, v_c$ with $v_b \rightsquigarrow_V v_c$ and so, by the inductive hypothesis via CR2(C.3), would be strongly normalizing. Since every reduct is strongly normalizing then so is $\mathsf{box}\, v_b$.
- We must show that $v_c \in RED_V(\varepsilon, \tau)$ for any v_c such that $\mathsf{box}\, v_b \rightsquigarrow_V^* \mathsf{box}\, v_c$. The reduction is either from reflexivity or by stepping to some intermediate term. If it is by reflexivity then v_c is reducible by hypothesis. Only the congruence reduction rule applies to $\mathsf{box}\, v_b$, so the intermediate term must be of the form $\mathsf{box}\, v_d$, which means we can use the inductive hypothesis on v_d via CR2(C.3) to show that v_c is reducible.

□

Unboxing terms are reducible when built from reducible subterms.

Lemma C.14.

$$\begin{aligned} & v_1 \in RED_V(\Delta, \Box\, \tau_1) \\ & \wedge\ (\forall v_3 \in RED_V(\varepsilon, \tau_1).\ v_2[x \backslash v_3] \in RED_V(\Delta, \tau_2)) \\ & \implies \mathtt{let}(\mathsf{box}\, x) = v_1\ \mathtt{in}\ v_2 \in RED_V(\Delta, \tau_2) \end{aligned}$$

Proof. By induction on $\nu(v_1) + \nu(v_2)$. $\mathtt{let}(\mathsf{box}\, x) = v_1\ \mathtt{in}\ v_2$ is neutral so we can use CR3(C.4). There are three possibilities for a reduct of $\mathtt{let}(\mathsf{box}\, x) = v_1\ \mathtt{in}\ v_2$:

- It can reduce to $v_2[x \backslash v_3]$ if v_1 is of the form $\mathsf{box}\, v_3$.
- It can reduce by congruence to $\mathtt{let}(\mathsf{box}\, x) = v_4\ \mathtt{in}\ v_2$ where $v_1 \rightsquigarrow_V v_4$.
- It can reduce by congruence to $\mathtt{let}(\mathsf{box}\, x) = v_1\ \mathtt{in}\ v_5$ where $v_2 \rightsquigarrow_V v_5$.

In the first case we have $v_3 \in RED_V(\varepsilon, \tau_1)$ from $v_1 \in RED_V(\Delta, \Box\, \tau_1)$ and so the reduct is reducible by hypothesis. The second and third cases are reducible by the inductive hypothesis using CR2(C.3). □

C.8 Computations

Return terms are reducible when built from reducible subterms.

Lemma C.15.

$$\begin{aligned} & v_r \in RED_V(\Delta, \tau) \\ & \implies \mathtt{return}\ v_r \in RED_C(\Delta, \tau) \end{aligned}$$

Proof. By induction on $\nu(v_r)$. We must show three things:

- We must show that $\texttt{return}\ v_r \in SN_C$. The only reduction rule that can apply to $\texttt{return}\ v_r$ is the congruence rule. Thus any reduct would be of the form $\texttt{return}\ v_t$ with $v_r \rightsquigarrow_V v_t$ and so, by the inductive hypothesis via CR2(C.3), would be strongly normalizing. Since every reduct is strongly normalizing then so is $\texttt{return}\ v_r$.
- We must show that $v_t \in RED_V(\Delta, \tau)$ for any v_t such that $\texttt{return}\ v_r \rightsquigarrow_V^* \texttt{return}\ v_t$. The reduction is either from reflexivity or by stepping to some intermediate term. If it is by reflexivity then v_t is reducible by hypothesis. Only the congruence reduction rule applies to $\texttt{return}\ v_r$, so the intermediate term must be of the form $\texttt{return}\ v_s$, which means we can use the inductive hypothesis on v_s via CR2(C.3) to show that v_t is reducible.
- We must show that various properties hold for any term of the form $\Uparrow(\ell(\Sigma))(v_a)$ or $\texttt{let}\ x = \Uparrow(\ell(op))(v_a)\ \texttt{in}\ c_k$ that $\texttt{return}\ v_r$ reduces to. The reduction would have to be by stepping to some intermediate term as it can't be via reflexivity. Only the congruence reduction rule applies to $\texttt{return}\ v_r$, so the intermediate term must be of the form $\texttt{return}\ v_t$, which means we can use the inductive hypothesis on v_t via CR2(C.3) to show that the properties would hold.

□

Non-neutral perform terms are reducible when built from reducible subterms.

Lemma C.16.

$$\begin{aligned} &\ell_e : \Sigma \in \Delta \ \wedge\ op : \tau_p \rightarrowtail \tau_a \in \Sigma \ \wedge\ v_p \in RED_V(\varepsilon, \tau_p) \\ &\implies \Uparrow(\ell_e(op))(v_p) \in RED_C(\Delta, \Box\, \tau_a) \end{aligned}$$

Proof. By induction on $\nu(v_p)$. We must show three things:

- We must show that $\Uparrow(\ell_e(op))(v_p) \in SN_C$. The only reduction rule that can apply to $\Uparrow(\ell_e(op))(v_p)$ is the congruence rule. Thus any reduct would be of the form $\Uparrow(\ell_e(op))(v_q)$ with $v_p \rightsquigarrow_V v_q$ and so, by the inductive hypothesis via CR2(C.3), would be strongly normalizing. Since every reduct is strongly normalizing then so is $\Uparrow(\ell_e(op))(v_p)$.
- We must show that $v_q \in RED_V(\varepsilon, \tau_p)$ for any v_q such that $\Uparrow(\ell_e(op))(v_p) \rightsquigarrow_V^* \Uparrow(\ell_e(op))(v_q)$. The reduction is either from reflexivity or by stepping to some intermediate term. If it is by reflexivity then v_q is reducible by hypothesis. Only the congruence reduction rule applies to $\Uparrow(\ell_e(op))(v_p)$, so the intermediate term must be of the form $\Uparrow(\ell_e(op))(v_r)$, which means we can use the inductive hypothesis on v_r via CR2(C.3) to show that v_q is reducible.
- We must show that various properties hold for any term of the form $\texttt{return}\ v_r$, $\Uparrow(\ell_f(\Sigma))(v_q)$ (where $\ell_f \neq \ell_e$) or $\texttt{let}\ x = \Uparrow(\ell(op))(v_p)\ \texttt{in}\ c_k$ that $\Uparrow(\ell_e(op))(v_p)$ reduces to. The reduction would have to be by stepping to some intermediate term as it can't be via reflexivity. Only the congruence reduction rule applies to $\Uparrow(\ell_e(op))(v_p)$, so the intermediate term must be of the form $\Uparrow(\ell_e(op))(v_q)$, which means we can use the inductive hypothesis on v_q via CR2(C.3) to show that the properties would hold.

□

Non-neutral bind terms are reducible when built from reducible subterms.

Lemma C.17.

$$\begin{aligned} &\ell_e : \Sigma \in \Delta \ \wedge\ op : \tau_p \rightarrowtail \tau_a \in \Sigma \ \wedge\ v_p \in RED_V(\varepsilon, \tau_p) \\ &\wedge\ (\forall v_a \in RED_V(\Delta, \Box\, \tau_a).\ c_k[x \backslash v_a] \in RED_C(\Delta, \tau_k)) \\ &\implies \texttt{let}\ x = \Uparrow(\ell_e(op))(v_p)\ \texttt{in}\ c_k \in RED_C(\Delta, \tau_k) \end{aligned}$$

Proof. By induction on $\nu(v_p) + \nu(c_k)$. We must show three things:

- We must show that $\texttt{let}\, x = \Uparrow(\ell_e(op))(v_p)\, \texttt{in}\, c_k \in SN_C$. The only reduction rule that can apply to $\texttt{let}\, x = \Uparrow(\ell_e(op))(v_p)\, \texttt{in}\, c_k$ is the congruence rule. Thus any reducts would be one of two forms:
  – $\texttt{let}\, x = \Uparrow(\ell_e(op))(v_q)\, \texttt{in}\, c_k$ with $v_p \rightsquigarrow_V v_q$
  – $\texttt{let}\, x = \Uparrow(\ell_e(op))(v_p)\, \texttt{in}\, c_j$ with $c_k \rightsquigarrow_C c_j$
  in both cases the reduct is strongly normalizing by the inductive hypothesis via CR2(C.3). Since every reduct is strongly normalizing then so is $\texttt{let}\, x = \Uparrow(\ell_e(op))(v_p)\, \texttt{in}\, c_k$.
- We must show that $v_q \in RED_V(\varepsilon, \tau_p)$ and $\forall v_a \in RED_V(\Delta, \Box\, \tau_a).\ c_j[x\backslash v_a] \in RED_C(\Delta, \tau_k)$ for any v_q and c_j such that $\texttt{let}\, x = \Uparrow(\ell_e(op))(v_p)\, \texttt{in}\, c_k \rightsquigarrow_V^* \texttt{let}\, x = \Uparrow(\ell_e(op))(v_q)\, \texttt{in}\, c_j$. The reduction is either from reflexivity or by stepping to some intermediate term. If it is by reflexivity then v_q and c_j are reducible by hypothesis. Only the congruence reduction rule applies to $\texttt{let}\, x = \Uparrow(\ell_e(op))(v_p)\, \texttt{in}\, c_k$, so the intermediate term must be of the form $\texttt{let}\, x = \Uparrow(\ell_e(op))(v_r)\, \texttt{in}\, c_l$, which means we can use the inductive hypothesis on v_r and c_l via CR2(C.3) to show that v_q and c_j are reducible.
- We must show that various properties hold for any term of the form $\texttt{return}\, v_r$, $\Uparrow(\ell_g(\Sigma))(v_p)$ or $\texttt{let}\, x = \Uparrow(\ell_f(op))(v_q)\, \texttt{in}\, c_j$ (where $\ell_f \neq \ell_e$) that $\texttt{let}\, x = \Uparrow(\ell_f(op))(v_p)\, \texttt{in}\, c_k$ reduces to. The reduction would have to be by stepping to some intermediate term as it can't be via reflexivity. Only the congruence reduction rule applies to $\texttt{let}\, x = \Uparrow(\ell_f(op))(v_p)\, \texttt{in}\, c_k$, so the intermediate term must be of the form $\texttt{let}\, x = \Uparrow(\ell_f(op))(v_r)\, \texttt{in}\, c_l$, which means we can use the inductive hypothesis on v_r and c_l via CR2(C.3) to show that the properties would hold.

□

Bind terms are reducible when built from reducible subterms.

Lemma C.18.

$$\begin{aligned} & c_1 \in RED_C(\Delta, \tau_1, \alpha) \\ \wedge\ & (\forall v \in RED_V(\Delta, \tau_1, \alpha).\ c_2[x\backslash v] \in RED_C(\Delta, \tau_2)) \\ \Longrightarrow\ & \texttt{let}\, x = c_1\, \texttt{in}\, c_2 \in RED_C(\Delta, \tau_2) \end{aligned}$$

Proof. By induction on α and $\nu(c_1) + \nu(c_2)$.

We first consider the case where c_1 is of the form $\Uparrow(\ell_e(op))(v_p)$. Then $c_1 \in RED_C(\Delta, \tau_1, \alpha)$ gives us, via reflexivity, that $\ell_e : \Sigma \in \Delta$, $op : \tau_p \rightarrowtail \tau_a \in \Sigma$ and $v_p \in RED_V(\varepsilon, \tau_p)$, with $\tau_1 = \Box\, \tau_a$. Reducibility at $\Box$ types does not depend on the ordinal, so the hypothesis on c_2 gives us $\forall v_a \in RED_V(\Delta, \Box\, \tau_a).\ c_2[x\backslash v_a] \in RED_C(\Delta, \tau_2)$, and the case follows by Lemma C.17.

Otherwise $\texttt{let}\, x = c_1\, \texttt{in}\, c_2$ is neutral, so we can use CR3(C.4). There are four possibilities for a reduct of $\texttt{let}\, x = c_1\, \texttt{in}\, c_2$:

- It can reduce by congruence to $\texttt{let}\, x = c_3\, \texttt{in}\, c_2$ where $c_1 \rightsquigarrow_C c_3$. By CR2(C.3) we have $c_3 \in RED_C(\Delta, \tau_1, \alpha)$, so the reduct is reducible by the inductive hypothesis.
- It can reduce by congruence to $\texttt{let}\, x = c_1\, \texttt{in}\, c_4$ where $c_2 \rightsquigarrow_C c_4$. For any $v \in RED_V(\Delta, \tau_1, \alpha)$ we have that $c_4[x\backslash v]$ is a reduct of $c_2[x\backslash v]$ and so is reducible by CR2(C.3). Thus the reduct is reducible by the inductive hypothesis.
- If c_1 is of the form $\texttt{return}\, v_1$ then it can reduce to $c_2[x\backslash v_1]$. From $c_1 \in RED_C(\Delta, \tau_1, \alpha)$ we have, via reflexivity, that $v_1 \in RED_V(\Delta, \tau_1, \alpha)$, so the reduct is reducible by hypothesis.
- If c_1 is of the form $\texttt{let}\, y = \Uparrow(\ell_e(op))(v_p)\, \texttt{in}\, c_3$ then it can reduce by the commuting rule to $\texttt{let}\, y = \Uparrow(\ell_e(op))(v_p)\, \texttt{in}(\texttt{let}\, x = c_3\, \texttt{in}\, c_2)$. From $c_1 \in RED_C(\Delta, \tau_1, \alpha)$ we have, via reflexivity, that $\ell_e : \Sigma \in \Delta$, $op : \tau_p \rightarrowtail \tau_a \in \Sigma$ and $v_p \in RED_V(\varepsilon, \tau_p)$, and that for any $v_a \in RED_V(\Delta, \Box\, \tau_a)$ there is some $\beta < \alpha$ with $c_3[y\backslash v_a] \in RED_C(\Delta, \tau_1, \beta)$. For any such v_a we have $(\texttt{let}\, x = c_3\, \texttt{in}\, c_2)[y\backslash v_a] = \texttt{let}\, x = c_3[y\backslash v_a]\, \texttt{in}\, c_2$, which is reducible by the inductive hypothesis on

β, using the monotonicity of the predicates in α for the hypothesis on c_2. The reduct is then reducible by Lemma C.17.

□

Run terms are reducible when built from reducible subterms.

Lemma C.19.

$$c_r \in RED_C(\varepsilon, \tau) \implies \mathsf{run}(c_r) \in RED_V(\Delta, \tau)$$

Proof. By induction on $\nu(c_r)$. $\mathsf{run}(c_r)$ is neutral so we can use CR3(C.4). There are two possibilities for a reduct of $\mathsf{run}(c_r)$:

- It can reduce by congruence to $\mathsf{run}(c_s)$ where $c_r \rightsquigarrow_C c_s$. By CR2(C.3) we have $c_s \in RED_C(\varepsilon, \tau)$, so the reduct is reducible by the inductive hypothesis.
- If c_r is of the form $\mathtt{return}\ v$ then it can reduce to v. From $c_r \in RED_C(\varepsilon, \tau)$ we have, via reflexivity, that $v \in RED_V(\varepsilon, \tau)$, and thus that $v \in RED_V(\Delta, \tau)$ by effect weakening (C.5).

□

C.9 Effect capability types

Location terms are reducible when they are bound in the effect context.

Lemma C.20.

$$\ell : \Sigma \in \Delta \implies \ell \in RED_V(\Delta, \lessdot(\Sigma))$$

Proof. No reduction rules apply to ℓ, so it is strongly normalizing and its only reduct, via reflexivity, is ℓ itself, for which we have $\ell : \Sigma \in \Delta$ by hypothesis. The conditions for reducts of the forms $\mathsf{out}_L\, v$ and $\mathsf{out}_R\, v$ hold vacuously. □

Selection terms are reducible when built from reducible subterms.

Lemma C.21.

$$v_s \in RED_V(\Delta, \lessdot(\Sigma_1 + \Sigma_2)) \implies \mathsf{out}_L\, v_s \in RED_V(\Delta, \lessdot(\Sigma_1))$$
$$v_s \in RED_V(\Delta, \lessdot(\Sigma_1 + \Sigma_2)) \implies \mathsf{out}_R\, v_s \in RED_V(\Delta, \lessdot(\Sigma_2))$$

Proof. By induction on $\nu(v_s)$. For left selection, we must show three things:

- We must show that $\mathsf{out}_L\, v_s \in SN_V$. The only reduction rule that can apply to $\mathsf{out}_L\, v_s$ is the congruence rule. Thus any reduct would be of the form $\mathsf{out}_L\, v_r$ with $v_s \rightsquigarrow_V v_r$ and so, by the inductive hypothesis via CR2(C.3), would be strongly normalizing. Since every reduct is strongly normalizing then so is $\mathsf{out}_L\, v_s$.
- We must show, for any v_r such that $\mathsf{out}_L\, v_s \rightsquigarrow_V^* \mathsf{out}_L\, v_r$, that there are some Σ and β with $v_r \in RED_V(\Delta, \lessdot(\Sigma_1 + \Sigma), \beta)$. Only the congruence reduction rule applies to $\mathsf{out}_L\, v_s$, so $v_s \rightsquigarrow_V^* v_r$ and thus $v_r \in RED_V(\Delta, \lessdot(\Sigma_1 + \Sigma_2))$ by CR2(C.3), giving $v_r \in RED_V(\Delta, \lessdot(\Sigma_1 + \Sigma_2), \beta)$ for some β.
- We must show that various properties hold for any term of the form ℓ or $\mathsf{out}_R\, v_r$ that $\mathsf{out}_L\, v_s$ reduces to. Only the congruence reduction rule applies to $\mathsf{out}_L\, v_s$, so every reduct is of the form $\mathsf{out}_L\, v_r$ and the conditions hold vacuously.

The case for right selection follows by the same reasoning. □

Perform terms are reducible when built from reducible subterms.

Lemma C.22.

$$op : \tau_p \rightarrowtail \tau_a \in \Sigma \ \wedge \ v_l \in RED_V(\Delta, \lambda(\Sigma), \alpha) \ \wedge \ v_a \in RED_V(\varepsilon, \tau_p)$$
$$\implies \Uparrow(v_l(op))(v_a) \in RED_C(\Delta, \Box\, \tau_a)$$

Proof. By induction on α and $\nu(v_l) + \nu(v_a)$.

We first consider the case where v_l is a location ℓ. Then $v_l \in RED_V(\Delta, \lambda(\Sigma), \alpha)$ gives us, via reflexivity, that $\ell : \Sigma \in \Delta$, and the case follows by Lemma C.16.

Otherwise $\Uparrow(v_l(op))(v_a)$ is neutral, so we can use CR3(C.4). There are four possibilities for a reduct of $\Uparrow(v_l(op))(v_a)$:

- It can reduce by congruence to $\Uparrow(v_m(op))(v_a)$ where $v_l \rightsquigarrow_V v_m$. By CR2(C.3) we have $v_m \in RED_V(\Delta, \lambda(\Sigma), \alpha)$, so the reduct is reducible by the inductive hypothesis.
- It can reduce by congruence to $\Uparrow(v_l(op))(v_b)$ where $v_a \rightsquigarrow_V v_b$. By CR2(C.3) we have $v_b \in RED_V(\varepsilon, \tau_p)$, so the reduct is reducible by the inductive hypothesis.
- If v_l is of the form $\mathtt{out}_L\, v_m$ then it can reduce to $\Uparrow(v_m(\mathrm{left}(op)))(v_a)$. From $v_l \in RED_V(\Delta, \lambda(\Sigma), \alpha)$ we have, via reflexivity, some Σ_2 and $\beta < \alpha$ with $v_m \in RED_V(\Delta, \lambda(\Sigma + \Sigma_2), \beta)$. Since $\mathrm{left}(op) : \tau_p \rightarrowtail \tau_a \in \Sigma + \Sigma_2$, the reduct is reducible by the inductive hypothesis on β.
- If v_l is of the form $\mathtt{out}_R\, v_m$ then it can reduce to $\Uparrow(v_m(\mathrm{right}(op)))(v_a)$, which is reducible by the same reasoning as the previous case.

□

C.10 Inductive types

Ret terms are reducible when built from reducible subterms.

Lemma C.23.

$$v_r \in RED_V(\varepsilon, \tau_1)$$
$$\implies \mathtt{ret}(v_r) \in RED_V(\Delta, W_\Sigma(\tau_1))$$

Proof. By induction on $\nu(v_r)$. We must show three things:

- We must show that $\mathtt{ret}(v_r) \in SN_V$. The only reduction rule that can apply to $\mathtt{ret}(v_r)$ is the congruence rule. Thus any reduct would be of the form $\mathtt{ret}(v_t)$ with $v_r \rightsquigarrow_V v_t$ and so, by the inductive hypothesis via CR2(C.3), would be strongly normalizing. Since every reduct is strongly normalizing then so is $\mathtt{ret}(v_r)$.
- We must show that $v_t \in RED_V(\varepsilon, \tau_1)$ for any v_t such that $\mathtt{ret}(v_r) \rightsquigarrow_V^* \mathtt{ret}(v_t)$. The reduction is either from reflexivity or by stepping to some intermediate term. If it is by reflexivity then v_t is reducible by hypothesis. Only the congruence reduction rule applies to $\mathtt{ret}(v_r)$, so the intermediate term must be of the form $\mathtt{ret}(v_s)$, which means we can use the inductive hypothesis on v_s via CR2(C.3) to show that v_t is reducible.
- We must show that various properties hold for any term of the form $op(v_3, v_4)$ that $\mathtt{ret}(v_r)$ reduces to. Only the congruence reduction rule applies to $\mathtt{ret}(v_r)$, so every reduct is of the form $\mathtt{ret}(v_t)$ and the condition holds vacuously.

□

Operation terms are reducible when built from reducible subterms.

Lemma C.24.

$$op : \tau_p \rightarrowtail \tau_a \in \Sigma \ \wedge \ v_p \in RED_V(\varepsilon, \tau_p)$$
$$\wedge \ v_k \in RED_V(\Delta, \Box\, \tau_a \rightarrow W_\Sigma(\tau_1))$$
$$\implies op(v_p, v_k) \in RED_V(\Delta, W_\Sigma(\tau_1))$$

Proof. By induction on $\nu(v_p) + \nu(v_k)$. We must show three things:

- We must show that $op(v_p, v_k) \in SN_V$. The only reduction rule that can apply to $op(v_p, v_k)$ is the congruence rule. Thus any reduct would be of the form $op(v_q, v_k)$ with $v_p \rightsquigarrow_V v_q$ or $op(v_p, v_l)$ with $v_k \rightsquigarrow_V v_l$ and so, by the inductive hypothesis via CR2(C.3), would be strongly normalizing. Since every reduct is strongly normalizing then so is $op(v_p, v_k)$.
- We must show that various properties hold for any term of the form $\mathtt{ret}(v_2)$ that $op(v_p, v_k)$ reduces to. Only the congruence reduction rule applies to $op(v_p, v_k)$, so every reduct is of the form $op(v_q, v_l)$ and the condition holds vacuously.
- We must show, for any v_q and v_l such that $op(v_p, v_k) \rightsquigarrow_V^* op(v_q, v_l)$, that $v_q \in RED_V(\varepsilon, \tau_p)$ and that for any $v_b \in SN_V$ whose reducts of the form $\mathtt{box}\, v_c$ all have $v_c \in RED_V(\varepsilon, \tau_a)$ there is some β with $v_l\, v_b \in RED_C(\Delta, W_\Sigma(\tau_1), \beta)$. Only the congruence reduction rule applies to $op(v_p, v_k)$, so $v_p \rightsquigarrow_V^* v_q$ and $v_k \rightsquigarrow_V^* v_l$, and thus v_q and v_l are reducible by hypothesis and CR2(C.3). The condition on v_b states precisely that $v_b \in RED_V(\Delta, \Box\, \tau_a)$, so from $v_l \in RED_V(\Delta, \Box\, \tau_a \to W_\Sigma(\tau_1))$ we have $v_l\, v_b \in RED_C(\Delta, W_\Sigma(\tau_1))$, and hence $v_l\, v_b \in RED_C(\Delta, W_\Sigma(\tau_1), \beta)$ for some β. Taking α to be a strict upper bound of these β gives $op(v_p, v_k) \in RED_V(\Delta, W_\Sigma(\tau_1), \alpha)$.

□

Reification terms are reducible when built from reducible subterms.

Lemma C.25.

$$c \in RED_C(\Delta;\, \ell : \Sigma, \Box\, \tau, \alpha) \implies \Downarrow(\ell.\, c) \in RED_C(\Delta, W_\Sigma(\tau), \alpha)$$

Proof. By induction on α and $\nu(c)$, assuming without loss of generality that ℓ is not bound in Δ. $\Downarrow(\ell.\, c)$ is neutral so we can use CR3(C.4). There are six possibilities for a reduct of $\Downarrow(\ell.\, c)$:

- It can reduce by congruence to $\Downarrow(\ell.\, c')$ where $c \rightsquigarrow_C c'$. By CR2(C.3) we have $c' \in RED_C(\Delta;\, \ell : \Sigma, \Box\, \tau, \alpha)$, so the reduct is reducible by the inductive hypothesis.
- If c is of the form $\mathtt{return}\ (\mathtt{box}\, v)$ then it can reduce to $\mathtt{return}\ \mathtt{ret}(v)$. From the reducibility of c we have, via reflexivity, that $\mathtt{box}\, v \in RED_V(\Delta;\, \ell : \Sigma, \Box\, \tau)$ and thus that $v \in RED_V(\varepsilon, \tau)$. The reduct is then reducible by Lemmas C.23 and C.15.
- If c is of the form $\Uparrow(\ell(op))(v)$ then it can reduce to $\mathtt{return}\ op(v, \lambda(\mathtt{box}\, x).\ \mathtt{return}\ \mathtt{ret}(x))$. From the reducibility of c we have, via reflexivity, that $op : \tau_p \rightarrowtail \tau_a \in \Sigma$ and $v \in RED_V(\varepsilon, \tau_p)$. For any $v_b \in RED_V(\Delta, \Box\, \tau_a)$, the body of the continuation with v_b substituted for x is reducible by Lemmas C.14, C.23 and C.15, so the continuation is reducible by Lemma C.8. The reduct is then reducible by Lemmas C.24 and C.15.
- If c is of the form $\mathtt{let}\, x = \Uparrow(\ell(op))(v)\ \mathtt{in}\, c_2$ then it can reduce to $\mathtt{return}\ op(v, \lambda x.\ \Downarrow(\ell.\, c_2))$. From the reducibility of c we have, via reflexivity, that $op : \tau_p \rightarrowtail \tau_a \in \Sigma$ and $v \in RED_V(\varepsilon, \tau_p)$, and that for any $v_b \in RED_V(\Delta, \Box\, \tau_a)$ there is some $\beta < \alpha$ with $c_2[x \backslash v_b] \in RED_C(\Delta;\, \ell : \Sigma, \Box\, \tau, \beta)$. By the inductive hypothesis on β we then have $\Downarrow(\ell.\, c_2[x \backslash v_b]) = (\Downarrow(\ell.\, c_2))[x \backslash v_b] \in RED_C(\Delta, W_\Sigma(\tau), \beta)$, so the continuation is reducible by Lemma C.8. The reduct is then reducible by Lemmas C.24 and C.15.
- If c is of the form $\Uparrow(\ell_2(op))(v)$ with $\ell_2 \neq \ell$ then it can reduce to

  $$\mathtt{let}(\mathtt{box}\, x) = \Uparrow(\ell_2(op))(v)\ \mathtt{in}\ \mathtt{return}\ \mathtt{ret}(x)$$

  From the reducibility of c we have, via reflexivity, that $\ell_2 : \Sigma_2 \in \Delta;\, \ell : \Sigma$, and since $\ell_2 \neq \ell$ that $\ell_2 : \Sigma_2 \in \Delta$, along with $op : \tau_p \rightarrowtail \tau_a \in \Sigma_2$ and $v \in RED_V(\varepsilon, \tau_p)$. For any $v_b \in RED_V(\Delta, \Box\, \tau_a)$, the body of the binding with v_b substituted for x is reducible by Lemmas C.14, C.23 and C.15, so the reduct is reducible by Lemma C.17.

- If c is of the form $\mathtt{let}\, x = \Uparrow(\ell_2(op))(v)\, \mathtt{in}\, c_2$ with $\ell_2 \neq \ell$ then it can reduce to $\mathtt{let}\, x = \Uparrow(\ell_2(op))(v)\, \mathtt{in} \Downarrow(\ell.\, c_2)$. From the reducibility of c we have, via reflexivity, that $\ell_2 : \Sigma_2 \in \Delta$, $op : \tau_p \rightarrowtail \tau_a \in \Sigma_2$ and $v \in RED_V(\varepsilon, \tau_p)$, and that for any $v_b \in RED_V(\Delta, \square\, \tau_a)$ there is some $\beta < \alpha$ with $c_2[x \backslash v_b] \in RED_C(\Delta;\, \ell : \Sigma, \square\, \tau, \beta)$. By the inductive hypothesis on β we then have $(\Downarrow(\ell.\, c_2))[x \backslash v_b] \in RED_C(\Delta, W_\Sigma(\tau), \beta)$, so the reduct is reducible by Lemma C.17.

□

Recursion terms are reducible when built from reducible subterms.

Lemma C.26.

$$
\begin{aligned}
& v_s \in RED_V(\Delta, W_{\{op_1 : \tau_{p_1} \rightarrowtail \tau_{a_1}; \ldots; op_n : \tau_{p_n} \rightarrowtail \tau_{a_n}\}}(\tau_1), \alpha) \\
& \wedge\ (\forall v_r \in RED_V(\varepsilon, \tau_1, \alpha). \\
& \qquad v_e[x_r \backslash v_r] \in RED_V(\Delta, \tau_2, \alpha)) \\
& \wedge\ (\forall v_{p_1} \in RED_V(\varepsilon, \tau_{p_1}, \alpha),\ v_{k_1} \in RED_V(\Delta, \square\, \tau_{a_1} \longrightarrow \tau_2, \alpha). \\
& \qquad v_1[x_{p_1} \backslash v_{p_1}][x_{k_1} \backslash v_{k_1}] \in RED_V(\Delta, \tau_2, \alpha)) \\
& \ldots \\
& \wedge\ (\forall v_{p_n} \in RED_V(\varepsilon, \tau_{p_n}, \alpha),\ v_{k_n} \in RED_V(\Delta, \square\, \tau_{a_n} \longrightarrow \tau_2, \alpha). \\
& \qquad v_n[x_{p_n} \backslash v_{p_n}][x_{k_n} \backslash v_{k_n}] \in RED_V(\Delta, \tau_2, \alpha)) \\
& \Longrightarrow \\
& \mathtt{rec}\, v_s\, \{\mathtt{ret}(x_r) \Rightarrow v_e\, ;\, op_1(x_{p_1}, x_{k_1}) \Rightarrow v_1\, ;\, \ldots\, ;\, op_n(x_{p_n}, x_{k_n}) \Rightarrow v_n\} \in RED_V(\Delta, \tau_2, \alpha)
\end{aligned}
$$

Proof. Write H for the handler $\mathtt{ret}(x_r) \Rightarrow v_e\, ;\, op_1(x_{p_1}, x_{k_1}) \Rightarrow v_1\, ;\, \ldots\, ;\, op_n(x_{p_n}, x_{k_n}) \Rightarrow v_n$. By induction on α and $\nu(v_s) + \nu(v_e) + \nu(v_1) + \ldots + \nu(v_n)$. $\mathtt{rec}\, v_s\, \{H\}$ is neutral so we can use CR3(C.4). There are three kinds of reduct of $\mathtt{rec}\, v_s\, \{H\}$:

- It can reduce by congruence in v_s, v_e or one of the v_i. The subterms of the reduct remain reducible by CR2(C.3) – for the branches this is because each substitution instance of the reduced branch is a reduct of the corresponding substitution instance of the original branch – so the reduct is reducible by the inductive hypothesis.
- If v_s is of the form $\mathtt{ret}(v_r)$ then it can reduce to $v_e[x_r \backslash v_r]$. From $v_s \in RED_V(\Delta, W_\Sigma(\tau_1), \alpha)$ we have, via reflexivity, that $v_r \in RED_V(\varepsilon, \tau_1, \alpha)$, so the reduct is reducible by hypothesis.
- If v_s is of the form $op_i(v_p,\, v_k)$ then it can reduce to $v_i[x_{p_i} \backslash v_p][x_{k_i} \backslash v_c]$ where

$$v_c = \lambda x_a.\ \mathtt{let}\, x_w = v_k\, x_a\, \mathtt{in}\, \mathtt{return}\, (\mathtt{rec}\, x_w\, \{H\})$$

  From $v_s \in RED_V(\Delta, W_\Sigma(\tau_1), \alpha)$ we have, via reflexivity, that $v_p \in RED_V(\varepsilon, \tau_{p_i}, \alpha)$ and that for any $v_a \in RED_V(\Delta, \square\, \tau_{a_i})$ there is some $\beta < \alpha$ with $v_k\, v_a \in RED_C(\Delta, W_\Sigma(\tau_1), \beta)$. We show that $v_c \in RED_V(\Delta, \square\, \tau_{a_i} \longrightarrow \tau_2, \alpha)$, for which, by Lemma C.8, it is sufficient to show that $\mathtt{let}\, x_w = v_k\, v_a\, \mathtt{in}\, \mathtt{return}\, (\mathtt{rec}\, x_w\, \{H\})$ is reducible for any $v_a \in RED_V(\Delta, \square\, \tau_{a_i})$. For any $v_w \in RED_V(\Delta, W_\Sigma(\tau_1), \beta)$ we have that $\mathtt{rec}\, v_w\, \{H\}$ is reducible by the inductive hypothesis on β, using the monotonicity of the predicates in α for the hypotheses on the branches, and thus $\mathtt{return}\, (\mathtt{rec}\, v_w\, \{H\})$ is reducible by Lemma C.15. Thus $\mathtt{let}\, x_w = v_k\, v_a\, \mathtt{in}\, \mathtt{return}\, (\mathtt{rec}\, x_w\, \{H\})$ is reducible by Lemma C.18, and so v_c is reducible. The reduct is then reducible by the hypothesis for the op_i branch.

□

C.11 Reducible substitutions

A substitution σ is a sequence of single variable substitutions $[x_1 \backslash v_1] \cdots [x_n \backslash v_n]$, applied to a term one at a time from left to right. We write $\mathrm{FV}(\sigma)$ for the free variables of the values of σ.

We define a notion of reducibility for substitutions as a reducibility predicate $RED_S(\Gamma)$ indexed by a typing context Γ. A reducible substitution has one entry for each variable binding of the context, with the values for local bindings reducible in the effect context containing the locations bound earlier in the context, and the values for global bindings reducible in the empty effect context. The freshness conditions ensure that the entries of a reducible substitution do not interact with one another, so that it behaves as a simultaneous substitution.

$$
\begin{aligned}
&RED_S(\varepsilon) = \{\, \varepsilon \,\} \\
&RED_S(\Gamma;\, x : \tau) = \{\, \sigma \cdot [x \backslash v] \mid x \notin \mathrm{FV}(\sigma) \ \wedge\ \sigma \in RED_S(\Gamma) \ \wedge\ v \in RED_V(\mathrm{locs}(\Gamma), \tau) \,\} \\
&RED_S(\Gamma;\, x :_{\square} \tau) = \{\, \sigma \cdot [x \backslash v] \mid x \notin \mathrm{FV}(\sigma) \ \wedge\ \sigma \in RED_S(\Gamma) \ \wedge\ v \in RED_V(\varepsilon, \tau) \,\} \\
&RED_S(\Gamma;\, \ell : \Sigma) = RED_S(\Gamma)
\end{aligned}
$$

where ε is the empty sequence.

Reducible substitutions can be extended with reducible values and with location bindings:

Lemma C.27. *Providing that x does not occur free in the values of σ:*

$$
\begin{aligned}
&\sigma \in RED_S(\Gamma) \ \wedge\ v \in RED_V(\mathrm{locs}(\Gamma), \tau) \\
&\quad \Longrightarrow\ \sigma \cdot [x \backslash v] \in RED_S(\Gamma;\, x : \tau) \\
&\sigma \in RED_S(\Gamma) \ \wedge\ v \in RED_V(\varepsilon, \tau) \\
&\quad \Longrightarrow\ \sigma \cdot [x \backslash v] \in RED_S(\Gamma;\, x :_{\square} \tau) \\
&\sigma \in RED_S(\Gamma) \ \Longrightarrow\ \sigma \in RED_S(\Gamma;\, \ell : \Sigma)
\end{aligned}
$$

Proof. Immediate from the definition of RED_S. □

Reducible substitutions substitute reducible values for variables:

Lemma C.28.

$$
\begin{aligned}
&\sigma \in RED_S(\Gamma_1;\, x : \tau;\, \Gamma_2) \ \Longrightarrow\ x[\sigma] \in RED_V(\mathrm{locs}(\Gamma_1;\, \Gamma_2), \tau) \\
&\sigma \in RED_S(\Gamma_1;\, x :_{\square} \tau;\, \Gamma_2) \ \Longrightarrow\ x[\sigma] \in RED_V(\varepsilon, \tau)
\end{aligned}
$$

Proof. By induction on Γ_2. The entries for the bindings before x bind other variables, so they leave the term x unchanged, and the freshness conditions for the entries for the bindings after x ensure that they do not affect the value v substituted for x, so $x[\sigma] = v$. For a local binding the definition gives $v \in RED_V(\mathrm{locs}(\Gamma_1), \tau)$ and the result follows by effect weakening (C.5). For a global binding the definition gives $v \in RED_V(\varepsilon, \tau)$ directly. □

Finally, reducible substitutions can be divided. We write $\sigma/\square$ for the subsequence of σ consisting of the entries for the global bindings of the context.

Lemma C.29.

$$
\sigma \in RED_S(\Gamma) \ \Longrightarrow\ \sigma/\square \in RED_S(\Gamma/\square)
$$

Proof. By induction on Γ. The entries for local bindings are dropped along with their bindings, the freshness conditions for the remaining entries are implied by those for σ, and the conditions on the entries for global bindings are unchanged. □

Note that a term typed in $\Gamma/\square$ contains no free occurrences of the local variables or locations of Γ, so, using the freshness conditions, for such a term v we have $v[\sigma] = v[\sigma/\square]$.

$$
\begin{array}{ll}
\wr 1 \int = 1 & \wr x \int = x \\
\wr A \times B \int = \wr A \int \times \wr B \int & \wr (e, f) \int = (\wr e \int, \wr f \int) \\
\wr A \to B \int = \Box \wr A \int \to \wr B \int & \wr \pi_1\, e \int = \pi_1 \wr e \int \\
 & \wr \pi_2\, e \int = \pi_2 \wr e \int \\
\wr \varepsilon \int = \varepsilon & \wr \lambda x.\, e \int = \lambda(\mathsf{box}\, x).\ \mathtt{return}\, \wr e \int \\
\wr \Gamma;\, x : A \int = \wr \Gamma \int;\, x :_{\Box} \wr A \int & \wr f\, e \int = \mathsf{run}(\wr f \int\ \mathsf{box} \wr e \int)
\end{array}
$$

Fig. 16. Embedding of Simply-Typed Lambda Calculus into values

C.12 Reducibility theorem

Well-typed terms are reducible under any reducible substitution:

$$
\begin{array}{lllll}
\Gamma \vdash_V v : \tau & \wedge & \sigma \in \mathit{RED}_S(\Gamma) & \Longrightarrow & v[\sigma] \in \mathit{RED}_V(\mathrm{locs}(\Gamma), \tau) \\
\Gamma \vdash_C c : \tau & \wedge & \sigma \in \mathit{RED}_S(\Gamma) & \Longrightarrow & c[\sigma] \in \mathit{RED}_C(\mathrm{locs}(\Gamma), \tau)
\end{array}
$$

Proof. By mutual induction on the typing derivations for v and c. The variable cases follow from Lemma C.28. Every other case consists of applying the corresponding lemma from the preceding subsections (C.6–C.26). The premises of those lemmas are met as follows.

- Premises typed in Γ itself use the inductive hypothesis with σ directly.
- Premises typed under additional bindings use the inductive hypothesis with σ extended by the extension lemma (C.27): the reducible values quantified over by the lemmas for the binding forms are exactly those required by the extension lemma.
- Premises typed in the divided context $\Gamma/\Box$ use the inductive hypothesis with $\sigma/\Box$ via the division lemma (C.29).

□

As a corollary, since the substitution that replaces every variable with itself is reducible, we get:

$$
\begin{array}{lll}
\Gamma \vdash_V v : \tau & \Longrightarrow & v \in \mathit{RED}_V(\mathrm{locs}(\Gamma), \tau) \\
\Gamma \vdash_C c : \tau & \Longrightarrow & c \in \mathit{RED}_C(\mathrm{locs}(\Gamma), \tau)
\end{array}
$$

from which it immediately follows that well-typed terms are strongly normalizing.

D EMBEDDING THE SIMPLY-TYPED LAMBDA CALCULUS

We define an embedding $\wr \_ \int$ on types, contexts and terms of the Simply-Typed Lambda Calculus (STLC) in Fig.16.

Note that the embedding produces contexts with only global bindings so that:

Lemma D.1.

$$\wr \Gamma \int / \Box = \wr \Gamma \int$$

This embedding maps well-typed STLC terms to well-typed values:

Lemma D.2.

$$\Gamma \vdash_{STLC} e : t \quad \Longrightarrow \quad \wr \Gamma \int \vdash_V \wr e \int : \wr t \int$$

which is proved by induction on the STLC typing derivation. The case for the app rule relies on D.1 in order to apply `run` to the embedded expression.

Reduction of STLC terms is modelled by the reduction of the embedded terms:

Lemma D.3.

$$e_1 \rightsquigarrow_{STLC} e_2 \quad \Longrightarrow \quad \wr e_1 \int \rightsquigarrow_V^* \wr e_2 \int$$

We prove this via induction on the STLC reduction relation. All cases are trivial except for the function beta-reduction case:

$$\wr (\lambda x.\, e_1)\, e_2 \int \rightsquigarrow_V^* \wr e_1[x \backslash e_2] \int$$

which expands to:

$$\mathtt{run}((\lambda(\mathtt{box}\, x).\ \mathtt{return} \wr e_1 \int)\ \mathtt{box} \wr e_2 \int) \rightsquigarrow_V^* \wr e_1[x \backslash e_2] \int$$

applying the function beta rule, the unboxing rule for the global modality, the `run` reduction rule and reflexivity, this becomes:

$$\wr e_1[x \backslash e_2] \int = \wr e_1 \int [x \backslash \wr e_2 \int]$$

which follows by induction on e_1.

Normal forms of STLC are embedded to normal forms:

Lemma D.4.

$$e \not\rightsquigarrow_{STLC} \quad \Longrightarrow \quad \wr e \int \not\rightsquigarrow_V$$

This follows from observing that the neutral forms and normal forms of STLC embed to neutral and normal forms respectively. Since our language is strongly normalizing (2.6) this is sufficient to show that the embedding correctly models STLC.

E DENOTATIONAL SEMANTICS

In this section we prove that the denotational semantics is sound and adequate with respect to the operational semantics. The full denotational semantics are in Fig. 17, Fig. 18, Fig. 19 and Fig. 20.

E.1 Weakening

We will need lemmas showing that weakening is equivalent to projection for local bindings:

Lemma E.1.

$$\llbracket \Gamma \vdash_V v : \tau_1 \rrbracket \circ \pi_{1(\llbracket \Gamma \rrbracket, \llbracket \tau_2 \rrbracket)} = \llbracket \Gamma;\, x : \tau_2 \vdash_V v : \tau_1 \rrbracket$$
$$\llbracket \Gamma \vdash_C c : \tau_1 \rrbracket \circ \pi_{1(\llbracket \Gamma \rrbracket, \llbracket \tau_2 \rrbracket)} = \llbracket \Gamma;\, x : \tau_2 \vdash_C c : \tau_1 \rrbracket$$

and global bindings:

Lemma E.2.

$$\llbracket \Gamma \vdash_V v : \tau_1 \rrbracket \circ \pi_{1(\llbracket \Gamma \rrbracket, \Box(\llbracket \tau_2 \rrbracket))} = \llbracket \Gamma;\, x :_\Box \tau_2 \vdash_V v : \tau_1 \rrbracket$$
$$\llbracket \Gamma \vdash_C c : \tau_1 \rrbracket \circ \pi_{1(\llbracket \Gamma \rrbracket, \Box(\llbracket \tau_2 \rrbracket))} = \llbracket \Gamma;\, x :_\Box \tau_2 \vdash_C c : \tau_1 \rrbracket$$

and effect capability bindings:

Lemma E.3.

$$\llbracket \Gamma \vdash_V v : \tau_1 \rrbracket \circ \pi_{1(\llbracket \Gamma \rrbracket, \lightning(\llbracket \Sigma \rrbracket))} = \llbracket \Gamma;\, \ell : \Sigma \vdash_V v : \tau_1 \rrbracket$$
$$\llbracket \Gamma \vdash_C c : \tau_1 \rrbracket \circ \pi_{1(\llbracket \Gamma \rrbracket, \lightning(\llbracket \Sigma \rrbracket))} = \llbracket \Gamma;\, \ell : \Sigma \vdash_C c : \tau_1 \rrbracket$$

Proof. By mutual induction on the typing derivations for v and c. We prove slightly stronger forms that allow additional bindings after x, so that we can use inductive hypotheses under additional bindings. The cases all follow from standard equalities about products and projections. □

We also need a lemma showing that weakening from $\Gamma/\Box$ to Γ is equivalent to $!_\Gamma/\Box$:

$$[\![\tau]\!] : \widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$$
$$[\![\tau_1 \times \tau_2]\!] = [\![\tau_1]\!] \times [\![\tau_2]\!]$$
$$[\![1]\!] = 1$$
$$[\![\tau_1 + \tau_2]\!] = [\![\tau_1]\!] + [\![\tau_2]\!]$$
$$[\![\tau_1 \to \tau_2]\!] = \mathcal{T}([\![\tau_2]\!])^{[\![\tau_1]\!]}$$

(a) Types

$$[\![\Gamma]\!] : \widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$$
$$[\![\varepsilon]\!] = 1$$
$$[\![\Gamma;\, x : \tau]\!] = [\![\Gamma]\!] \times [\![\tau]\!]$$

(b) Contexts

$$[\![\Gamma \vdash_V v : \tau]\!] : \mathrm{Hom}_{\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}}([\![\Gamma]\!], [\![\tau]\!])$$
$$[\![\Gamma_1;\, x : \tau;\, \Gamma_2 \vdash_V x : \tau]\!] = \pi_{|\Gamma_1|}$$
$$[\![\Gamma \vdash_V \lambda x.\, c : \tau_1 \to \tau_2]\!] = \lambda([\![\Gamma;\, x : \tau_1 \vdash_C c : \tau_2]\!])$$
$$[\![\Gamma \vdash_V (v_1, v_2) : \tau_1 \times \tau_2]\!] = \langle [\![\Gamma \vdash_V v_1 : \tau_1]\!], [\![\Gamma \vdash_V v_2 : \tau_2]\!] \rangle$$
$$[\![\Gamma \vdash_V \pi_1\, v : \tau_1]\!] = \pi_{1([\![\tau_1]\!],[\![\tau_2]\!])} \circ [\![\Gamma \vdash_V v : \tau_1 \times \tau_2]\!]$$
$$[\![\Gamma \vdash_V \pi_2\, v : \tau_2]\!] = \pi_{2([\![\tau_1]\!],[\![\tau_2]\!])} \circ [\![\Gamma \vdash_V v : \tau_1 \times \tau_2]\!]$$
$$[\![\Gamma \vdash_V () : 1]\!] = !_{[\![\Gamma]\!]}$$
$$[\![\Gamma \vdash_V \mathtt{in}_L\, v : \tau_1 + \tau_2]\!] = \iota_{1([\![\tau_1]\!],[\![\tau_2]\!])} \circ [\![\Gamma \vdash_V v : \tau_1]\!]$$
$$[\![\Gamma \vdash_V \mathtt{in}_R\, v : \tau_1 + \tau_2]\!] = \iota_{2([\![\tau_1]\!],[\![\tau_2]\!])} \circ [\![\Gamma \vdash_V v : \tau_2]\!]$$
$$[\![\Gamma \vdash_V \mathtt{case}\, v_1\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_2\, ;\, \mathtt{in}_R\, x_2 \Rightarrow v_3\, \} : \tau_3]\!] =$$
$$[[\![\Gamma;\, x_1 : \tau_1 \vdash_V v_2 : \tau_3]\!], [\![\Gamma;\, x_2 : \tau_2 \vdash_V v_3 : \tau_3]\!]]$$
$$\circ\, dist_{([\![\Gamma]\!],[\![\tau_1]\!],[\![\tau_2]\!])} \circ \langle id_{[\![\Gamma]\!]}, [\![\Gamma \vdash_V v_1 : \tau_1 + \tau_2]\!] \rangle$$

(c) Values

$$[\![\Gamma \vdash_C c : \tau]\!] : \mathrm{Hom}_{\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}}([\![\Gamma]\!], \mathcal{T}([\![\tau]\!]))$$
$$[\![\Gamma \vdash_C \mathtt{return}\, v : \tau]\!] = \eta^{\mathcal{T}}_{[\![\tau]\!]} \circ [\![\Gamma \vdash_V v : \tau]\!]$$
$$[\![\Gamma \vdash_C \mathtt{let}\, x = c_1\, \mathtt{in}\, c_2 : \tau_2]\!] =$$
$$\mu^{\mathcal{T}}_{[\![\tau_2]\!]} \circ \mathcal{T}([\![\Gamma;\, x : \tau_1 \vdash_C c_2 : \tau_2]\!]) \circ t^{\mathcal{T}}_{[\![\Gamma]\!],[\![\tau_1]\!]} \circ \langle id_{[\![\Gamma]\!]}, [\![\Gamma \vdash_C c_1 : \tau_1]\!] \rangle$$
$$[\![\Gamma \vdash_C v_1\, v_2 : \tau_2]\!] = ev_{([\![\tau_1]\!],[\![\tau_2]\!])} \circ \langle [\![\Gamma \vdash_V v_1 : \tau_1 \to \tau_2]\!], [\![\Gamma \vdash_V v_2 : \tau_1]\!] \rangle$$

(d) Computations

Where:

- $\pi_{1(A,B)}$ and $\pi_{2(A,B)}$ are the projections for $A \times B$
- $!_A$ is the unique morphism from A to the terminal object 1
- $\iota_{1(A,B)}$ and $\iota_{2(A,B)}$ are the injections for $A + B$
- $[m_1, m_2]$ is the copairing of m_1 and m_2
- $dist_{(A,B,C)}$ is the distributivity isomorphism from $A \times (B + C)$ to $(A \times B) + (A \times C)$
- $\lambda(m)$ is the currying of m
- $ev_{(A,B)}$ is the evaluation map for B^A
- $\eta^{\mathcal{T}}$, $\mu^{\mathcal{T}}$ and $t^{\mathcal{T}}$ are the unit, multiplication and strength of $\mathcal{T}$

Fig. 17. Denotational semantics: Basic constructs

$$[\![\tau]\!] : \widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$$
$$[\![\Box\,\tau]\!] = \Box[\![\tau]\!]$$

(a) Types

$$[\![\Gamma]\!] : \widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$$
$$[\![\Gamma;\, x :_\Box \tau]\!] = [\![\Gamma]\!] \times \Box[\![\tau]\!]$$

(b) Contexts

$$[\![\Gamma \vdash_V v : \tau]\!] : \mathrm{Hom}_{\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}}([\![\Gamma]\!], [\![\tau]\!])$$
$$[\![\Gamma_1;\, x :_\Box \tau;\, \Gamma_2 \vdash_V x : \tau]\!] = \epsilon^\Box_{[\![\tau]\!]} \circ \pi_{|\Gamma_1|}$$
$$[\![\Gamma \vdash_V \mathtt{box}\, v : \Box\,\tau]\!] = \Box[\![\Gamma/\Box \vdash_V v : \tau]\!] \circ \delta^\Box_{[\![\Gamma/\Box]\!]} \circ !_\Gamma/\Box$$
$$[\![\Gamma \vdash_V \mathtt{let}(\mathtt{box}\, x) = v_1 \,\mathtt{in}\, v_2 : \tau_2]\!] = [\![\Gamma;\, x :_\Box \tau_1 \vdash_V v_2 : \tau_2]\!] \circ \langle id_{[\![\Gamma]\!]}, [\![\Gamma \vdash_V v_1 : \Box\,\tau_1]\!]\rangle$$
$$\begin{aligned}[\![\Gamma \vdash_V \mathtt{case}^\Box\, v_1\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_2\,;\, \mathtt{in}_R\, x_2 \Rightarrow v_3\,\} : \tau_3]\!] = \\ [[\![\Gamma;\, x_1 :_\Box \tau_1 \vdash_V v_2 : \tau_3]\!], [\![\Gamma;\, x_2 :_\Box \tau_2 \vdash_V v_3 : \tau_3]\!]] \\ \circ\, dist_{([\![\Gamma]\!], \Box[\![\tau_1]\!], \Box[\![\tau_2]\!])} \circ \langle id_{[\![\Gamma]\!]}, d^\Box_{([\![\tau_1]\!], [\![\tau_2]\!])} \circ \Box[\![\Gamma/\Box \vdash_V v_1 : \tau_1 + \tau_2]\!] \circ \delta^\Box_{[\![\Gamma/\Box]\!]} \circ !_\Gamma/\Box\rangle\end{aligned}$$
$$[\![\Gamma \vdash_V \mathtt{run}(c) : \tau]\!] = \epsilon^\Box_{[\![\tau]\!]} \circ \rho_{[\![\tau]\!]} \circ \Box[\![\Gamma/\Box \vdash_C c : \tau]\!] \circ \delta^\Box_{[\![\Gamma/\Box]\!]} \circ !_\Gamma/\Box$$

(c) Values

Where:

- $\epsilon^\Box$ and $\delta^\Box$ are the counit and comultiplication of $\Box$
- $d^\Box_{(A,B)}$ is the distribution isomorphism from $\Box(A + B)$ to $(\Box\, A) + (\Box\, B)$
- $!_\Gamma/\Box : \Gamma \to \Gamma/\Box$ is the morphism that projects the global bindings from Γ
- ρ_P is the isomorphism from $\Box\,\mathcal{T}(P)$ to $\Box\, P$

Fig. 18. Denotational semantics: Locality

Lemma E.4.

$$[\![\Gamma/\Box \vdash_V v : \tau]\!] \circ !_\Gamma/\Box = [\![\Gamma \vdash_V v : \tau]\!]$$
$$[\![\Gamma/\Box \vdash_C c : \tau]\!] \circ !_\Gamma/\Box = [\![\Gamma \vdash_C c : \tau]\!]$$

Proof. By mutual induction on the typing derivations for v and c. We prove slightly stronger forms that allow additional bindings after $\Gamma/\Box$, so that we can use inductive hypotheses under additional bindings. The cases all follow from standard equalities about products and projections. □

E.2 Substitution

We will need lemmas showing that the semantics are compatible with substitution for local bindings:

Lemma E.5.

$$[\![\Gamma;\, x : \tau_2 \vdash_V v_1 : \tau_1]\!] \circ \langle id_{[\![\Gamma]\!]}, [\![\Gamma \vdash_V v_2 : \tau_2]\!]\rangle = [\![\Gamma \vdash_V v_1[x \backslash v_2] : \tau_1]\!]$$
$$[\![\Gamma;\, x : \tau_2 \vdash_C c : \tau_1]\!] \circ \langle id_{[\![\Gamma]\!]}, [\![\Gamma \vdash_V v : \tau_2]\!]\rangle = [\![\Gamma \vdash_C c[x \backslash v] : \tau_1]\!]$$

Proof. By mutual induction on the typing derivations for v_1 and c. We prove slightly stronger forms that allow additional bindings after x, so that we can use inductive hypotheses under additional bindings. The cases all follow from standard equalities about products and projections, along with the weakening lemmas (E.1, E.2, E.3) when moving under a binding. □

and similarly for global bindings.

$$[\![\tau]\!] : \widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$$
$$[\![W_\Sigma(\tau)]\!] = W_{[\![\Sigma]\!]}([\![\tau]\!])$$

(a) Types

$$[\![\Gamma \vdash_V v : \tau]\!] : \mathrm{Hom}_{\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}}([\![\Gamma]\!], [\![\tau]\!])$$
$$[\![\Gamma \vdash_V op(v_1, v_2) : W_\Sigma(\tau)]\!] =$$
$$op_{W_{[\![\Sigma]\!]}} \circ \langle \Box[\![\Gamma/\Box \vdash_V v_1 : \tau_p]\!] \circ \delta^\Box_{[\![\Gamma/\Box]\!]} \circ !_\Gamma/\Box, [\![\Gamma \vdash_V v_2 : \Box\, \tau_a \to W_\Sigma(\tau)]\!] \rangle$$
$$[\![\Gamma \vdash_V \mathtt{ret}(v) : W_\Sigma(\tau)]\!] = \mathtt{ret}_{W_{[\![\Sigma]\!]}} \circ \Box[\![\Gamma/\Box \vdash_V v : \tau]\!] \circ \delta^\Box_{[\![\Gamma/\Box]\!]} \circ !_\Gamma/\Box$$
$$[\![\Gamma \vdash_V \mathtt{rec}\, v_s\, \{\mathtt{ret}(x_r) \Rightarrow v_e\,;\, op_1(x_{p_1}, x_{k_1}) \Rightarrow v_1\,;\, \ldots\,;\, op_n(x_{p_n}, x_{k_n}) \Rightarrow v_n\} : \tau_2]\!] =$$
$$fold_{([\![\Sigma]\!],[\![\tau_1]\!])}([[\![\Gamma;\, x_r :_\Box \tau_1 \vdash_V v_e : \tau_2]\!],$$
$$[\![\Gamma;\, x_{p_1} :_\Box \tau_{p_1};\, x_{k_1} : \Box\, \tau_{a_1} \to \tau_2 \vdash_V v_1 : \tau_2]\!], \ldots,$$
$$[\![\Gamma;\, x_{p_n} :_\Box \tau_{p_n};\, x_{k_n} : \Box\, \tau_{a_n} \to \tau_2 \vdash_V v_n : \tau_2]\!]])$$
$$\circ \langle id_{[\![\Gamma]\!]}, [\![\Gamma \vdash_V v_s : W_\Sigma(\tau_1)]\!] \rangle$$

(b) Values

Where:

- $[\![\Sigma]\!]$ is the κ-bounded Lawvere theory corresponding to the signature Σ
- $\mathtt{ret}_{W_L}$ and op_{W_L} are the components of the structure map of the initial algebra $W_L(P)$
- $fold_{(L,P)}(m)$ is the parameterized recursor of $W_L(P)$, given by initiality of $W_L(P)$, applied to m.

Fig. 19. Denotational semantics: Inductive types

Lemma E.6.

$$[\![\Gamma;\, x :_\Box \tau_2 \vdash_V v_1 : \tau_1]\!] \circ \langle id_{[\![\Gamma]\!]}, \Box[\![\Gamma/\Box \vdash_V v_2 : \tau_2]\!] \circ \delta^\Box_{[\![\Gamma/\Box]\!]} \circ !_\Gamma/\Box \rangle = [\![\Gamma \vdash_V v_1[x \backslash v_2] : \tau_1]\!]$$
$$[\![\Gamma;\, x :_\Box \tau_2 \vdash_C c : \tau_1]\!] \circ \langle id_{[\![\Gamma]\!]}, \Box[\![\Gamma/\Box \vdash_V v : \tau_2]\!] \circ \delta^\Box_{[\![\Gamma/\Box]\!]} \circ !_\Gamma/\Box \rangle = [\![\Gamma \vdash_C c[x \backslash v] : \tau_1]\!]$$

Proof. By mutual induction on the typing derivations for v_1 and c. We prove slightly stronger forms that allow additional bindings after x, so that we can use inductive hypotheses under additional bindings. The global variable case follows from the comonad laws of $\Box$ and weakening from $\Gamma/\Box$ to Γ (E.4). The other cases all follow from standard equalities about products and projections, along with the weakening lemmas (E.1, E.2, E.3) when moving under a binding. □

E.3 Soundness

$$\Gamma \vdash_V v_1 : \tau \wedge v_1 \rightsquigarrow_V v_2 \implies [\![\Gamma \vdash_V v_1 : \tau]\!] = [\![\Gamma \vdash_V v_2 : \tau]\!]$$

Proof. By induction on the reduction relation. The congruence rules all follow immediately by the inductive hypothesis, leaving just the rules from Fig. 5.

Products For the product beta rules:

$$\pi_1(v_1, v_2) \rightsquigarrow_V v_1$$
$$\pi_2(v_1, v_2) \rightsquigarrow_V v_2$$

the cases follow from the universal property of products.

$$[\![\tau]\!] : \widehat{\mathrm{Law}_\kappa^{\mathrm{op}}} \qquad [\![\Gamma]\!] : \widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}$$
$$[\![\yo(\Sigma)]\!] = \yo([\![\Sigma]\!]) \qquad [\![\Gamma; \ell : \Sigma]\!] = [\![\Gamma]\!] \times \yo([\![\Sigma]\!])$$

(a) Types (b) Contexts

$$[\![\Gamma \vdash_V v : \tau]\!] : \mathrm{Hom}_{\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}}([\![\Gamma]\!], [\![\tau]\!])$$
$$[\![\Gamma_1; \ell : \Sigma; \Gamma_2 \vdash_V \ell : \yo(\Sigma)]\!] = \pi_{|\Gamma_1|}$$
$$[\![\Gamma \vdash_V \mathtt{out}_L\, v : \yo(\Sigma_1)]\!] = \yo(\iota_{1\,([\![\Sigma_1]\!],[\![\Sigma_2]\!])}) \circ [\![\Gamma \vdash_V v : \yo(\Sigma_1 + \Sigma_2)]\!]$$
$$[\![\Gamma \vdash_V \mathtt{out}_R\, v : \yo(\Sigma_2)]\!] = \yo(\iota_{2\,([\![\Sigma_1]\!],[\![\Sigma_2]\!])}) \circ [\![\Gamma \vdash_V v : \yo(\Sigma_1 + \Sigma_2)]\!]$$

(c) Values

$$[\![\Gamma \vdash_C c : \tau]\!] : \mathrm{Hom}_{\widehat{\mathrm{Law}_\kappa^{\mathrm{op}}}}([\![\Gamma]\!], \mathcal{T}([\![\tau]\!]))$$
$$[\![\Gamma \vdash_C \Uparrow(v_1(op))(v_2) : \Box\, \tau_a]\!] =$$
$$op_{\mathcal{T}} \circ \langle [\![\Gamma \vdash_V v_1 : \yo(\Sigma)]\!], \Box[\![\Gamma/\Box \vdash_V v_2 : \tau_p]\!] \circ \delta^{\Box}_{[\![\Gamma/\Box]\!]} \circ !_{\Gamma/\Box} \rangle$$
$$[\![\Gamma \vdash_C \Downarrow(\ell.\, c) : W_\Sigma(\tau)]\!] = \Downarrow_{\Sigma [\![\tau]\!]} \circ \lambda([\![\Gamma; \ell : \Sigma \vdash_C c : \Box\, \tau]\!])$$

(d) Computations

Where:

- $[\![\Sigma]\!]$ is the κ-bounded Lawvere theory corresponding to the signature Σ
- $\yo(\iota_{1\,(L_1,L_2)})$ and $\yo(\iota_{2\,(L_1,L_2)})$ are the Yoneda images of the injections in Law_κ
- $op_{\mathcal{T}}$ is the reflection morphism for op (Section 3.7)
- $\delta^{\Box}$ is the comultiplication of $\Box$
- $!_\Gamma/\Box$ is the morphism that projects the global bindings from Γ
- $\lambda(m)$ is the currying of m
- $\Downarrow_\Sigma$ is the reification isomorphism for Σ and τ (Section 3.9)

Fig. 20. Denotational semantics: Effect reflection

Sums For the sum rules:

$$\mathtt{case}^m(\mathtt{in}_L\, v_1)\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_2\, ;\, \mathtt{in}_R\, x_2 \Rightarrow v_3\, \} \rightsquigarrow_V v_2[x_1 \backslash v_1]$$
$$\mathtt{case}^m(\mathtt{in}_R\, v_1)\, \{\, \mathtt{in}_L\, x_1 \Rightarrow v_2\, ;\, \mathtt{in}_R\, x_2 \Rightarrow v_3\, \} \rightsquigarrow_V v_3[x_2 \backslash v_1]$$

the cases follow from the universal property of sums, their distribution over products and the substitution lemma (E.5).

Global modality For the global modality rule:

$$\mathtt{let}(\mathtt{box}\, x) = \mathtt{box}\, v_1\, \mathtt{in}\, v_2 \rightsquigarrow_V v_2[x \backslash v_1]$$

the case follows from the global substitution lemma (E.6).

For the run rule:

$$\mathtt{run}(\mathtt{return}\, v) \rightsquigarrow_V v$$

the case follows from the definition of ρ, the comonad laws of $\Box$ and weakening from $\Gamma/\Box$ to Γ (E.4).

Inductive types For the $\mathtt{rec}/\mathtt{ret}$ rule:

$$\mathtt{rec}(\mathtt{ret}(v_s))\,\{\mathtt{ret}(x) \Rightarrow v_e\,;\ldots\} \rightsquigarrow_V v_e[x\backslash v_s]$$

the case follows from the computation rule of the recursor for the $\mathtt{ret}_{W_L}$ component and the global substitution lemma (E.6).
For the $\mathtt{rec}/op$ rule:

$$\mathtt{rec}(op(v_1,\,v_2))\,\{H_1\,;\,op(x_1,x_2) \Rightarrow v_3\,;\,H_2\} \rightsquigarrow_V$$
$$v_3[x_1\backslash v_1][x_2\backslash \lambda x_a.\ \mathtt{let}\,x_w = v_2\,x_a\,\mathtt{in}\,\mathtt{return}\,(\mathtt{rec}\,x_w\,\{H_1\,;\,op(x_1,x_2) \Rightarrow v_3\,;\,H_2\})]$$

the case follows from the computation rule of the recursor for the op_{W_L} component, using the global substitution lemma (E.6) for x_1 and the local substitution lemma (E.5) for x_2. Unfolding the semantics of abstraction, let, application and return, the value substituted for x_2 denotes the recursive application of the recursor to the continuation, up to the monad laws of $\mathcal{T}$, standard equalities about products, exponentials and the strength, and weakening by the bindings of x_a and x_w (E.1, E.2).

Functions The case for the function beta rule:

$$(\lambda x.\,c)\,v \rightsquigarrow_C c[x\backslash v]$$

follows from the universal property of exponentials and the local substitution lemma (E.5).

Bind The case for the bind/return rule:

$$\mathtt{let}\,x = \mathtt{return}\,v\,\mathtt{in}\,c \rightsquigarrow_C c[x\backslash v]$$

follows from the unit laws of the monad $\mathcal{T}$, the unit axiom of its strength and the local substitution lemma (E.5).
The case for the bind commuting rule:

$$\mathtt{let}\,x_2 = \mathtt{let}\,x_1 = \Uparrow(\ell(op))(v)\,\mathtt{in}\,c_1\,\mathtt{in}\,c_2 \rightsquigarrow_C \mathtt{let}\,x_1 = \Uparrow(\ell(op))(v)\,\mathtt{in}\,\mathtt{let}\,x_2 = c_1\,\mathtt{in}\,c_2$$

follows from the associativity law of the monad $\mathcal{T}$, the axioms of its strength and the local weakening lemma (E.1) applied to c_2.

Reflection The cases for the $\mathtt{out}_L$ and $\mathtt{out}_R$ rules:

$$\Uparrow((\mathtt{out}_L\,v_1)(op))(v_2) \rightsquigarrow_C \Uparrow(v_1(\mathrm{left}(op)))(v_2)$$
$$\Uparrow((\mathtt{out}_R\,v_1)(op))(v_2) \rightsquigarrow_C \Uparrow(v_1(\mathrm{right}(op)))(v_2)$$

follow from the definition of the reflection morphisms (Section 3.7): precomposition with the Yoneda image of an injection maps the reflection of op to the reflection of $\mathrm{left}(op)$ (respectively $\mathrm{right}(op)$).

Reification The cases for the $\Downarrow$ rules:

$$\Downarrow(\ell.\ \mathtt{return}\,(\mathtt{box}\,v)) \rightsquigarrow_C \mathtt{return}\,\mathtt{ret}(v)$$
$$\Downarrow(\ell_1.\ \Uparrow(\ell_1(op))(v)) \rightsquigarrow_C \mathtt{return}\,op(v,\,\lambda(\mathtt{box}\,x).\ \mathtt{return}\,\mathtt{ret}(x))$$
$$\Downarrow(\ell_1.\ \mathtt{let}\,x = \Uparrow(\ell_1(op))(v)\,\mathtt{in}\,c) \rightsquigarrow_C \mathtt{return}\,op(v,\,\lambda x.\ \Downarrow(\ell_1.\,c))$$
$$\Downarrow(\ell_1.\ \Uparrow(\ell_2(op))(v)) \rightsquigarrow_C \mathtt{let}(\mathtt{box}\,x) = \Uparrow(\ell_2(op))(v)\,\mathtt{in}\,\mathtt{return}\,\mathtt{ret}(x) \qquad (\ell_1 \neq \ell_2)$$
$$\Downarrow(\ell_1.\ \mathtt{let}\,x = \Uparrow(\ell_2(op))(v)\,\mathtt{in}\,c) \rightsquigarrow_C \mathtt{let}\,x = \Uparrow(\ell_2(op))(v)\,\mathtt{in}\,\Downarrow(\ell_1.\,c) \qquad (\ell_1 \neq \ell_2)$$

follow from the definition of the reification isomorphism $\Downarrow_\Sigma$ (Section 3.9). The first rule corresponds to the $\mathtt{ret}_{W_{[\![\Sigma]\!]}}$ component of the isomorphism in Lemma 3.8, the second and third rules to its $op_{W_{[\![\Sigma]\!]}}$ components, and the final two rules to the operations of the ambient theory, which the isomorphism leaves in place.

□

value contexts, $C_V[\_]$ ::=

$$
\begin{aligned}
&\_ \mid \lambda x.\, C_C[\_] \mid (C_V[\_], v) \mid (v, C_V[\_]) \mid \pi_1\, C_V[\_] \mid \pi_2\, C_V[\_] \\
&\mid \mathtt{in}_L\, C_V[\_] \mid \mathtt{in}_R\, C_V[\_] \\
&\mid \mathtt{case}^m\, C_V[\_]\, \{\, \mathtt{in}_L\, x \Rightarrow v\, ;\, \mathtt{in}_R\, x \Rightarrow v\, \} \\
&\mid \mathtt{case}^m\, v\, \{\, \mathtt{in}_L\, x \Rightarrow C_V[\_]\, ;\, \mathtt{in}_R\, x \Rightarrow v\, \} \\
&\mid \mathtt{case}^m\, v\, \{\, \mathtt{in}_L\, x \Rightarrow v\, ;\, \mathtt{in}_R\, x \Rightarrow C_V[\_]\, \} \\
&\mid \mathtt{box}\, C_V[\_] \mid \mathtt{let}(\mathtt{box}\, x) = C_V[\_]\, \mathtt{in}\, v \mid \mathtt{let}(\mathtt{box}\, x) = v\, \mathtt{in}\, C_V[\_] \mid \mathtt{run}(C_C[\_]) \\
&\mid op(C_V[\_],\, v) \mid op(v,\, C_V[\_]) \mid \mathtt{ret}(C_V[\_]) \mid \mathtt{out}_L\, C_V[\_] \mid \mathtt{out}_R\, C_V[\_] \\
&\mid \mathtt{rec}\, C_V[\_]\, \{\mathtt{ret}(x) \Rightarrow v\, ;\, op(x,x) \Rightarrow v\, ;\, \ldots\, ;\, op(x,x) \Rightarrow v\} \\
&\mid \mathtt{rec}\, v\, \{\mathtt{ret}(x) \Rightarrow C_V[\_]\, ;\, op(x,x) \Rightarrow v\, ;\, \ldots\, ;\, op(x,x) \Rightarrow v\} \\
&\mid \mathtt{rec}\, v\, \{\mathtt{ret}(x) \Rightarrow v\, ;\, \ldots\, ;\, op(x,x) \Rightarrow C_V[\_]\, ;\, \ldots\, ;\, op(x,x) \Rightarrow v\}
\end{aligned}
$$

computation contexts, $C_C[\_]$ ::=

$$
\begin{aligned}
&\mathtt{return}\, C_V[\_] \mid \mathtt{let}\, x = C_C[\_]\, \mathtt{in}\, c \mid \mathtt{let}\, x = c\, \mathtt{in}\, C_C[\_] \\
&\mid C_V[\_]\, v \mid v\, C_V[\_] \mid \Uparrow(C_V[\_](op))(v) \mid \Uparrow(v(op))(C_V[\_]) \mid \Downarrow(\ell.\, C_C[\_])
\end{aligned}
$$

Fig. 21. Syntax of execution contexts

E.4 Contextual equivalence

In order to prove adequacy we need to provide a precise definition of contextual equivalence. The syntax of value and computation evaluation contexts is given in Fig. 21. We write $C_V[v]$ and $C_C[v]$ for the value and computation respectively obtained by replacing the _ in the evaluation context with v.

We write $C_V[\_] : \Gamma_1 \vdash_V \tau_1 \Rightarrow \Gamma_2 \vdash_V \tau_2$ and $C_C[\_] : \Gamma_1 \vdash_V \tau_1 \Rightarrow \Gamma_2 \vdash_C \tau_2$ for the mutually inductive context typing relations that indicate that if $\Gamma_1 \vdash_V v : \tau_1$ then $\Gamma_2 \vdash_V C_V[v] : \tau_2$ and $\Gamma_2 \vdash_C C_C[v] : \tau_2$ respectively.

Two values are contextually equivalent if they reduce to the same normal forms in closed boolean evaluation contexts.

$$
\begin{aligned}
\Gamma \vdash_V v_1 \cong_{\text{ctx}} v_2 : \tau \quad = \quad & \forall C_V[\_].C_V[\_] : \Gamma \vdash_V \tau \Rightarrow \vdash_V 1 + 1 \\
& \implies \quad \forall \omega.C_V[v_1] \rightsquigarrow_V^* \omega \iff C_V[v_2] \rightsquigarrow_V^* \omega
\end{aligned}
$$

E.5 Compositionality

The denotational semantics is compositional:

Lemma E.7.

$$
\begin{aligned}
&[\![\Gamma_1 \vdash_V v_1 : \tau_1]\!] = [\![\Gamma_1 \vdash_V v_2 : \tau_1]\!] \quad \wedge \quad C_V[\_] : \Gamma_1 \vdash_V \tau_1 \Rightarrow \Gamma_2 \vdash_V \tau_2 \quad \implies \quad [\![\Gamma_2 \vdash_V C_V[v_1] : \tau_2]\!] = [\![\Gamma_2 \vdash_V C_V \\
&[\![\Gamma_1 \vdash_V v_1 : \tau_1]\!] = [\![\Gamma_1 \vdash_V v_2 : \tau_1]\!] \quad \wedge \quad C_C[\_] : \Gamma_1 \vdash_V \tau_1 \Rightarrow \Gamma_2 \vdash_C \tau_2 \quad \implies \quad [\![\Gamma_2 \vdash_C C_C[v_1] : \tau_2]\!] = [\![\Gamma_2 \vdash_C C_C[
\end{aligned}
$$

Proof. By induction on the evaluation context typing rules. All the denotations only refer to subterms via the denotations of those subterms, so all the cases that correspond to typing rules go through directly using the induction hypothesis on the subterm that is an evaluation context. The other case is for the _ rule, which goes through by hypothesis. □

E.6 Adequacy

The denotational semantics is adequate with respect to the operational semantics.

$$[\![\vdash_V v_1 : \tau]\!] = [\![\vdash_V v_2 : \tau]\!] \implies \vdash_V v_1 \cong_{\text{ctx}} v_2 : \tau$$

Proof. Given a context $C_V[\_]$ such that $C_V[\_] :\vdash_V \tau \Rightarrow \vdash_V 1 + 1$, we know by confluence (2.5) and strong normalization (2.6) that there is a unique normal form ω_1 such that $C_V[v_1] \rightsquigarrow_V^* \omega_1$ and a unique normal form ω_2 such that $C_V[v_2] \rightsquigarrow_V^* \omega_2$. We know by subject reduction (2.4) that these normal forms are either $\texttt{in}_L()$ or $\texttt{in}_R()$. We also know by compositionality (E.7) and soundness (3.9) that $[\![\vdash_V \omega_1 : 1 + 1]\!] = [\![\vdash_V \omega_2 : 1 + 1]\!]$. Since we know that $[\![\vdash_V \texttt{in}_L()]\!] \neq [\![\vdash_V \texttt{in}_R()]\!]$ it must be the case that $\omega_1 = \omega_2$. □

F OXCAML IMPLEMENTATION

The implementation of our OxCaml effect reflection interface in terms of unchecked effects[1] is shown in Fig. 22. It assumes that handlers can close over local values to implement `reify` – but not `reify_global`. Unfortunately, this is not yet permitted by OxCaml due to the runtime not supporting pointers to stack allocated values from different fibres. If stack allocation were disabled then this code would be allowed.

[1]For convenience, we use the new handler syntax from OCaml 5.3. However, the most recently released version of OxCaml is based on OCaml 5.2 so the actual implementation uses `Effect.Deep.match_with` instead.

```
module Handler (E : Sig) = struct

  type t = { perform : 'r. 'r E.t -> 'r }

end

module Reflection (E : Sig) = struct
  open Term(E)
  open Handler(E)

  let reify f = exclave
    let module Eff =
      struct type 'a Effect.t += C : 'a E.t -> 'a Effect.t end
    in
    let handler = { perform = fun o -> Effect.perform (Eff.C o) } in
    match f handler with
    | v -> Return v
    | effect Eff.C o, k -> Perform(o, fun x -> Effect.continue k x)

  let reify_global f =
    let module Eff =
      struct type 'a Effect.t += C : 'a E.t -> 'a Effect.t end
    in
    let handler =
      { perform = fun o -> Effect.perform (Eff.C o) }
    in
    match f handler with
    | v -> Return v
    | effect Eff.C o, k -> Perform(o, fun x -> Effect.continue k x)

  let perform t =
   fun h -> h.perform t

end

module Select (L : Sig) (R : Sig) = struct
  open Sum(L)(R)

  let outl ({perform} : Handler(Sum(L)(R)).t) : Handler(L).t =
    exclave {perform = fun op -> perform (Left op)}

  let outr ({perform} : Handler(Sum(L)(R)).t) : Handler(R).t =
    exclave {perform = fun op -> perform (Right op)}

end
```

Fig. 22. Implementation